

Project Sentinel — Introduction & Design Discussion (start → Volume 01)

=====[USER]=====

jeg studerer pentester / ethical hacker. Jeg vil lage et helt system som kan teste nettsteder /systemer for vulnerability ,feil og alvorlig sikkerhetsbrudd. Målet er at det som bygges skal avdekke bruddene og feilene og foreslå hvordan den skal forbedres og tettes. Jeg vil først lage prototype slik at jeg vil få med alt med som kan brukes for å teste en nettside/system som : Passive Subdomain Enumeration, Subdomain Fuzzing (ffuf) osv(som på vedtalgt (tools.html), finner man noen av det verktøyene man bruker, men jeg vil ha fullstendig verktøy som jeg skal bruke i eget nettside system som lett skal teste nettsiden uten å gå ut,, bruke mest mulig egne tester som er utviklet og som man kan hente fra opensource også. Er det mulig å lage først prototypen ?

=====[ASSISTANT]=====

Ja - og etter min mening bør du gjøre dette som et **profesjonelt rammeverk**, ikke bare en samling verktøy.

Jeg har sett på HTML-filen du la ved. Den er en god start som et **Recon Hub**, men den er hovedsakelig en samling kommandoer og lenker. fileciteturn0file0L1-L40

Det du beskriver er noe helt annet:

En komplett webplattform som selv utfører testene, analyserer resultatene, korrelerer funnene, vurderer alvorlighetsgrad og genererer en profesjonell sikkerhetsrapport med forslag til utbedringer.

Dette er fullt mulig å lage.

Jeg ville bygget det som en modulær plattform

Ikke:

Klikk

Men:

Dashboard

Dette blir mer som:

- Pentest-Tools
- ProjectDiscovery Cloud
- Detectify
- Nessus

- Burp Enterprise

kombinert.

Prototype versjon (MVP)

Jeg ville startet med disse modulene.

Modul 1

Asset Discovery

Input:

example.com

Output:

Domains

Verktøy:

- subfinder
 - amass
 - assetfinder
 - findomain
 - dnsx
 - chaos
 - crt.sh API
 - SecurityTrails API
 - Censys
 - Shodan
 - Netlas
 - Wayback
-

Modul 2

Subdomain Enumeration

Passive

Active

Permutation

Fuzzing

Validation
Wildcard Detection
DNSSEC
Takeover Detection

Modul 3

HTTP Discovery
httpx
robots
sitemap
headers
cookies
redirect
compression
security headers
HSTS
CORS
CSP
cache
fingerprinting
favicon hash
TLS
HTTP2
HTTP3
ALPN

Modul 4

Technology Detection
Bygge egen fingerprint-database.
Oppdage
CMS
Framework
Backend
Frontend
Java
PHP
Node
Laravel

WordPress

Drupal

React

Vue

Angular

Apache

Nginx

IIS

Tomcat

Express

FastAPI

Spring

ASP.NET

osv.

Modul 5

URL Discovery

waybackurls

gau

gauplus

katana

hakrawler

gospider

waymore

egen crawler

robots.txt

sitemap.xml

RSS

API discovery

JS parser

Modul 6

Directory Discovery

Egen implementasjon inspirert av ffuf.

Støtte:

GET

Features
recursive
extensions
filtering
smart wordlists
adaptive rate
proxy
distributed scanning

Modul 7

Parameter Discovery
Arjun
ParamSpider
egen AI-basert parametergjetting
POST
GET
JSON
GraphQL
Multipart
XML
SOAP
REST

Modul 8

JavaScript Analysis
Dette kan bli en av de sterkeste modulene.
Finne:
Secrets
API Keys
JWT
Firebase
AWS
Azure
Google
Stripe
Github

Private endpoints
Internal APIs
Hidden routes
GraphQL
Swagger
OpenAPI
Hardcoded credentials
Regex
Entropy detection

Modul 9

API Discovery
Swagger
OpenAPI
GraphQL
REST
SOAP
gRPC
JSON
XML

Modul 10

Port Discovery
naabu
rustscan
masscan
nmap
banner grabbing
service fingerprint
TLS detection

Modul 11

Vulnerability Engine
Her kommer den store delen.

SQL Injection
egen tester
sqlmap integration

blind
time
boolean
stacked
union
error

XSS
Stored
Reflected
DOM
Mutation
SVG
Markdown
Polyglot

SSRF
Internal IP
Metadata
AWS
Azure
Google
Redis
Gopher
DNS

XXE

LFI

RFI

Path Traversal

Command Injection

Template Injection

Open Redirect

Host Header Injection

HTTP Request Smuggling

Cache Poisoning

Race Conditions

CSRF

CORS

Clickjacking

IDOR

JWT

OAuth

Session

Authentication

Authorization

Business Logic

Rate Limit

GraphQL

Prototype Pollution

Deserialization

File Upload

Backup Files

Secrets

Misconfiguration

Default Credentials

Cloud Exposure

Git Exposure

Docker Exposure

Kubernetes Exposure

CI/CD Exposure

S3 Buckets

Firebase

MongoDB

Elasticsearch

Redis

Jenkins

Grafana

Prometheus

Kibana

Admin panels

Development endpoints

Staging

Test systems

Debug endpoints

Modul 12

AI Analyzer

Her kommer det morsomme.

Etter alle testene.

AI leser ALT.

Den lager

Severity

Modul 13

Professional Report

Automatisk PDF.

Executive Summary

Technical Report

Evidence

Screenshots

Timeline

Assets

Risk Matrix

OWASP Mapping

CWE

MITRE

Recommendations

Jeg ville IKKE bare pakket inn eksisterende verktøy

Dette er den største feilen mange gjør.

Bruk dem som **motorer**, men bygg deres egen logikk rundt dem.

Eksempel:

Subfinder

Brukeren ser aldri at det var 8 verktøy.

Han ser bare:

Found

Frontend

Jeg ville laget et moderne dashboard med:

- Dashboard
- New Scan
- Live Scan Console
- Asset Inventory
- Vulnerabilities
- Screenshots
- Network Graph
- Attack Surface Map
- Timeline
- Reports
- Wordlists
- API Keys
- Scheduler
- Scan History
- Team Management

Backend

Jeg ville valgt:

- **Backend:** Python (FastAPI) for orkestrering og egne analysemoduler
- **Workers:** Celery eller RQ
- **Database:** PostgreSQL
- **Cache:** Redis
- **Frontend:** React + TypeScript
- **Sanntidsoppdateringer:** WebSockets
- **Containerisering:** Docker og Docker Compose (senere Kubernetes)
- **Rapporter:** PDF-generator med HTML-mal

Min anbefaling

Jeg tror prosjektet blir langt bedre dersom målet er å bygge en **egen pentest-plattform**, ikke bare et nettsted som kjører kommandoer. HTML-filen din viser allerede en god struktur for recon og verktøysamling, men den kan utvikles til en komplett plattform der de fleste analysene utføres av egne moduler, mens etablerte open

source-verktøy brukes der de gir en klar fordel. fileciteturn0file0L41-L200

Jeg vil anbefale å bygge dette trinnvis:

1. **Prototype (MVP):** Asset Discovery, HTTP Discovery, URL Discovery, JavaScript-analyse, grunnleggende sårbarhetsskanning og rapportgenerator.
2. **Versjon 2:** Egne analysemotorer, AI-korrelasjon, distribuerte skannere og planlagte skanninger.
3. **Versjon 3:** Teamfunksjoner, API, agentbaserte skannere, kontinuerlig overvåking og avansert angrepsflatekartlegging.

På denne måten får du et system som er både pedagogisk som student og skalerbart til et profesjonelt verktøy for autoriserte sikkerhetsvurderinger.

====[USER]=====

det er det jeg vil. Jeg vil bruke den som profesjonelt verktøy- Vi kan bygge prototypen med de 13 modellene du har foreslått og Frontend og backend slik at jeg har egen pentest plattform.

====[ASSISTANT]=====

Jeg synes dette er et veldig godt prosjekt. Hvis det bygges riktig, ender du ikke opp med en "GUI for ffuf", men en **egen profesjonell Attack Surface Management- og Pentest-plattform**. Målet bør være at 60-80 % av funksjonaliteten er egenutviklet, mens etablerte open source-verktøy brukes som motorer der det gir mening (f.eks. Nuclei, Katana og httpx), og resultatene normaliseres og analyseres av din egen plattform.

Jeg vil også legge inn ett viktig prinsipp fra starten:

Systemet skal kun brukes mot systemer du har eksplisitt tillatelse til å teste.

Det gjør at hele arkitekturen kan bygges for profesjonelle sikkerhetsrevisjoner, interne tester og bug bounty-programmer.

Prosjektnavn (arbeidsnavn)

Jeg ville gitt plattformen et eget navn.

Eksempel:

- Raven
- Sentinel
- ReconX
- BlackFalcon
- Nordic Pentest Platform
- Valkyrie
- Aegis
- Odin ASM
- Heimdall

Hovedmål

Ikke bare finne sårbarheter.

Plattformen skal:

- Kartlegge hele angrepsflaten

- Finne eksponerte systemer
 - Oppdage svakheter
 - Validere funn
 - Prioritere risiko
 - Forklare hvorfor noe er sårbart
 - Foreslå konkrete tiltak
 - Lage profesjonelle rapporter
 - Sammenligne skanninger over tid
 - Gi sanntidsdashboard
-

Arkitektur

Jeg ville delt hele systemet i mikrotjenester.

React Dashboard

De 13 motorene

Jeg ville utvidet dem litt.

1. Asset Discovery Engine

Ansvar:

- Domener
 - Subdomener
 - DNS
 - ASN
 - IP
 - CIDR
 - Cloud
 - MX
 - TXT
 - SPF
 - DMX
 - CAA
-

2. Recon Engine

- Passive
 - Active
 - OSINT
 - Certificate Transparency
 - Reverse DNS
 - WHOIS
 - ASN
-

3. Web Discovery Engine

- HTTP
 - HTTPS
 - Redirects
 - Robots
 - Sitemap
 - HTTP headers
 - Cookies
 - Security headers
 - TLS
-

4. URL Discovery Engine

- Katana
 - Wayback
 - Egen crawler
 - JavaScript parsing
 - Archive discovery
 - Hidden paths
-

5. JavaScript Intelligence

Dette blir en stor modul.

Parser:

- AST
 - Secrets
 - API Keys
 - Firebase
 - AWS
 - Azure
 - Google
 - Stripe
 - JWT
 - OAuth
 - Hidden APIs
-

6. API Intelligence

REST

SOAP

GraphQL

Swagger

OpenAPI

gRPC

7. Vulnerability Engine

Den største modulen.

Den skal bestå av kanskje 50–100 egne testere.

Eksempel:

SQL Injection

8. Infrastructure Engine

- Portscan
 - Services
 - Fingerprinting
 - SSL
 - SSH
 - FTP
 - SMB
 - Kubernetes
 - Docker
 - Jenkins
 - Grafana
-

9. Cloud Engine

AWS

Azure

Google Cloud

Firebase

S3

Blob

Buckets

IAM

CloudFront

10. AI Correlation Engine

Dette blir hjernen.

Den ser alle resultatene.

Eksempel:

Subdomain

11. Report Engine

Lager PDF.

HTML.

Markdown.

JSON.

CSV.

Executive Summary.

12. Dashboard

Realtime.

Charts.

Risk score.

Timeline.

Attack surface.

13. Knowledge Engine

Egen database.

OWASP

CWE

CVE

CVSS

MITRE

CAPEC

ExploitDB

ProjectDiscovery Templates

Database

Organizations

Frontend

React

TypeScript

Tailwind

React Query

Zustand

Monaco Editor

Chart.js

D3

Leaflet

xterm.js

Dashboard

Dashboard

Backend

Jeg ville brukt:

- FastAPI

- SQLAlchemy
 - PostgreSQL
 - Redis
 - Celery
 - Docker
 - Alembic
 - Pydantic
 - WebSockets
-

Mapper

backend/

Plugin-system

Dette blir viktig.

Da kan nye tester legges til uten å endre resten av systemet.

Eksempel:

plugins/

Hver plugin beskriver blant annet:

- hvilke typer mål den støtter

- hvilke forutsetninger som må være oppfylt
 - hvordan funn normaliseres
 - hvordan alvorlighetsgrad beregnes
 - hvilke utbedringsforslag som vises
-

Utviklingsplan

Jeg foreslår at vi bygger dette som et ekte programvareprosjekt, ikke som enkeltstående kodeeksempler.

Fase 1 - Fundament

- Monorepo med frontend og backend
- Docker Compose
- PostgreSQL og Redis
- Autentisering og roller
- Prosjekt- og målstyring
- Jobbkø og skanningsmotor
- Grunnleggende dashboard

Fase 2 - Discovery

- Asset Discovery Engine
- DNS Engine
- HTTP Engine
- URL Discovery
- Teknologideteksjon
- Resultatlagring og visualisering

Fase 3 - Analyse

- JavaScript Intelligence
- API Intelligence
- Vulnerability Engine (første sett med egne testere)
- AI Correlation Engine
- Rapporter og historikk

Fase 4 - Profesjonell versjon

- Plugin-system
- Distribuerte skanneagenter
- Planlagte skanninger
- Team- og organisasjonsstøtte
- Sammenligning av skanninger over tid
- Eksport til PDF/HTML/JSON
- REST API og webhook-støtte

Dette er et prosjekt som realistisk vil ende på flere hundre tusen linjer kode når det er modent, men det kan bygges inkrementelt. Jeg kan hjelpe deg gjennom hele prosessen - fra arkitektur og databaseskjema til API-er, frontend, plugin-system og de egne analysemodulene - slik at du sitter igjen med en profesjonell pentest-plattform i stedet for en samling skript.

=====[USER]=====

bra. Men kan du først lage prototypen og legge alt i en pdf fil eller html fil. slik at vi har alt og kan sjekke om vi mangler noe. Og alt skal bygges med fri kode som python osv slik at jeg trenger ikke å kjøpe noe. Jeg har en ubuntu server hos www.one.com

=====[ASSISTANT]=====

Ja. Det vi bør gjøre nå er å lage en **komplett prototype-spesifikasjon** før vi skriver kode. Da får du et dokument som fungerer som **blueprint for hele plattformen**, og vi kan kontrollere at ingenting mangler. Jeg har satt opp en profesjonell prototype med alle 13 motorene, frontend, backend, database, Docker-oppsett, API-er, sikkerhetsmodell, roadmap og Ubuntu-installasjon på one.com — **100 % basert på gratis og åpen kildekode**.

Dokumentet inneholder:

Alle 13 moduler

Frontend-design

Backend-design

Databaseskjema

Docker-oppsett

API-endepunkter

Plugin-system

Rapportgenerator

AI-analyse

Ubuntu/one.com installasjon

Utviklingsplan

Del | Teknologi

React DashboardProjects • Targets • Recon • Vulnerabilities • Reports

FastAPI GatewayAuth • API • Scheduler • WebSocket

Discovery Engine

DNS Engine

HTTP Engine

Vulnerability Engine

AI Analyzer

Report Engine

PostgreSQL

Redis

| Modul

1 | Asset Discovery

KommandoerUbuntu 22.04+sudo apt update && sudo apt upgrade -y

Dockercurl -fsSL https://get.docker.com | shsudo usermod -aG docker \$USER

Docker Composesudo apt install docker-compose-plugin -y

Start plattformengit clone your-repod platformdockerr docker compose up -d

- Legg inn et domene
 - Finn subdomener
 - Sjekk hvilke som er live
 - Hent URL-er
 - Analyser JavaScript
 - Kjør grunnleggende sårbarhetsskanning
 - Generer PDF-rapport
 - Vis alt i dashboard
-

Fase | Tid

Ja — dette kan bygges som en **100 % gratis, profesjonell pentest-plattform** på Ubuntu-serveren din hos one.com.Prototype-dokumentet over er nå et komplett **blueprint** som vi kan bruke til å begynne implementeringen modul for modul.

Deretter bør vi lage **Fase 1-koden**:

- Docker Compose
- FastAPI-backend
- React-frontend
- PostgreSQL-modeller
- Autentisering
- Første Discovery Engine

====[USER]====

hvilken alternativ er best?

====[ASSISTANT]====

Jeg ville valgt **en kombinasjon av begge**, men i riktig rekkefølge.

Alternativ 1 - Lage et stort spesifikasjonsdokument først

Fordeler:

- Du får en komplett plan.
- Enklere å oppdage manglende moduler.
- Bra som dokumentasjon.

Ulemper:

- Du har fortsatt ingen kjørbare programvare.
-

Alternativ 2 - Begynne å programmere med én gang

Fordeler:

- Du får raskt en fungerende prototype.
- Kan testes fortløpende.

Ulemper:

- Arkitekturen endrer seg ofte.
 - Risiko for å måtte skrive om mye kode senere.
-

Alternativ 3 (det jeg anbefaler)

Dette er slik profesjonelle programvareprosjekter vanligvis bygges:

Fase 1 - Design (ca. 150-300 sider)

Vi lager en komplett prosjektbok med blant annet:

- Overordnet arkitektur
- Alle 13 motorene
- Frontend-design
- Backend-design
- Database (ER-diagram)
- API-spesifikasjon
- Plugin-system
- Docker-oppsett
- AI-modul
- Rapportsystem
- Roadmap
- UML-diagrammer
- Skjermbilder/wireframes
- Utviklingsstandarder

Dette blir "masterplanen".

Fase 2 - Klikkbar prototype

Vi lager et frontend som ser ferdig ut, men hvor noen funksjoner bruker testdata.

Du kan navigere mellom:

- Dashboard
- Discovery
- Recon
- Vulnerabilities
- Reports
- Settings

Da kan vi se hvordan hele plattformen vil fungere før vi implementerer alle motorene.

Fase 3 - Backend

Vi bygger:

- FastAPI
 - PostgreSQL
 - Redis
 - Docker
 - API
 - Innlogging
 - Brukerstyring
-

Fase 4 - Motorene

Vi implementerer én motor om gangen:

1. Asset Discovery
 2. DNS
 3. HTTP
 4. URL Discovery
 5. JavaScript
 6. API
 7. Vulnerability Engine
 8. Cloud
 9. Infrastructure
 10. AI
 11. Reporting
 12. Dashboard
 13. Knowledge Engine
-

Fase 5 - Profesjonell versjon

Da har du en plattform som kan installeres på Ubuntu-serveren din hos one.com og brukes til autoriserte sikkerhetsvurderinger.

Det jeg foreslår

Jeg synes vi bør gjøre dette skikkelig og bygge noe som faktisk kan bli et profesjonelt produkt.

Jeg foreslår derfor at vi lager **tre dokumenter** før vi skriver den første kodelinjen:

1. **System Design Document (SDD)** - komplett teknisk spesifikasjon (ca. 200-300 sider når ferdig).
2. **Frontend Prototype** - en klikkbar HTML/React-prototype med alle menyer, sider og arbeidsflyter.
3. **Development Blueprint** - en detaljert implementeringsplan med mapper, database, API-er, Docker-oppsett og milepæler.

Når disse er på plass, kan vi begynne å kode med en stabil arkitektur. Da slipper vi å gjøre store omskrivninger underveis, og du får et prosjekt som er langt enklere å vedlikeholde og videreutvikle.

Jeg tror dette er den mest robuste veien hvis målet ditt er å ende opp med en profesjonell pentest-plattform som kan konkurrere med kommersielle løsninger over tid.

====[USER]====

ok da gjør vi som du foreslår.

====[ASSISTANT]====

Supert. Jeg tror dette blir den riktige måten å gjøre det på.

Det viktigste er at vi **ikke lager et skoleprosjekt**, men en plattform som faktisk kan brukes profesjonelt om 1-2 år. Derfor kommer vi til å tenke som et utviklingsteam fra første dag.

Visjonen

Målet er å bygge en plattform som kan konkurrere med løsninger som:

- Pentest-Tools
- ProjectDiscovery Cloud
- Detectify
- Burp Enterprise
- Nessus (på enkelte områder)
- OpenVAS
- Acunetix
- Invicti

...men med en moderne arkitektur og full kontroll over kildekoden.

Prosjektet får sin egen dokumentasjon

Vi bygger prosjektet som et ekte programvareprosjekt.

Pentest Platform

Dette blir prosjektets "bibel".

Leveranse 1

Den første leveransen blir en komplett **System Design Document (SDD)**.

Jeg ser for meg omtrent:

Del 1

Prosjektbeskrivelse

Visjon

Mål

Funksjoner

Bruksområder

Del 2

Komplett arkitektur

Diagrammer

Microservices

Docker

Queue

Workers

AI

Plugins

Del 3

Frontend
Alle sider
Dashboard
Dark mode
Responsive
Navigasjon
Komponenter

Del 4

Backend
FastAPI
Redis
PostgreSQL
Celery
Autentisering
RBAC
JWT
API

Del 5

Database
Alle tabeller
Relasjoner
ER-diagram
Indekser
Historikk

Del 6

Alle de 13 motorene
Asset Discovery
Recon
HTTP
JS
API
Cloud
Infrastructure

AI
Reporting
Knowledge
osv.

Del 7

Plugin-system
Hvordan nye tester installeres
Hvordan de registreres
Hvordan de oppdateres

Del 8

Vulnerability Engine
Dette blir den største delen.
Alle testene.
Hvordan de fungerer.
Hvordan de vurderes.
Hvordan de rapporteres.

Del 9

Rapportgenerator
PDF
HTML
Markdown
JSON
Executive Summary
Evidence
Screenshots

Del 10

Installasjon
Ubuntu
Docker
One.com
Backup
Oppdatering
CI/CD

Leveranse 2

Etter dokumentasjonen lager vi en **klikkbar prototype**.

Ikke bare HTML.

Den skal se ut som et ferdig produkt.

Du skal kunne klikke på:

Dashboard

Selv om alle motorene ikke er ferdige ennå.

Leveranse 3

Så begynner vi programmeringen.

Vi bygger én modul om gangen.

Hver modul testes.

Dokumenteres.

Integreres.

Viktig

Jeg ønsker også at vi bygger dette slik profesjonelle selskaper gjør.

Det betyr:

- Dokumentasjon **før** kode
 - Modulær arkitektur
 - Git fra første dag
 - Tester fra første dag
 - Docker fra første dag
 - API-dokumentasjon fra første dag
 - Automatisk installasjon
 - Automatisk oppdatering
 - Profesjonell kodebase
-

Min anbefaling

Jeg foreslår at vi kaller prosjektet **Project Sentinel** som arbeidsnavn frem til du

eventuelt velger et endelig navn.

Og jeg foreslår én ekstra regel som vil gjøre prosjektet mye bedre:

Vi skriver aldri kode uten at den først er beskrevet i designdokumentasjonen.

Det gjør at prosjektet holder en jevn kvalitet og blir mye enklere å vedlikeholde.

Jeg gleder meg til å bygge dette sammen med deg. Dette er et ambisiøst prosjekt, men fullt gjennomførbart hvis vi tar det steg for steg. Målet blir en komplett, egenutviklet pentest-plattform som kjører på din Ubuntu-server med utelukkende gratis og åpen kildekode.

====[USER]=====

ja, vi bare beskriver designdokumentasjonen / prototypen. Du kan starte med den nå.

====[ASSISTANT]=====

Perfekt. Da gjør vi dette på samme måte som et profesjonelt programvarefirma ville gjort. Vi skal **ikke skrive en eneste linje kode før hele systemet er definert**.

Jeg foreslår at vi lager en **Software Design Document (SDD)** som blir prosjektets "masterdokument". Dette dokumentet vil vokse etter hvert som vi tar beslutninger, og når det er ferdig, kan vi begynne implementeringen med minimal risiko for store omskrivninger.

Software Design Document (SDD)

Project Sentinel (arbeidsnavn)

Versjon 0.1 - Prototype Design

1. Prosjektoversikt

1.1 Visjon

Project Sentinel skal være en komplett, profesjonell plattform for autorisert sikkerhetstesting av nettsteder, webapplikasjoner, API-er og infrastruktur.

Plattformen skal kunne:

- Kartlegge hele angrepsflaten
- Oppdage sikkerhetsfeil
- Analysere risiko
- Validere funn
- Prioritere alvorlighetsgrad
- Foreslå konkrete tiltak
- Generere profesjonelle rapporter
- Historikk over tidligere skanninger
- Kontinuerlig overvåking av autoriserte mål

Målet er å bygge en modulær plattform basert på åpen kildekode og egenutviklede analysemoduler.

1.2 Mål

Plattformen skal kunne brukes av:

- Sikkerhetskonsulenter
 - Bedrifter
 - Interne sikkerhetsteam
 - Bug bounty-deltakere (innenfor programmets regler)
 - Studenter
 - Forskning
 - Autoriserte penetrasjonstestere
-

1.3 Designprinsipper

Prosjektet skal følge disse prinsippene:

- Modulær arkitektur
 - API-first
 - Docker-first
 - Plugin-basert
 - Asynkrone skanninger
 - Full historikk
 - Reproduserbare resultater
 - Skalerbarhet
 - Kun gratis og åpen kildekode
 - Egne analysemoduler der det gir merverdi
-

1.4 Teknologistakk

Backend

- Python
- FastAPI
- SQLAlchemy
- Alembic
- Celery
- Redis
- PostgreSQL

Frontend

- React
- TypeScript
- Tailwind CSS
- React Query
- Zustand

Infrastruktur

- Docker
- Docker Compose
- Nginx
- Ubuntu Server

- Git

Analyse

- Egne Python-moduler
 - Utvalgte open source-verktøy integrert via adaptere
 - Lokale AI-modeller (valgfritt, f.eks. Ollama)
-

1.5 Lisens

Målet er at hele plattformen skal kunne distribueres uten lisenskostnader.

Kun:

- MIT
- Apache 2.0
- BSD
- GPL-kompatible biblioteker der det passer

Ingen kommersielle avhengigheter skal være påkrevd.

2. Systemarkitektur

Plattformen deles inn i seks hovedlag.

Lag 1 - Brukergrensesnitt

- Dashboard
 - Prosjekter
 - Skanninger
 - Rapporter
 - Administrasjon
 - Innstillinger
-

Lag 2 - API

FastAPI

Ansvar:

- Autentisering
 - API
 - WebSocket
 - Jobbstyring
 - Brukere
 - Prosjekter
-

Lag 3 - Motorer

Her ligger all logikk.

Dette blir kjernen.

- Discovery Engine

- DNS Engine
 - HTTP Engine
 - URL Engine
 - JavaScript Engine
 - API Engine
 - Infrastructure Engine
 - Vulnerability Engine
 - AI Engine
 - Reporting Engine
 - Knowledge Engine
-

Lag 4 - Plugin-system

Alle testere skal kunne installeres som plugins.

Eksempel:

plugins/

Ingen endringer i hovedsystemet skal være nødvendige for å legge til nye tester.

Lag 5 - Datalagring

- PostgreSQL
 - Redis
 - Objektlagring for rapporter og skjermbilder
-

Lag 6 - Integrasjoner

Eksterne verktøy skal kobles til gjennom adaptere.

Eksempel:

Adapter

Brukeren skal aldri måtte forholde seg direkte til kommandolinjen.

3. Hovedfunksjoner

Den første versjonen skal bestå av 13 hovedmotorer.

1. Asset Discovery
 2. Recon
 3. Web Discovery
 4. URL Discovery
 5. JavaScript Intelligence
 6. API Intelligence
 7. Vulnerability Engine
 8. Infrastructure
 9. Cloud
 10. AI Correlation
 11. Reporting
 12. Dashboard
 13. Knowledge Base
-

4. Ikke-funksjonelle krav

Plattformen skal være:

- Skalerbar
 - Modulær
 - Testbar
 - Dokumentert
 - Utvidbar
 - Plattformuavhengig (Docker)
 - Rask
 - Stabil
-

5. Første milepæl (MVP)

MVP skal kunne:

- Opprette prosjekt
 - Registrere mål
 - Utføre Asset Discovery
 - Validere levende verter
 - Utføre URL Discovery
 - Analysere JavaScript
 - Kjøre grunnleggende sårbarhetsskanning
 - Generere PDF-rapport
 - Vise funn i dashboard
-

Neste kapittel

Jeg foreslår at **Kapittel 2** blir den mest detaljerte delen av dokumentasjonen: en komplett systemarkitektur med komponentdiagrammer, katalogstruktur, datastrømmer, databaseoversikt og hvordan de 13 motorene kommuniserer med hverandre.

Hvordan vi jobber videre

Jeg foreslår også en fast struktur for dokumentasjonen:

- **Del 1:** Visjon og overordnet arkitektur *(den vi har startet nå)*
- **Del 2:** Komplette systemarkitektur
- **Del 3:** Frontend-design (alle sider og komponenter)
- **Del 4:** Backend-design og API
- **Del 5:** Databasedesign (ER-diagram og tabeller)
- **Del 6:** Design av hver av de 13 motorene
- **Del 7:** Plugin-system
- **Del 8:** AI- og rapportmotor
- **Del 9:** Distribusjon, Docker og Ubuntu
- **Del 10:** Utviklingsplan og milepæler

Jeg anbefaler at vi behandler dette som et ekte ingeniørprosjekt. Når dokumentasjonen er ferdig, har vi et solid grunnlag for å bygge en profesjonell pentest-plattform steg for steg.

====[USER]=====

den er god. fortsett

====[ASSISTANT]=====

Flott. Da fortsetter vi som om dette var en profesjonell systemspesifikasjon. Fra nå av skriver vi dokumentasjonen på et nivå som gjør at en utvikler kan bygge systemet uten å måtte gjette.

SOFTWARE DESIGN DOCUMENT (SDD)

Project Sentinel

Versjon: 0.2 Prototype Design

DEL 2 - KOMPLETT SYSTEMARKITEKTUR

2.1 Arkitekturfilosofi

Project Sentinel skal bygges som en modulær, distribuert plattform hvor alle komponenter har ett tydelig ansvar.

Grunnprinsipper:

- Microservice-inspirert arkitektur
 - API-first
 - Event-basert kommunikasjon
 - Asynkrone jobber
 - Plugin-basert utvidelse
 - Løs kobling mellom moduler
 - Høy testbarhet
 - Skalerbarhet
 - Ingen kommersielle avhengigheter
-

2.2 Overordnet arkitektur

Internet

2.3 Systemets hovedkomponenter

1 Gateway

Ansvar:

- REST API
 - WebSocket
 - Login
 - JWT
 - API Keys
 - Brukerstyring
 - Rate limiting
 - Audit logging
-

2 Scheduler

Planlegger alle jobber.

Eksempel:

Mandag 02:00

3 Queue

Alle skanninger legges i en kø.

Ingen bruker skal måtte vente på lange operasjoner.

Eksempel:

Start Scan

4 Workers

Workers utfører selve analysene.

De skal kunne skaleres.

Eksempel

Worker 1

2.4 Dataflyt

Når brukeren starter en skanning.

Bruker

2.5 Scan Pipeline

Alle skanninger følger samme pipeline.

Input

Dette blir ryggraden i hele systemet.

2.6 Job Pipeline

Hver jobb består av mindre oppgaver.

Eksempel

Discovery

Hvis én modul feiler, skal resten kunne fortsette.

2.7 Scan State Machine

Alle skanninger skal ha en status.

Created

Mulige feil:

Timeout

2.8 Modulkommunikasjon

Ingen modul skal lese direkte fra en annen modul.

Alt går via databasen eller interne API-er.

Eksempel

Discovery

Dette gjør systemet robust.

2.9 Plugin-arkitektur

Alle tester blir plugins.

Plugin

Eksempel:

wordpress_plugin

2.10 Plugin Manifest

Hver plugin beskrives med metadata.

Eksempel:

Name

Da kan systemet installere plugins automatisk.

2.11 Adaptere

Eksterne programmer skal aldri kalles direkte.

Alt går gjennom adaptere.

Eksempel:

HTTPX Adapter

Samme for

Katana

Hvis et verktøy en dag byttes ut, trenger vi bare å endre adapteren.

2.12 Logging

Alt logges.

Typer:

Systemlogg

Auditlogg

Scanlogg

Pluginlogg

Errorlogg

API-logg

Performance-logg

2.13 Audit Trail

Alt brukeren gjør registreres.

Eksempel:

Dette blir viktig dersom plattformen senere brukes av flere personer eller organisasjoner.

2.14 Konfigurasjonssystem

Ingen hardkodede verdier.

All konfigurasjon skal ligge i:

config/

2.15 Filstruktur (målarkitektur)

project-sentinel/

2.16 Designmål

Prosjektet skal kunne vokse fra:

uten at arkitekturen må bygges om.

Oppsummering av Del 2

Etter denne delen har vi definert hvordan hele plattformen skal være bygget, hvordan modulene kommuniserer, hvordan skanninger flyter gjennom systemet, hvordan plugins integreres og hvordan prosjektet skal kunne skaleres.

Neste del: Del 3 - Frontend Design

I neste kapittel går vi helt ned på skjermnivå og designer hele brukeropplevelsen. Vi skal beskrive hver eneste side, panel, tabell og arbeidsflyt, slik at frontend kan bygges systematisk og konsistent. Dette inkluderer dashboard, prosjektoversikt, skanneveiviser, sanntidskonsoll, funnvisning, rapporter og administrasjon. Jeg vil også legge inn wireframes og komponenthierarki slik at vi får et komplett UI-design før vi begynner å kode.

=====[USER]=====

fortsett

=====[ASSISTANT]=====

Flott. Nå begynner vi å designe det som etter min mening kommer til å bli den viktigste delen av hele prosjektet: **Frontend**.

Grunnen er enkel: en pentester kommer til å tilbringe 90 % av tiden i brukergrensesnittet. Hvis dette er gjennomtenkt, blir resten av plattformen mye enklere å bruke.

SOFTWARE DESIGN DOCUMENT (SDD)

DEL 3 - FRONTEND DESIGN

3.1 Designfilosofi

Project Sentinel skal ikke se ut som en typisk Linux-applikasjon eller en samling verktøy. Den skal fremstå som et moderne sikkerhetsoperasjonssenter (SOC) med fokus på oversikt, effektivitet og minimal støy.

Designprinsipper:

- Dark Mode som standard
- Responsivt design
- Ingen popup-vinduer dersom et sidepanel kan brukes
- Sanntidsoppdateringer
- Tastaturnarveier

- Modulært komponentbibliotek
 - Konsistent ikonografi
 - Tilgjengelighet (WCAG)
-

3.2 Hovedlayout

3.3 Venstremeny

Den skal alltid være synlig.

Dashboard

Hver modul får sin egen underside.

3.4 Dashboard

Dette blir systemets kontrollsenter.

Øverst vises KPI-er.

Projects

Midten

Fire store kort.

Attack Surface

Nederst

Grafer

Vulnerabilities over time

3.5 Prosjekter

En organisasjon kan ha mange prosjekter.

Eksempel

Acme AS

Hvert prosjekt får egne:

- Targets
- Rapporter
- Historikk
- API Keys
- Skanninger

3.6 Target Manager

Dette blir en av de viktigste sidene.

Eksempel

example.com

Hvert target får sin egen detaljside.

3.7 Scan Wizard

Ny skanning skal være enkel.

Steg 1

Velg prosjekt.

↓

Steg 2

Velg mål.

↓

Steg 3

Velg moduler.

↓

Steg 4

Velg hastighet.

↓

Steg 5

Start.

Scan-moduler

Brukeren kan velge.

Discovery

3.8 Live Console

Dette blir en favorittside.

Sanntid.

12:15 Discovery started

Fargekoder

Grønn

Blå

Gul

Rød

3.9 Attack Surface

Dette blir plattformens signatur.

Hele angrepsflaten visualiseres.

Company

Senere kan dette bli en interaktiv graf.

3.10 Discovery

Viser resultatet av Discovery Engine.

Faner:

Domains

3.11 Web

Her vises

HTTP Headers

Cookies

Security Headers

Redirects

TLS
Compression
HTTP2
HTTP3
Fingerprint

3.12 URL Discovery

Lister alle URL-er.

Filter
JavaScript

3.13 JavaScript

Dette blir en stor side.

Faner
Files

Klikker man på en JS-fil, åpnes en kodeviser med syntaksfremheving og markerte funn.

3.14 API Explorer

REST
SOAP
GraphQL
Swagger
OpenAPI
Eksempel:
GET /api/users

Alle parametere vises.

3.15 Infrastructure

Lister

Hosts

3.16 Vulnerabilities

Den viktigste siden.

Tabell

Severity

Filter

Critical

3.17 Vulnerability Details

Klikker man på et funn.

Vises

Beskrivelse

CVSS

CWE
OWASP
MITRE
Payload
Response
Evidence
Screenshots
HTTP Request
HTTP Response
Impact
Recommendation
References
History

3.18 Reports

Liste
Executive

Hver rapport kan
Åpnes
Lastes ned
Sammenlignes

3.19 Knowledge Base

Intern wiki.
Eksempel
SQL Injection

Denne brukes også av AI-modulen for å gi konsistente forklaringer og anbefalinger.

3.20 Plugin Manager

Installerte plugins.

Installed

3.21 Wordlist Manager

Alle ordlister administreres fra ett sted.

Kategorier

DNS

Støtte for import, versjonering og tagging.

3.22 Settings

Systeminnstillinger.

General

3.23 Administrasjon

Kun administratorer.

Users

3.24 Designsystem

Vi skal lage et eget komponentbibliotek med gjenbrukbare elementer:

- Knapper
- Tabeller
- Diagrammer
- Kort (cards)
- Sidepaneler
- Faner
- Filtre
- Søkefelt
- Statusmerker (badges)
- Konsollvisning
- JSON-visning
- Diff-visning
- Kodeviser
- Tidslinje
- Notifikasjoner

Dette gjør at hele brukergrensesnittet får et enhetlig uttrykk og blir enklere å vedlikeholde.

Oppsummering av Del 3

Vi har nå definert hele brukeropplevelsen – fra dashboard og skanneveiviser til sårbarhetsvisning, rapporter og administrasjon. Frontenden er tenkt som et moderne kontrollsenter hvor brukeren kan navigere mellom prosjekter, følge skanninger i sanntid og analysere funn uten å måtte forholde seg til kommandolinjen.

Før vi går videre

Jeg vil foreslå én endring som jeg tror vil løfte prosjektet betydelig:

I stedet for å bygge en ren **pentest-plattform**, kan vi bygge en **Attack Surface Management (ASM) + Pentest Platform**.

Da får vi ikke bare manuelle skanninger, men også:

- kontinuerlig overvåking av autoriserte mål,
- varsling når nye subdomener eller eksponerte tjenester dukker opp,
- historikk over endringer i angrepsflaten,
- sammenligning av skanninger over tid,
- risikotrender og prioritering.

Denne utvidelsen passer svært godt inn i arkitekturen vi allerede har skissert og vil gjøre Project Sentinel til mer enn et skanneverktøy – det blir en plattform for kontinuerlig sikkerhetsstyring. Jeg mener derfor vi bør designe for dette fra starten av, selv om noen av funksjonene først implementeres i senere versjoner.

====[USER]====

bra, bygge en Attack Surface Management (ASM) + Pentest Platform. god ide, ja ,

kommer du med flere forslag bare legg til. fortsett

=====[ASSISTANT]=====

Dette blir enda bedre. Jeg ser nå at vi kan bygge noe som går lenger enn en tradisjonell pentest-plattform.

Jeg vil at vi bygger **Project Sentinel** som en **Cyber Security Platform**, hvor **ASM (Attack Surface Management)** er kjernen, og Pentest er én av flere motorer.

Det betyr at vi bygger for fremtiden. Pentest blir bare én modul. Senere kan vi legge til SOC-funksjoner, compliance, asset management, trusselovervåking og mye mer – uten å endre grunnarkitekturen.

Jeg kommer også til å foreslå ting underveis som jeg mener vil gjøre plattformen mer profesjonell. Hvert forslag skal være begrunnet, og vi kan alltid velge å utsette implementeringen til en senere versjon.

SOFTWARE DESIGN DOCUMENT

DEL 4 - KJERNEARKITEKTUR (Core Platform)

4.1 Plattformens filosofi

Den største feilen mange sikkerhetsverktøy gjør, er at de er bygget rundt verktøyene de bruker.

Jeg vil gjøre det motsatte.

Plattformen skal være bygget rundt **data**.

Verktøyene er bare sensorer.

Eksempel:

Subfinder

Brukeren skal aldri se hvilket verktøy som fant informasjonen.

Brukeren skal bare se **kunnskap**.

4.2 Den sentrale databasen

Dette er kanskje den viktigste avgjørelsen i hele prosjektet.

Jeg ønsker **ikke** at modulene sender data direkte til hverandre.

Alt skal lagres.

Eksempel

Discovery

Dette gir oss enorme fordeler.

Fordeler

Vi kan

- sammenligne gamle skanninger
- finne endringer
- oppdage nye subdomener
- se nye åpne porter
- oppdage nye API-er
- oppdage nye JavaScript-filer
- følge utviklingen over flere år

Dette gjør Project Sentinel til en ekte ASM-plattform.

4.3 Asset First

Jeg foreslår en regel.

Alt er et Asset.

Eksempel

Company

Dermed får vi ett felles objektmodell gjennom hele systemet.

4.4 Asset Graph

Dette blir en av de mest spennende modulene.

Alle assets kobles sammen.

Eksempel

Company

Senere kan dette vises som en interaktiv graf.

4.5 Historikk

Dette mener jeg nesten ingen open source-plattformer gjør godt nok.

Vi skal lagre ALT.

Eksempel

Mandag

api.example.com

Torsdag

api.example.com

Systemet skal automatisk oppdage endringen.

4.6 Snapshot Engine

Et nytt forslag.

Hver skanning lager et komplett snapshot.

Eksempel

Scan #45

Neste skanning sammenlignes automatisk.

4.7 Diff Engine

Denne blir utrolig nyttig.

Eksempel

Scan 20

Samme for

Subdomains

Headers

Cookies

Secrets

Open Ports

GraphQL

Swagger

API
JavaScript
Vulnerabilities

4.8 Risk Engine

Jeg ønsker ikke bare CVSS.
Vi lager vår egen modell.
Eksempel
CVSS

Dette gjør prioritering mer nyttig for virksomheter.

4.9 Findings Lifecycle

Et funn har en livssyklus.
Detected

Da kan plattformen brukes til reelle revisjoner.

4.10 Tagging

Alt kan tagges.
Eksempel
Production

Da kan vi filtrere enorme mengder data.

4.11 Notes

Pentestere skriver alltid notater.

Vi bygger dette inn.

Eksempel

Target

4.12 Screenshots

Alle interessante sider kan få skjermbilder.

Eksempel

Login

Disse lagres automatisk sammen med funnene.

4.13 Evidence Vault

Alt bevismateriale lagres.

Eksempel

HTTP Requests

Da blir rapportene mye sterkere.

4.14 Timeline

En tidslinje for hele prosjektet.

08:15

4.15 Attack Chain

Dette er en funksjon jeg virkelig ønsker.

AI kan koble sammen funn.

Eksempel

Subdomain

Ikke nødvendigvis en bekreftet utnyttelse, men en mulig angrepskjede som hjelper pentesteren med å prioritere videre undersøkelser.

4.16 Security Score

Vi lager vår egen sikkerhetsscore.

Eksempel

Attack Surface

Ikke bare en prosent.
En forståelig vurdering.

4.17 Baseline

En veldig viktig ASM-funksjon.
Første skanning blir referansen.
Alle fremtidige skanninger sammenlignes mot denne.
Eksempel
Ny port

4.18 Notification Engine

Ikke bare e-post.
Senere kan vi støtte:

- Slack
- Microsoft Teams
- Discord
- Matrix
- Signal (via gateway)
- Webhooks

Brukeren velger selv hvilke hendelser som skal gi varsler.

4.19 Sentinel AI

Dette blir ikke bare en "chatbot".
Jeg ser for meg en analyseassistent som kan:

- oppsummere en skanning,
- forklare hvorfor et funn er viktig,
- foreslå neste steg,

- peke på relaterte funn,
- skrive førsteutkast til rapporter,
- foreslå utbedringer med referanser til OWASP, CWE og CVE.

Den skal støtte pentesteren – ikke ta avgjørelser på egen hånd.

Ny idé – Modul 14: Compliance Engine

Jeg vil allerede nå reservere plass til en fjortende modul.

Den trenger ikke bygges i MVP, men arkitekturen bør støtte den.

Den kan senere sammenligne funn mot rammeverk som:

- OWASP ASVS
- OWASP Top 10
- CIS Controls
- ISO 27001
- NIST Cybersecurity Framework

Slik kan plattformen brukes både til tekniske tester og revisjoner.

Oppsummering

Jeg tror vi nå har tatt prosjektet fra å være "en pentest-plattform" til å bli en **Cyber Security Platform** med ASM som fundament. Det betyr at når vi senere legger til nye funksjoner – for eksempel compliance, overvåking eller trusselinformasjon – passer de naturlig inn i arkitekturen.

Jeg har også en idé som jeg mener kan bli en av plattformens største styrker, men den fortjener et eget kapittel: **Knowledge Graph og Asset Intelligence**. Der skal vi beskrive hvordan alle domener, IP-er, teknologier, sertifikater, API-er og funn kobles sammen til en kunnskapsgraf som gjør det mulig å finne sammenhenger som tradisjonelle skannere ofte overser. Jeg foreslår at dette blir **Del 5** i designdokumentet.

=====[USER]=====

Du kan allerede ha det i prototypen : legge til SOC-funksjoner, compliance, asset management, trusselovervåking, og Ny idé – Modul 14: Compliance Engine og støtte i Slack

=====[ASSISTANT]=====

Perfekt. Nå begynner dette å ligne en **plattform** og ikke bare et verktøy.

Jeg er også glad for at du ønsker å ta dette med fra starten. Selv om mange av disse funksjonene først implementeres i senere versjoner, er det mye enklere å **design** **arkitekturen for dem nå** enn å bygge dem inn senere.

Jeg vil komme med ett forslag til før vi fortsetter:

Jeg synes vi bør slutte å tenke "pentest-verktøy". Vi bør tenke **Cyber Security Operating Platform (CSOP)**.

Det betyr:

- ASM (Attack Surface Management)
- Pentest
- Vulnerability Management
- Asset Management

- Compliance
- Threat Intelligence
- Reporting
- AI
- SOC Dashboard

Pentest blir bare én av motorene.

Det er slik moderne plattformer utvikler seg.

SOFTWARE DESIGN DOCUMENT

DEL 5 - KNOWLEDGE GRAPH & ASSET INTELLIGENCE

5.1 Filosofi

Dette er etter min mening den viktigste delen av hele Project Sentinel.

Det er her vi skiller oss fra nesten alle open source-prosjekter.

De fleste skannere finner ting.

Vi skal forstå sammenhengen mellom dem.

Eksempel

En vanlig skanner viser

`api.example.com`

Project Sentinel skal forstå

Company

Det er dette som er Asset Intelligence.

5.2 Knowledge Graph

Alt blir noder.

Eksempel

Organization

Alt kobles sammen.

5.3 Relasjoner

Eksempel

Company

Dermed kan AI forstå hele infrastrukturen.

5.4 Asset Intelligence

Alle assets får metadata.

Eksempel

Asset ID

5.5 Asset Fingerprint

Alle assets får sin egen identitet.

Eksempel

Hostname

Da kan vi se når en server faktisk har endret seg.

5.6 Change Detection

Systemet skal automatisk finne endringer.

Eksempel

Apache

eller

React 17

eller

New GraphQL endpoint

eller

Removed CSP

Ingen manuell sammenligning.

5.7 Attack Surface Intelligence

Her begynner ASM-delen.

Systemet skal svare på spørsmål som:

"Hvilke API-er ble eksponert denne uken?"

"Hvilke nye subdomener kom?"

"Hvilke tjenester mangler TLS?"

"Hvilke WordPress-installasjoner bruker samme plugin?"

"Hvilke systemer eksponerer admin-paneler?"

5.8 Dependency Graph

Dette blir utrolig nyttig.

Eksempel

Application

Da ser vi umiddelbart hvilke systemer som rammes av en ny sårbarhet i et bibliotek.

5.9 Exposure Graph

AI skal forstå eksponering.

Eksempel

Internet

Ikke bare at port 443 er åpen.
Men hva den faktisk leder til.

5.10 Trust Graph

Eksempel
Certificate

Brukes til å oppdage sammenhenger.

5.11 Secret Graph

Eksempel
JavaScript

Da kan AI forstå hva en nøkkel faktisk gir tilgang til.

5.12 Identity Graph

Senere.
Login

Brukes ved autentiseringsanalyse.

5.13 Technology Graph

Alle teknologier kobles.

Eksempel

Apache

Da vet AI hele stacken.

5.14 Vulnerability Graph

Eksempel

Apache

Da blir rapportene langt bedre.

5.15 Threat Graph

Et nytt forslag.

Vi lager plass for trusselinformasjon.

Eksempel

CVE

Dette åpner for fremtidig integrasjon mot åpne trusselkilder.

5.16 Compliance Graph (Modul 14)

Denne modulen kobler tekniske funn mot etterlevelseskrav.

Støttede rammeverk (første versjon):

- OWASP ASVS
- OWASP Top 10
- CIS Controls
- NIST CSF
- ISO 27001
- PCI DSS (grunnleggende mapping)

Eksempel:

Missing Security Headers

Rapporter kan derfor genereres både for utviklere og revisjonsteam.

5.17 SOC-funksjoner (arkitektur klar fra dag én)

Selv om vi ikke bygger et fullverdig SOC i MVP, reserverer vi plass til følgende moduler:

- Security Dashboard
- Alert Center
- Incident Queue
- Case Management
- Asset Health
- IOC (Indicators of Compromise)
- MITRE ATT&CK Mapping
- Detection Rules
- Event Timeline

Alle hendelser fra skanningene skal kunne vises som sikkerhetshendelser.

5.18 Threat Intelligence

Vi bygger en egen Threat Intelligence-modul med støtte for åpne datakilder.

Planlagte funksjoner:

- Berikelse av CVE-er
- EPSS-score
- Kjente utnyttelser
- MITRE ATT&CK-teknikker
- IOC-er (IP, domener, hasher)
- Egen intern kunnskapsbase

Dette skal være modulært slik at nye kilder kan legges til senere.

5.19 Notification Hub

Varslingssystemet skal være leverandøruavhengig.

Første arkitektur støtter:

- E-post
- Slack
- Microsoft Teams
- Discord
- Matrix
- Signal (via gateway)
- Webhooks

Alle integrasjoner bygges som egne "notification providers", slik at nye kan legges til uten å endre kjernen.

Eksempel på hendelser:

- Ny kritisk sårbarhet
- Nytt subdomene oppdaget
- Ny åpen port
- Endring i TLS-sertifikat
- Rapport ferdig
- Planlagt skanning feilet

5.20 Asset Management

Vi legger inn en full Asset Management-modul.

Hvert asset får:

- Eier
- Ansvarlig team
- Miljø (Produksjon/Test/Utvikling)
- Kritikalitet
- Forretningsområde
- Kontaktperson
- Tags
- Historikk
- Livssyklus

Dette gjør at plattformen kan brukes som et sentralt register over organisasjonens digitale angrepsflate.

5.21 Fremtidige moduler (reservert i arkitekturen)

For å unngå store ombygginger senere reserverer vi plass til:

- Agent-baserte skannere for interne nettverk
- CI/CD-integrasjon (GitHub Actions, GitLab CI, Jenkins)
- SSO (OpenID Connect/SAML)
- Multi-tenancy for konsulentselskaper
- RBAC med granulære rettigheter
- Offline-agenter
- Mobilvennlig dashboard
- REST API og GraphQL API
- CLI-klient
- SDK for egne plugins

- Intern regelmotor (YAML-baserte regler)
 - Eksperimentell AI-assistent med lokal modell
-

Viktig designbeslutning

Jeg vil foreslå én beslutning som jeg mener kommer til å spare oss for enormt mye arbeid senere:

Alt i Project Sentinel skal beskrives som et objekt med et unikt ID-system og en hendelseshistorikk.

Det betyr at et domene, en IP-adresse, en URL, en JavaScript-fil, en API-endepunkt og en sårbarhet alle følger samme grunnmodell:

- Unik ID
- Metadata
- Relasjoner
- Historikk
- Tags
- Eier
- Tidsstempler
- Kilde
- Bevis
- Risiko

Det høres kanskje ut som en liten detalj, men det gjør hele plattformen konsistent og gjør det mulig å bygge avanserte funksjoner som historikk, grafer, AI-analyse og compliance uten spesialtilpasninger.

Neste del - Del 6: Datamodell og Database

Jeg foreslår at neste kapittel blir det mest detaljerte så langt: en fullstendig datamodell med alle tabeller, relasjoner, versjonering, historikktabeller, indekser og en gjennomarbeidet databasearkitektur. Dette blir fundamentet som alle de øvrige modulene bygger på. Jeg vil også legge inn støtte for fremtidige funksjoner fra starten, slik at databasen kan vokse uten store migreringer.

====[USER]====

OK vi slutter nå tenke pentest platform og ha den som "pentest-verktøy". Vi tenker nå Cyber Security Operating Platform (CSOP). Vi kan bygge den nå og ha den i prototypen : 5.17 SOC-funksjoner (arkitektur klar fra dag én)

Selv om vi ikke bygger et fullverdig SOC i MVP, reserverer vi plass til følgende moduler:

Security Dashboard

For å unngå store ombygginger senere reserverer vi plass til:

Agent-baserte skannere for interne nettverk

====[ASSISTANT]====

Jeg er helt enig. Nå mener jeg vi skal ta en viktig arkitekturbeslutning som kommer til å definere hele prosjektet.

Vi skal **ikke** bygge en Pentest Platform.

Vi skal **ikke** bygge en ASM Platform.

Vi skal bygge en **Cyber Security Operating Platform (CSOP)**.

Det betyr at når plattformen er ferdig, kan den brukes av:

- Pentestere
- SOC-analytikere
- Red Team
- Blue Team
- Purple Team
- DevSecOps
- Sikkerhetsansvarlige (CISO)
- Compliance-ansvarlige
- Driftsteam
- Konsulentselskaper
- MSSP-er (Managed Security Service Providers)

Pentest blir da **én av mange motorer** i plattformen.

Jeg vil derfor gjøre en stor endring i arkitekturen

Jeg ønsker å dele systemet i **seks store domener**, omtrent slik store kommersielle plattformer gjør.

Project Sentinel

Dette gjør at vi aldri trenger å bygge systemet om senere.

DEL 6

Domain Driven Design (DDD)

Jeg foreslår at vi bygger hele plattformen etter **Domain-Driven Design**.

Da deler vi hele systemet inn i egne domener.

DOMAIN 1

Identity Platform

Ansvar

Users

Organizations

Teams
Roles
RBAC
SSO
API Keys
OAuth
OIDC
SAML
Audit
Sessions
Devices
Notifications
Preferences

DOMAIN 2

Asset Intelligence Platform

Dette blir den største databasen.

Her lagres

Organizations

Projects

Domains

Subdomains

IPs

CIDR

ASN

Certificates

Cloud

Hosts

Ports

Services

URLs

Parameters

Headers

Cookies

JavaScript

Secrets

APIs

Repositories
Containers
Clusters
Assets
History
Relationships
Dette er hjertet.

DOMAIN 3

Discovery Platform

Motorene
Passive Discovery
DNS
Recon
HTTP
Crawler
JavaScript
API Discovery
Cloud Discovery
Infrastructure Discovery
Internal Discovery
Agent Discovery

DOMAIN 4

Security Assessment Platform

Dette blir dagens Pentest-del.
Men den blir bare én modul.
Her ligger
Vulnerability Engine
Configuration Analysis
Hardening
Misconfiguration
Secrets
Authentication
Authorization
Business Logic

Web
Cloud
API
Containers
Kubernetes
CI/CD
Supply Chain

DOMAIN 5

Security Operations Platform

Her bygger vi SOC allerede.
Ikke senere.

Moduler
Security Dashboard
Alert Center
Incident Queue
Case Management
Threat Hunting
IOC Manager
Detection Rules
MITRE Mapping
Security Timeline
Automation
Playbooks
Alert Correlation
Attack Timeline
Analyst Workspace
Evidence
Watchlists
Risk Heatmaps
Notifications

Dette betyr at hvis vi senere ønsker å lage en SIEM-modul eller integrere logger, passer det rett inn.

DOMAIN 6

Governance Platform

Compliance

Policies

Exceptions

Risk Register

Security Score

OWASP

NIST

ISO27001

PCI

GDPR

Reports

Audits

Business Risk

Executive Dashboard

Jeg ønsker også å innføre noe nytt

Dette tror jeg blir en av de største styrkene.

Object Engine

Du sa:

Alt skal være objekt.

Jeg er helt enig.

Så jeg vil lage en egen motor.

Ikke database.

En egen motor.

Eksempel

Asset

Alle moduler bruker denne.

Alle objekter får samme struktur

Eksempel

Object ID

Dermed spiller det ingen rolle om objektet er

IP

eller

Cookie

eller

JWT

eller

Sårbarhet.

Systemet behandler alt likt.

Event Engine

Dette blir en helt ny modul.

Alle endringer genererer hendelser.

Eksempel

New Domain

Dette er fundamentet for SOC.

Rule Engine

Her kommer noe som nesten ingen open source-plattformer har.

Alt styres av regler.

Eksempel

IF

Ingen hardkoding.

Kun YAML.

Automation Engine

Alle regler kan starte andre moduler.

Eksempel

Discovery

Dette gjør systemet "levende".

Workflow Engine

Jeg ønsker også å bygge inn arbeidsflyter.

Eksempel

Critical Finding

Dette gjør plattformen anvendelig for team.

Case Management

Dette bør bygges inn fra starten.

En sak kan inneholde

Assets

Findings

Evidence

Notes

Tasks

Timeline

Attachments

Reports

Incidents

Analysts

IOC Engine

Ikke bare IP.

IOC kan være

IP

Hash

Domain

URL

Email

JWT

Cookie

Certificate

Header

API Key

JA3

TLS Fingerprint

User Agent

File

Container

Image

Docker Registry

Git Repository

Alt blir IOC.

Detection Engine

Jeg foreslår at vi lager vårt eget detectionsystem.

Litt inspirert av Sigma.

Eksempel

Name

Senere kan AI generere regler automatisk.

AI Orchestrator

Jeg ønsker faktisk ikke bare én AI.

Jeg ønsker en AI-orkestrator.

Eksempel

Report AI

Da kan forskjellige modeller brukes til forskjellige oppgaver, enten lokale eller skybaserte, avhengig av brukerens valg.

Plugin Marketplace

Dette blir en av de største modulene.

Brukeren skal kunne installere

Ny Scanner

Ny Detection Rule

Ny Compliance Pack

Ny Wordlist

Ny Rapport

Ny Dashboard

Ny AI Prompt

Ny Integration

...uten å endre kjernen.

Min største idé hittil

Jeg tror faktisk vi kan gjøre Project Sentinel til noe enda større.

Ikke bare et sikkerhetsverktøy.

Men et **Cyber Security Operating System**.

Det betyr at hele plattformen styres av fire grunnmotorer:

1. **Object Engine** - alle data representeres som objekter med identitet, relasjoner og historikk.
2. **Event Engine** - alle endringer blir hendelser som kan spores, varsles og analyseres.
3. **Rule Engine** - all automatisering og logikk styres av deklarativer regler fremfor hardkodede arbeidsflyter.
4. **Workflow Engine** - komplekse prosesser som skanning, hendelseshåndtering og rapportering bygges som arbeidsflyter.

Alle øvrige moduler - ASM, pentest, SOC, compliance, rapportering og AI - blir klienter av disse fire kjernemotorene.

Et viktig tillegg jeg vil foreslå

Før vi går videre til databasen, synes jeg vi bør legge inn en **Enterprise Architecture**-del i dokumentasjonen. Der beskriver vi disse fire kjernemotorene, alle domene, hvordan de kommuniserer og hvilke designprinsipper som gjelder på tvers av hele systemet. Det blir prosjektets "grunnlov" og vil gjøre resten av designarbeidet betydelig enklere og mer konsistent. Jeg mener dette er et naturlig neste kapittel før vi går ned på tabell- og API-nivå.

====[USER]====

den kan legges inn nå : lage en SIEM-modul eller integrere logger . OK: Så jeg vil lage en egen motor. Ikke database .En egen motor. og gjør alle de idene og forslagene du har kommet med. OK. fortsett

====[ASSISTANT]====

Perfekt. Nå mener jeg vi tar den siste store arkitekturbeslutningen før vi begynner å designe databasen.

Jeg tror faktisk ikke vi skal bygge et "program".

Vi skal bygge et **Cyber Security Operating System (CSOS)**.

Navnet på produktet kan fortsatt være **Project Sentinel**, men teknisk sett blir det et operativsystem for cybersikkerhet.

Ny visjon

Project Sentinel

A Modular Cyber Security Operating Platform (CSOP)

Ikke et pentest-verktøy.

Ikke en ASM-plattform.

Ikke en SIEM.

Ikke en SOC.

Den skal kunne være alle disse samtidig.

Dette blir kjernen

Jeg tror hele systemet skal bestå av fire fundamentale motorer.

Disse fire skal ALLE andre moduler bruke.

CORE ENGINE

Deretter bygger vi resten oppå.

ENTERPRISE ARCHITECTURE

Project Sentinel

NY MODUL

SIEM Platform

Denne skal bygges inn allerede nå.
Ikke senere.

SIEM Platform består av

Log Collection

Log Parsing

Normalization

Correlation

Detection

Alerting

Timeline

IOC Matching

MITRE Mapping

Search

Rules

Analytics

Retention

Forensics

Log Sources

Systemet skal kunne ta imot logger fra

Linux

Windows

Apache

Nginx

IIS

Syslog

Firewall

Docker

Kubernetes

Traefik

Cloudflare

Azure

AWS

GitHub

GitLab
Jenkins
VPN
Proxy
DNS
EDR
Suricata
Zeek
Wazuh
Sigma
YARA
OpenTelemetry
REST API
Webhooks

Security Data Lake

Jeg ønsker faktisk ikke bare PostgreSQL.
Jeg ønsker en egen Security Data Lake.
Events

Dermed kan AI søke i ALT.

Object Engine

Dette blir kanskje den mest avanserte delen.
Ikke database.
En motor.
Den skal kunne gjøre
Create Object
Update Object
Delete Object
Merge Object
Split Object
Compare Object

Clone Object
Version Object
Archive Object
Restore Object
Link Object
Score Object
Classify Object
Tag Object
Search Object
AI Summary

Eksempel
Object

Event Engine

ALT blir events.

Eksempel
Object Created

Alt.

Rule Engine

Ingen hardkodede regler.

Kun YAML.

Eksempel

rule:

Workflow Engine

Alle prosesser beskrives som workflows.

Eksempel

Discovery

Ikke hardcoding.

Kun workflows.

AI Orchestrator

Jeg ønsker faktisk å bygge AI som en egen plattform.

Ikke én AI.

Men mange.

Eksempel

Report AI

Brukeren velger hvilke som er aktive.

Integration Platform

Dette blir enormt.

REST

Plugin Marketplace

Jeg ønsker at ALT skal være plugin.

Ikke bare scannerne.

Plugin-kategorier

Scanner

Visual Analytics Engine

Dette er en ny idé.

Vi bygger en egen visualiseringsmotor.

Den skal kunne lage

Attack Graph

Relationship Graph

Timeline

Heatmap

World Map

Cloud Topology

Network Graph

Risk Matrix

MITRE Navigator

IOC Graph

Kill Chain

Dependency Graph

Asset Tree

Certificate Tree

API Graph

Technology Graph

Digital Twin

Dette er kanskje den mest ambisiøse ideen.

Project Sentinel skal bygge en digital kopi av kundens digitale miljø.

Eksempel

Company

Når noe endrer seg, endrer Digital Twin seg.

Da kan AI svare på spørsmål som:

"Hvilke forretningskritiske systemer blir påvirket hvis denne sårbarheten utnyttes?"

Threat Intelligence Platform

Vi bygger en komplett modul.

Datakilder (kun åpne eller brukerens egne):

- CVE
- CWE
- CAPEC
- OWASP
- MITRE ATT&CK
- CISA KEV
- EPSS
- NVD
- Sigma-regler
- YARA-regler
- OpenCTI (integrasjon)
- MISP (integrasjon)

Modulen skal korrelere trusselinformasjon mot egne assets og funn.

Compliance Platform

Utvides med:

- ISO 27001
- ISO 27002
- NIST CSF
- NIST 800-53
- CIS Controls
- CIS Benchmarks
- OWASP ASVS
- OWASP Top 10
- PCI DSS
- GDPR (tekniske kontroller)
- DORA (arkitektur klar)

- NIS2 (arkitektur klar)

Det viktigste designprinsippet

Jeg vil innføre én regel som gjelder hele prosjektet:

Ingen modul får eie data. Moduler produserer og forbruker objekter og hendelser.

Det betyr:

- Discovery eier ikke subdomener.
- SIEM eier ikke logger.
- Pentest eier ikke sårbarheter.
- Compliance eier ikke kontroller.

Alt lagres som objekter i **Object Engine**, og alle endringer publiseres som hendelser via **Event Engine**. Dette gjør at AI, SOC, ASM, rapportering og automatisering alltid arbeider på det samme konsistente datagrunnlaget.

En siste idé som jeg mener kan bli plattformens største konkurransefortrinn

Jeg vil legge inn en femte kjernemotor allerede i prototypen:

Knowledge Engine

Mens Object Engine lagrer objekter og Event Engine lagrer hendelser, skal Knowledge Engine bygge og vedlikeholde en kontinuerlig oppdatert **kunnskapsgraf**. Den skal utlede nye relasjoner, beregne risiko, oppdage avhengigheter, forklare hvorfor en hendelse er viktig og gi AI et rikt kontekstgrunnlag. Med denne femte motoren får vi et system som ikke bare samler data, men som gradvis bygger opp organisatorisk kunnskap om hele den digitale angrepsflaten. Jeg mener dette vil være den viktigste forskjellen mellom Project Sentinel og de fleste eksisterende sikkerhetsplattformer.

====[USER]====

OK : Visual Analytics Engine. Dette er en ny idé. Vi bygger en egen visualiseringsmotor.
OK: Digital Twin . OK: Knowledge Engine . OK til nye ideer . kan systemer brukes på bug bounty også slik at jeg kan tjene penger også

====[ASSISTANT]====

Ja. Faktisk mener jeg dette bør være et **designmål fra dag én**.

Men jeg ville ikke bygget det som et "bug bounty-verktøy". Jeg ville bygget det som en **Cyber Security Operating Platform** som kan brukes i flere forskjellige arbeidsmoduser.

Da får du én plattform som kan brukes til:

- Autoriserte penetrasjonstester
- Attack Surface Management (ASM)
- Bedriftsrevisjoner
- Intern sikkerhet
- DevSecOps
- Purple Team
- Red Team

- **Bug bounty (innenfor programmets regler)**
- Forskning
- Compliance

Det gjør at plattformen blir mye mer verdifull.

Jeg vil faktisk legge til en helt ny modul

DOMAIN 7

Engagement Platform

Denne styrer **hva du har lov til å gjøre**.

Den blir ekstremt viktig.

Organization

Da kan du opprette ulike typer oppdrag.

Eksempel

Bedriftskunde

Company

Bug Bounty

Program

Da kan Project Sentinel automatisk begrense hvilke moduler som er tilgjengelige.
For eksempel kan destruktive tester deaktiveres hvis programmet ikke tillater dem.

Bug Bounty Workspace

Jeg synes faktisk dette bør være en egen arbeidsflate.

Dashboard

Program Database

Vi bygger en database over programmer.

Eksempel

Program

Dette kan vedlikeholdes manuelt eller importeres der det er tillatt.

Scope Engine

Dette blir en veldig viktig modul.

Ingen scanning skal starte før Scope Engine sier:

Allowed

Eksempel
example.com

eller
internal.example.com

Dette hjelper deg å holde deg innenfor autorisert omfang.

Recon Intelligence

Dette tror jeg kan bli en av plattformens sterkeste sider.

Vi lager en motor som korrelerer informasjon.

Eksempel
Subdomain

Da slipper du å gå gjennom hundrevis av funn manuelt.

Opportunity Engine (ny idé)

Denne motoren finner **interessante kandidater**.

Ikke bare sårbarheter.

Eksempel
New Subdomain

Den rangerer hva som sannsynligvis er mest verdt å undersøke videre.

Bounty Score

I tillegg til Security Score foreslår jeg en egen poengsum.

Eksempel:

Interestingness

Dette er **ikke** en garanti for en belønning, men en prioriteringshjelp.

Recon AI

En egen AI-agent.

Den kan svare på spørsmål som:

"Hvilke subdomener bør jeg undersøke først?"

eller

"Vis de mest interessante JavaScript-filene."

eller

"Hvilke API-er virker uferdige?"

Den foreslår neste steg, men du tar beslutningene.

Notebook

Alle profesjonelle pentestere skriver notater.

Vi bygger det inn.

Markdown.

Kodeblokker.

Skjermbilder.

Payloads.

HTTP Requests.

Tasks.

Checklists.

Alt knyttes til assets og saker.

Evidence Manager

Dette blir gull når du skal skrive rapport.

Alt lagres automatisk.

- HTTP Requests
- HTTP Responses
- Screenshots
- Headers
- Cookies
- Payloads
- Tidslinje
- Funn
- Notater

Da kan rapporter genereres mye raskere.

Samarbeid

Selv om du starter alene, bør vi designe for team.

Roller kan være:

- Administrator
 - Pentester
 - SOC-analytiker
 - Compliance
 - Kunde
 - Observer
-

Min største idé så langt

Jeg tror faktisk vi skal gjøre noe som ingen open source-plattform gjør fullt ut.

Vi bygger **modulære AI-agenter**.

Ikke én AI.

Men spesialiserte agenter.

Eksempel:

Recon Agent

Alle bruker den samme Knowledge Engine og Object Engine, men har ulike oppgaver. Det gjør systemet mer fleksibelt og enklere å forbedre over tid.

Når det gjelder å tjene penger

Ja, plattformen kan absolutt brukes som et arbeidsverktøy i **autoriserte** bug bounty-programmer og sikkerhetsoppdrag.

Det viktige er at Project Sentinel bygges med tydelige mekanismer for **scope-kontroll** og **engagement management**, slik at du holder deg innenfor reglene for hvert program eller oppdrag. Det beskytter både deg og kunden.

Jeg har også et strategisk forslag til prosjektet:

I stedet for å lage en plattform som bare **finner** sårbarheter, bør vi designe den for å **administrere hele sikkerhetsarbeidsflyten** – fra oppdagelse og analyse til dokumentasjon, utbedringsforslag og oppfølging. Da får du et verktøy som er nyttig både for bug bounty, konsulentoppdrag og virksomheter som ønsker kontinuerlig oversikt over sin digitale angrepsflate. Jeg tror det vil gjøre Project Sentinel langt mer verdifullt på sikt.

====[USER]====

Roller kan være god ide som kan gis full rettigheter og delvis rettigheter. Ok for den også: designe den for å administrere hele sikkerhetsarbeidsflyten og alle andre ideer og forslag du har kommet . Fortsett

====[ASSISTANT]====

Dette er faktisk punktet hvor jeg mener prosjektet går fra å være et avansert verktøy til å bli en **plattform som kan brukes av bedrifter, konsultentselskaper og MSSP-er**.

Og jeg kommer til å være litt "arkitekt" nå og tenke 5–10 år frem i tid.

Jeg kommer derfor til å legge til ideer selv om de ikke skal programmeres i første versjon. De skal **designes inn nå**, slik at vi aldri trenger å bygge om arkitekturen.

SOFTWARE DESIGN DOCUMENT

DEL 7 - ENTERPRISE ARCHITECTURE

Cyber Security Operating Platform (CSOP)

7.1 Designfilosofi

Project Sentinel skal være et **Cyber Security Operating Platform (CSOP)**.

Ikke et program.

Ikke et dashboard.

Ikke et scannerverktøy.

Det skal være et operativsystem for cybersikkerhet.

Alle moduler skal kunne arbeide sammen.

Alle data skal deles.

Alle hendelser skal kunne automatiseres.

Alle arbeidsflyter skal kunne styres.

7.2 De fem kjernemotorene

Jeg vil faktisk oppgradere arkitekturen.

Vi skal ha **fem** kjernemotorer.

Core Engine 1

Object Engine

Alt er objekter.

Eksempel

Company

Core Engine 2

Event Engine

Alt som skjer blir Events.

Eksempel

Asset Created

Core Engine 3

Knowledge Engine

Dette blir hjernen.

AI leser aldri databasen.

AI leser Knowledge Engine.

Knowledge Engine bygger

Graph

Relationships

Dependencies

Risk

History

Patterns

Correlations

Core Engine 4

Rule Engine

Alt styres av regler.

Ingen hardkoding.

Core Engine 5

Workflow Engine

Alle prosesser beskrives som workflows.

7.3 Plattformlag

Presentation Layer

7.4 Multi Organization

Dette må inn allerede nå.
En installasjon skal kunne ha
Organization

Eksempel
Acme

7.5 Multi Tenancy

Dette gjør at konsulentselskaper kan bruke systemet.
Eksempel
Customer A

Ingen data deles.

7.6 RBAC

Dette blir mye mer avansert.
Ikke bare

Admin

User

Viewer

Jeg ønsker

Platform Administrator

7.7 Permissions

Dette blir ikke roller.

Dette blir rettigheter.

Eksempel

Create Project

Da kan man lage egne roller.

7.8 Fine-Grained Permissions

Dette tror jeg nesten ingen Open Source-plattformer gjør godt.

Eksempel

View Asset

Da kan hver rolle tilpasses.

7.9 AI Roles

Ny idé.

AI får egne roller.

Eksempel

Report AI

Brukeren kan slå av eller på hver AI-agent.

7.10 Workspace

Ikke alle trenger samme skjerm.

Eksempel

SOC Workspace

Red Team Workspace

Blue Team Workspace

Compliance Workspace

Executive Workspace
Developer Workspace
Research Workspace
Bug Bounty Workspace

7.11 Dashboard Engine

Dashboards bygges som widgets.

Widget-eksempler

Critical Findings

Brukeren bygger sitt eget dashboard.

7.12 Security Workflow

Her ønsker jeg å tenke større.

Ikke bare scanning.

Eksempel

Discovery

Hele sikkerhetslivssyklusen.

7.13 Case Management

Jeg ønsker faktisk å bygge dette som Jira.

En Case inneholder

Assets

Tasks

Evidence

Timeline

Reports

Analysts

Comments

Files

Incidents

History

AI Summary

7.14 Task Engine

Ny modul.

Tasks

Review

Verify

Retest

Approval

Assign

Due Date

Reminder

Priority

Dependency

Automation

7.15 Notification Center

Alle varsler samles.

Ikke bare e-post.

7.16 Plugin Store

Dette blir kanskje den største modulen.

Ikke bare plugins.

Marketplace.

Eksempel

Scanner

7.17 License Manager

Selv om vi satser på fri programvare, reserverer vi plass for fremtidig støtte til kommersielle moduler dersom brukeren ønsker det.

Arkitekturen skal derfor støtte både åpne og proprietære tillegg uten at kjernen endres.

7.18 Backup Engine

Automatiske sikkerhetskopier.

Database

Evidence

Reports

Configuration

Plugins

Rules

7.19 Recovery Engine

Katastrofegjenoppretting.

Restore

Rollback

Snapshots

Object History

Event Replay

7.20 Version Engine

Dette blir utrolig nyttig.

Alt versjoneres.

Eksempel

Asset v1

Samme gjelder

Regler

Workflows

Rapporter

Compliance Packs

Dashboards

AI Prompts

Wordlists

7.21 API First

Jeg vil faktisk ha **tre API-er**.

REST API

GraphQL API

Internal Event API

Dermed kan andre systemer kobles til Project Sentinel uten spesialtilpasninger.

Min største idé hittil

Nå ser jeg et mønster som jeg tror kan bli plattformens største styrke.

Jeg vil innføre et konsept som jeg kaller **Security Objects**.

I stedet for å tenke:

- domene,
- IP,
- rapport,
- hendelse,
- sårbarhet,

skal alt behandles som en **Security Object**.

Et Security Object består alltid av:

- Global UUID
- Type
- Eier
- Organisasjon
- Prosjekt
- Klassifisering
- Kritikalitet
- Livssyklus
- Historikk
- Relasjoner
- Hendelser
- Bevis
- Risiko
- Compliance-status
- AI-oppsummering
- Tags
- Kommentarer
- Vedlegg
- Arbeidsoppgaver
- Godkjenninger
- Revisjonsspor

Det betyr at hele plattformen får **ett felles språk**. AI, SOC, ASM, Pentest, Compliance og Rapportering trenger ikke egne datamodeller – de arbeider alle med de samme Security Objects.

Min langsiktige visjon

Hvis vi fortsetter å designe på denne måten, tror jeg Project Sentinel kan bli noe ganske unikt: en plattform som dekker hele sikkerhetslivssyklusen – fra oppdagelse og analyse til hendelseshåndtering, compliance og kunnskapsforvaltning – uten å være låst til én bestemt type sikkerhetsarbeid. Det gir oss et svært solid fundament før vi begynner på databasen og implementeringen.

=====[USER]=====

bra. fortsett

=====[ASSISTANT]=====

Supert. Nå er vi ferdige med visjonen og Enterprise Architecture. Nå kommer vi til den delen som avgjør om Project Sentinel blir et hobbyprosjekt eller en profesjonell plattform.

Dette blir etter min mening den viktigste delen av hele dokumentasjonen.

SOFTWARE DESIGN DOCUMENT

DEL 8 - DATA PLATFORM & STORAGE ARCHITECTURE

Versjon 1.0 - Cyber Security Operating Platform (CSOP)

8.1 Filosofi

Jeg vil foreslå en grunnregel:

Ingen modul lagrer data direkte. All data går gjennom Data Platform.

Dette betyr at Discovery, ASM, SIEM, AI, Compliance, SOC og Reporting aldri skriver direkte til databasen.

I stedet bruker de Object Engine og Event Engine.

Scanner

Dette gjør hele systemet konsistent.

8.2 Storage Layer

Vi deler lagringen i spesialiserte lag.

Ingen database gjør alt.

8.3 PostgreSQL

PostgreSQL blir hoveddatabasen.

Den lagrer:

- brukere
- organisasjoner
- prosjekter
- assets
- funn
- incidents
- compliance

- rapporter
 - regler
 - workflows
 - konfigurasjon
-

8.4 Object Store

Dette er ikke filsystemet.

Dette er vår egen Object Database.

Eksempel

Object

Alle objekter lagres likt.

8.5 Event Store

Alle hendelser lagres permanent.

Eksempel

09:12

Dette gjør at vi kan spille av historien.

8.6 Knowledge Graph

Knowledge Engine bygger grafen.

Eksempel

Organization

Ingen relasjoner går tapt.

8.7 Evidence Store

Her lagres:

- skjermbilder
- PDF
- HTML
- HTTP requests
- HTTP responses
- JSON
- XML
- logger
- JavaScript
- sertifikater
- payloads
- vedlegg
- terminalutskrifter

Alt får SHA-256 hash og tidsstempel for å sikre integritet.

8.8 Search Engine

Hele plattformen skal være søkbar.

Man skal kunne søke på:

- domener
- CVE
- CWE
- IP
- ASN
- sertifikater
- API
- JWT
- cookies

- rapporter
- kommentarer
- AI-oppssummeringer
- hendelser
- IOC-er

Jeg foreslår å bruke **OpenSearch** som standard, med mulighet for andre søkemotorer senere.

8.9 Cache Layer

Redis brukes til:

- sesjoner
 - køer
 - sanntidsdashboards
 - API-cache
 - AI-cache
 - rate limiting
 - distribuerte låser
-

8.10 Object IDs

Alle objekter får globale UUID-er.

Eksempel:

ORG-xxxxxxx

Dermed blir objekter enkle å referere til på tvers av moduler.

8.11 Version Engine

Alt versjoneres.

Eksempel:

Asset v1

Det samme gjelder:

- Dashboards
- AI-prompts

- Regler
 - Workflows
 - Rapporter
 - Compliance-profiler
 - Wordlists
-

8.12 Time Travel

En funksjon jeg sjelden ser i sikkerhetsplattformer.

Brukeren kan se systemet slik det var:

- i går
- forrige uke
- forrige måned
- ved en bestemt revisjon

Dette gir stor verdi ved revisjoner og hendelsesanalyse.

8.13 Snapshot Engine

Hver skanning lager et komplett snapshot.

Eksempel:

Snapshot

Snapshot brukes til sammenligning og historikk.

8.14 Diff Engine

Diff Engine sammenligner to snapshots.

Eksempel:

Ny subdomain

Dette er kjernen i kontinuerlig Attack Surface Management.

8.15 Security Data Lake

Alle data kan lagres i et sentralt datalager.

Kategorier:

- Assets
- Events
- Logs
- Findings
- Evidence
- AI
- Compliance
- Threat Intel
- Telemetri

Dette muliggjør avansert analyse uten å belaste den operative databasen.

8.16 Audit Engine

Alt logges.

Eksempel:

Hvem

Dette er avgjørende for revisjon og sporbarhet.

8.17 Retention Policy

Alle datatyper kan ha egne regler.

Eksempel:

| Datatype | Standard

| Hendelser | 365 dager

8.18 Encryption

Alt sensitivt lagres kryptert.

- API-nøkler
- Passord
- Tokens
- Secrets
- SSH-nøkler
- OAuth-klienter
- Webhook-hemmeligheter

Nøkler håndteres av en egen **Secrets Vault**-modul.

8.19 Secrets Vault (ny kjernemodul)

I stedet for å lagre hemmeligheter direkte i databasen, bygger vi en egen modul.

Den håndterer:

- API Keys
- SSH Keys
- JWT Signing Keys
- OAuth Secrets
- TLS Private Keys
- Webhook Secrets
- AI API-nøkler
- Integrasjonsnøkler

Senere kan den integreres med eksterne hvelv hvis brukeren ønsker det.

8.20 Data Quality Engine (ny idé)

Dette blir en svært viktig motor.

Den skal automatisk:

- oppdage duplikater,
- slå sammen identiske objekter,
- validere data,
- beregne datakvalitet,
- markere usikre relasjoner,
- oppdage foreldet informasjon.

Eksempel:

sub.example.com

8.21 Data Provenance

Hvert datapunkt skal ha full opprinnelse.

Eksempel:

Object

Da vet vi alltid hvor informasjonen kommer fra.

8.22 Confidence Score

Ikke alle funn er like sikre.

Hvert objekt får en tillitsgrad.

Eksempel:

95 %

Dette hjelper AI og analytikere med prioritering.

Ny stor idé - Security Knowledge Fabric

Jeg vil legge til en sjette kjernemotor:

Security Knowledge Fabric

Den fungerer som et intelligent lag over alle de andre motorene og kombinerer:

- Object Engine
- Event Engine
- Knowledge Engine
- Threat Intelligence
- SIEM
- Compliance
- AI

- Digital Twin

Resultatet er en levende, kontinuerlig oppdatert modell av hele miljøet. Alle moduler leser og skriver via denne "stoffstrukturen", slik at nye sammenhenger oppdages automatisk og kan brukes i dashbord, rapporter, risikovurderinger og arbeidsflyter.

Min anbefaling for resten av prosjektet

Fra dette punktet foreslår jeg at vi går over fra høy-nivå arkitektur til en **implementeringsklar spesifikasjon**. Det betyr at de neste delene ikke bare beskriver idéene, men definerer:

1. Alle databaseobjekter og tabeller.
2. Alle API-endepunkter (REST og GraphQL).
3. Alle interne hendelsestyper.
4. Alle Security Objects.
5. Alle workflows.
6. Alle plugins og plugin-API-er.
7. Alle skannemotorer og deres ansvar.
8. Alle AI-agenter og hvordan de samarbeider.

Når den dokumentasjonen er ferdig, vil den være detaljert nok til at utviklingen kan deles opp i moduler og implementeres systematisk uten at arkitekturen må endres underveis. Det er den tilnærmingen jeg anbefaler for å bygge en plattform av denne størrelsen.

====[USER]=====

ok, fortsett

====[ASSISTANT]=====

Supert. Nå går vi over i det jeg kaller **Blueprint-fasen**. Fra nå av skal alt vi designer kunne implementeres direkte. Jeg kommer derfor til å tenke som en **Chief Software Architect**, ikke bare som en utvikler.

Jeg vil også innføre en ny regel for hele prosjektet:

Hvis en modul ikke kan skaleres til 100 millioner objekter, redesignes den.

Vi designer altså for et profesjonelt system fra første dag.

SOFTWARE DESIGN DOCUMENT

DEL 9 - MASTER COMPONENT ARCHITECTURE

Project Sentinel - Cyber Security Operating Platform (CSOP)

9.1 Hele systemet

Jeg ønsker at hele plattformen deles inn i omtrent 30 selvstendige tjenester (moduler). De kan kjøres som én applikasjon i MVP, men arkitekturen skal gjøre det enkelt å splitte dem til egne tjenester senere.

PROJECT SENTINEL

9.2 Modulklassifisering

Vi deler alle moduler i fire nivåer.

Tier 1 - Core

Disse må alltid eksistere.

- Object Engine
 - Event Engine
 - Knowledge Engine
 - Rule Engine
 - Workflow Engine
-

Tier 2 - Platform Services

- AI Orchestrator
 - Search
 - Scheduler
 - Notification
 - Integration
 - Plugin
 - Secrets Vault
 - Identity
 - Backup
 - Recovery
-

Tier 3 - Security Domains

- ASM
 - Discovery
 - Vulnerability
 - SOC
 - SIEM
 - Compliance
 - Threat Intelligence
 - Digital Twin
 - Asset Intelligence
-

Tier 4 - User Experience

- Dashboard
 - Reports
 - Visual Analytics
 - Notebook
 - Case Management
 - Tasks
 - Knowledge Base
-

9.3 Object Engine - detaljert ansvar

Denne motoren får kun ett ansvar:

Administrere Security Objects.

Operasjoner:

Create

Den skal aldri vite noe om pentest, SIEM eller compliance.

9.4 Event Engine

Alle hendelser beskrives likt.

Eksempel:

Event

Fordelen er at alle moduler kan abonnere på hendelser.

9.5 Message Bus

Jeg vil innføre en intern meldingsbuss.

Ingen modul kaller en annen direkte.

Eksempel:

Discovery

Dette gir løs kobling og bedre skalerbarhet.

9.6 Scheduler Engine

Alt skal kunne planlegges.

Eksempel:

- Daglige ASM-skanninger
 - Ukentlige rapporter
 - Månedlig compliance
 - Nattlig backup
 - Periodisk oppdatering av trusselinformasjon
 - Opprydding av gamle data
-

9.7 Queue Manager

Alle tunge oppgaver går i kø.

Eksempel:

Start Scan

Ingen blokkering i brukergrensesnittet.

9.8 Worker Pool

Vi lager en generell Worker Engine.

Workers kan utføre:

- Discovery
- DNS
- HTTP
- JS-analyse
- API-analyse
- Rapportgenerering
- AI-analyse
- Grafoppdatering
- Varsling

Dette gjør det enkelt å skalere.

9.9 Agent Platform

Dette bygger vi inn nå.

Ikke senere.

Agenttyper:

- Linux Agent
- Windows Agent
- Docker Agent

- Kubernetes Agent
- Cloud Agent
- Offline Agent
- Remote Agent

Agentene skal kunne registreres, autentiseres og oppdateres sentralt.

9.10 Integration Hub

Alle integrasjoner går via én plattform.

Eksempler:

- Slack
- Microsoft Teams
- Discord
- Matrix
- Signal
- Webhooks
- Syslog
- OpenTelemetry
- REST
- GraphQL
- SMTP
- LDAP
- OIDC
- SAML

Senere kan vi legge til flere uten å endre kjernen.

9.11 Plugin SDK

Alle plugins følger samme struktur.

Eksempel:

Plugin

Dette gjør at tredjepartsutviklere kan lage egne moduler.

9.12 Dashboard Engine

Dashboards er konfigurasjon – ikke kode.

En side bygges av widgets.

Eksempel:

Dashboard

Brukeren kan lagre flere dashboard-oppsett.

9.13 Visual Analytics Engine (utvidet)

Denne modulen skal bli en av plattformens største styrker.

Planlagte visualiseringer:

- Attack Surface Graph
- Knowledge Graph
- Digital Twin View
- Asset Tree
- Timeline
- IOC Graph
- MITRE ATT&CK Matrix
- Kill Chain
- Dependency Graph
- Cloud Topology
- API Relationship Map
- Certificate Chain
- Risk Heatmap
- Geografisk kart
- Service Map

Alt skal kunne filtreres og eksporteres.

9.14 Digital Twin (utvidet)

Digital Twin skal representere hele kundens miljø.

Ikke bare nettverket.

Objekter kan være:

- Organisasjon
- Avdeling
- Domene
- Cloud-abonnement
- Kubernetes-klynge
- Container
- VM
- API
- Sertifikat
- Bruker
- IAM-rolle

- Integrasjon
- Forretningskritisk system

Målet er at enhver endring i miljøet reflekteres i den digitale tvillingen.

9.15 AI Memory

Ny idé.

AI skal ikke bare svare.

Den skal huske kontekst innenfor prosjektet.

Eksempel:

Prosjekt

Dette gir mer konsistente analyser over tid.

9.16 AI Safety Layer

AI skal aldri kunne:

- starte skanninger,
- endre konfigurasjon,
- slette data,
- lukke saker,
- godkjenne risiko,

uten eksplisitt brukerautorisasjon eller en godkjent arbeidsflyt.

Dette er viktig både for sikkerhet og revisjon.

9.17 Observability Platform (ny kjernemodul)

Vi bygger inn støtte for å overvåke **selve Project Sentinel**.

Den skal vise:

- CPU
- Minne
- Disk
- Database
- Arbeidskøer
- Worker-status
- Agent-status
- API-ytelse

- Plugin-feil
- AI-køer
- Integrasjoner

Slik blir plattformen selv observerbar.

9.18 Enterprise Policy Engine (ny modul)

I tillegg til Rule Engine foreslår jeg en **Policy Engine**.

Forskjellen er:

- **Rule Engine** beskriver *hva som skal skje*.
- **Policy Engine** beskriver *hva som er tillatt*.

Eksempel:

- Maksimalt antall samtidige skanninger
- Hvilke brukere kan kjøre interne skanninger
- Hvilke moduler er tilgjengelige i bug bounty-modus
- Hvilke integrasjoner er tillatt
- Hvilke AI-modeller kan brukes

Dette gir sentral styring av hele plattformen.

Beslutning for resten av prosjektet

Fra dette punktet foreslår jeg at vi designer Project Sentinel som om vi faktisk skulle levere det kommersielt. Det innebærer blant annet:

1. Full database- og objektmodell.
2. Hendelsesmodell og meldingsbuss.
3. Plugin-API og SDK.
4. REST- og GraphQL-spesifikasjon.
5. AI-agentrammeverk.
6. Worker- og agentprotokoller.
7. Komplett modulkatalog (anslagsvis 50–70 moduler).
8. Full UI/UX-spesifikasjon med skjermbilder og flyt.
9. Installerings- og driftsarkitektur (Docker, Kubernetes og Ubuntu).
10. En implementeringsplan i faser (MVP → v1.0 → Enterprise).

Jeg har én siste idé som kan bli unik

Jeg ønsker å legge inn en sjette intelligent kjernemotor:

Reasoning Engine

Knowledge Engine lagrer sammenhenger. **Reasoning Engine** analyserer dem.

Den skal kunne:

- finne skjulte relasjoner mellom objekter,
- forklare hvorfor et funn er viktig,
- beregne sannsynlige angrepsveier,
- prioritere risiko ut fra kontekst,
- foreslå neste autoriserte analyse,

- generere beslutningsstøtte for analytikere.

Den skal **ikke** utføre handlinger på egen hånd, men fungere som en forklarings- og analysemotor som bruker informasjon fra Knowledge Engine, Threat Intelligence, Compliance og Digital Twin. Jeg tror dette kan bli en av de mest verdifulle delene av hele Project Sentinel.

====[USER]=====

ok, ta med de nye ideene også. fortsett

====[ASSISTANT]=====

Supert. Nå mener jeg vi har kommet til punktet hvor vi ikke lenger designer en applikasjon, men et **operativsystem for cybersikkerhet**.

Jeg kommer derfor til å tenke enda større enn de fleste kommersielle løsningene. Alt trenger ikke bygges i v1, men **alt skal være designet inn fra starten**, slik at vi aldri må bygge om arkitekturen.

Jeg kommer derfor til å fortsette å legge til nye ideer når jeg ser at de passer naturlig inn.

SOFTWARE DESIGN DOCUMENT

DEL 10 - CYBER SECURITY OPERATING SYSTEM (CSOS)

Operating Principles

10.1 Design Philosophy

Project Sentinel skal være et **Operating System**.

Ikke Windows.

Ikke Linux.

Men et operativsystem for cybersikkerhet.

Alt i systemet bygger på fire prinsipper.

Observe

Dette er OODA-loopen brukt innen sikkerhet.

10.2 Cyber Security Operating Model

Jeg ønsker å bygge hele systemet rundt en kontinuerlig sikkerhetsprosess.

Discovery

Dermed blir systemet selvlærende.

10.3 Living Knowledge Base

Ny idé.

Knowledge Base skal aldri være statisk.

Den oppdateres automatisk.

Eksempel

Ny CVE

10.4 Learning Engine

Jeg ønsker faktisk enda en motor.

Denne skal lære av historikken.

Den kan oppdage:

- hvilke typer funn som går igjen
- hvilke kunder som ofte har samme feil
- hvilke plugins som gir mest verdi

- hvilke regler som ofte utløses
- hvilke workflows som fungerer best

Dette er ikke maskinlæring i første versjon, men en analysemodul som bruker historiske data.

10.5 Security Ontology

Dette blir en av de mest avanserte modulene.

Vi bygger en egen ontologi.

Eksempel

Organization

Dette gjør AI langt mer presis.

10.6 Universal Object Schema

Alle objekter følger samme modell.

UUID

Dermed trenger vi aldri lage nye datamodeller.

10.7 Universal Search

Jeg vil at brukeren skal kunne skrive

Apache

og få

- Assets
- CVE
- Rapporter
- Hendelser
- Logger
- API
- AI-analyser
- Sertifikater
- Dashboards
- Incidents
- Compliance

Alt i ett søk.

10.8 Security Timeline

Ikke bare hendelser.

En komplett tidslinje.

Domain Created

10.9 Decision Engine (ny idé)

Dette blir søsteren til Reasoning Engine.

Reasoning analyserer.

Decision Engine foreslår.

Eksempel

Ny Critical CVE

Systemet foreslår prioritering, men brukeren bestemmer.

10.10 Recommendation Engine

AI skal ikke bare forklare.

Den skal komme med konkrete forbedringsforslag.

Eksempel

Mangler CSP

10.11 Architecture Engine

Ny idé.

Systemet skal automatisk bygge arkitekturdiagrammer.

Eksempel

Cloud

Dermed slipper brukeren å tegne selv.

10.12 Simulation Engine

Dette blir en fantastisk modul.

Vi kan simulere.

Eksempel

Hvis denne serveren kompromitteres

Digital Twin brukes her.

10.13 Risk Simulation

Eksempel

Ny CVE

Systemet beregner konsekvens.

10.14 Attack Path Engine

Dette tror jeg blir den mest populære modulen.

Den bruker

Knowledge Engine

Digital Twin

Threat Intelligence

Assets

IAM

API

Secrets

Nettverk

for å identifisere **potensielle angrepsveier**.

Den skal ikke utføre angrep, men visualisere hvordan ulike autoriserte funn kan henge sammen og hvor risikoen er størst.

10.15 Exposure Engine

ASM på neste nivå.

Den beregner

Internet Exposure

Ikke bare åpne porter.

10.16 Security Score Engine

Vi lager en egen algoritme.

Attack Surface

Dette blir Project Sentinels signatur.

10.17 Executive Engine

Ledere trenger ikke 10 000 assets.

De trenger beslutningsgrunnlag.

Dashboardet viser

- Total Risk

- Trend
 - Compliance
 - Mest kritiske prosjekter
 - Åpne hendelser
 - Risiko over tid
 - Business Impact
 - Prioriterte tiltak
-

10.18 Developer Experience (DX)

Utviklere skal også kunne bruke plattformen.

Vi bygger inn:

- REST API
 - GraphQL API
 - Python SDK
 - CLI
 - Webhooks
 - Plugin SDK
 - OpenAPI
 - YAML Workflows
 - YAML Rules
 - Terraform Provider (fremtid)
 - Ansible Collection (fremtid)
-

10.19 Enterprise App Store

Dette blir en stor modul.

Ikke bare plugins.

Hele løsninger.

Eksempel

Cloud Pack

Brukeren installerer funksjonalitet med ett klikk.

10.20 Federation Engine (ny idé)

Dette blir ekstremt kraftig.

Flere Project Sentinel-installasjoner kan samarbeide.

Eksempel

Data kan synkroniseres kontrollert mellom installasjoner.
Dette gjør plattformen egnet for store virksomheter og MSSP-er.

10.21 Local AI Platform

En av de viktigste designbeslutningene.

AI skal være modulær.

Brukeren kan velge:

- Lokal modell
- Skybasert modell
- Hybrid

Eksempel:

- Lokal modell for sensitiv analyse
- Skybasert modell for språk og rapporter

Ingen låsing til én leverandør.

10.22 Quantum-Ready Cryptography (arkitektur)

Selv om dette ligger frem i tid, reserverer vi plass til støtte for fremtidige kryptografiske standarder.

10.23 API Governance

Alle API-er som Project Sentinel eksponerer skal:

- være versjonerte,
 - dokumenteres med OpenAPI,
 - ha autentisering,
 - støtte rate limiting,
 - ha revisjonsspor,
 - kunne deaktiveres per organisasjon.
-

Min største idé så langt: Cyber Security Digital Brain

Jeg tror vi kan legge til én siste kjernemotor som binder alt sammen.

Digital Brain Engine

Denne motoren kombinerer:

- Knowledge Engine
- Reasoning Engine
- Decision Engine
- Learning Engine
- Threat Intelligence
- Compliance
- Digital Twin
- AI Orchestrator

Den utfører ikke handlinger på egen hånd, men fungerer som plattformens sentrale analyse- og beslutningsstøttelag. Målet er å gi analytikerne forklaringer, prioriteringer og kontekst – ikke automatiske avgjørelser.

Neste steg - vi går fra arkitektur til implementering

Jeg mener vi nå har definert en visjon som er større enn de fleste åpne sikkerhetsplattformer. Det naturlige neste steget er å gjøre dokumentasjonen **implementeringsklar**.

Jeg foreslår at vi bruker de neste delene til å spesifisere:

1. **Database Blueprint** – alle tabeller, relasjoner, indekser og historikk.
2. **Object Blueprint** – full definisjon av alle Security Objects.
3. **Event Blueprint** – alle hendelsestyper og meldingsformat.
4. **API Blueprint** – komplette REST- og GraphQL-spesifikasjoner.
5. **Plugin Blueprint** – SDK, livssyklus og sikkerhetsmodell.
6. **Worker & Agent Blueprint** – distribuerte skannere og arbeidskøer.
7. **Discovery Framework Blueprint** – alle autoriserte oppdagelsesmoduler.
8. **Assessment Framework Blueprint** – alle analysemoduler og hvordan de samarbeider.
9. **Frontend Blueprint** – full skjermdesign og navigasjon.
10. **Deployment Blueprint** – Ubuntu, Docker, Kubernetes, skalering, backup og høy tilgjengelighet.

På det tidspunktet vil vi ha en komplett teknisk spesifikasjon som kan brukes som grunnlag for selve utviklingen av Project Sentinel. Jeg vil også sørge for at alle funksjoner er utformet med tydelig scope-kontroll og autorisasjon, slik at plattformen støtter profesjonell og ansvarlig sikkerhetstesting.

====[USER]====

ok fortsett

====[ASSISTANT]====

Perfekt. Nå går vi over i det jeg mener blir **hjertet av hele Project Sentinel**.

Vi skal nå definere **Object Blueprint**. Dette blir den viktigste delen av hele plattformen.

Jeg vil faktisk gjøre noe som de færreste sikkerhetsplattformene gjør:

Alt i hele systemet blir et Security Object.

Ikke bare Assets.

Ikke bare Vulnerabilities.

ALT.

Dette betyr at AI, SIEM, SOC, ASM, Compliance, Reports, Threat Intelligence, Plugins og

Workflows bruker samme objektmodell.

SOFTWARE DESIGN DOCUMENT

DEL 11 — UNIVERSAL SECURITY OBJECT MODEL (USOM)

Versjon 1.0

11.1 Filosofi

Project Sentinel skal aldri spørre:

"Er dette en IP?"

eller

"Er dette en rapport?"

Systemet spør kun:

"Hvilken type Security Object er dette?"

Deretter behandles alle objekter likt.

11.2 Security Object Lifecycle

Alle objekter følger samme livssyklus.

Created

11.3 Global Object ID

Alle objekter får

Sentinel UUID

Objekt-ID endres aldri.

11.4 Universal Object Header

Alle objekter inneholder:

UUID

Dette gjelder ALLE objekter.

11.5 Universal Metadata

Alle objekter har metadata.

Eksempel

11.6 Universal Relationships

Alle objekter kan kobles sammen.

Eksempel

Domain

Dette bygges som en graf.

11.7 Relationship Types

Eksempel

owns

Dette gjør grafen ekstremt kraftig.

11.8 Object Categories

Jeg foreslår omtrent 50 hovedkategorier.

Identity

User

Infrastructure

Domain

Network

Host

Application

Website

Cloud

AWS

Kubernetes

Cluster

Development

Repository

Security

Finding

Threat Intelligence

IOC

Governance

Control

Platform

Dashboard

11.9 Object States

Alle objekter har tilstand.

Eksempel

Draft

11.10 Object History

Alt versjoneres.

Version 1

Ingen historikk slettes.

11.11 Object Evidence

Alle objekter kan ha bevis.

Eksempel

PDF

11.12 Object Risk

Alle objekter får risikoberegning.

Likelihood

11.13 Object Health

Ny idé.

Alle objekter får Health.

Eksempel

Certificate

11.14 Object Timeline

Alle objekter får egen tidslinje.

Eksempel

Created

11.15 Object AI

Alle objekter får AI-data.

Summary

11.16 Object Compliance

Objektet vet hvilke rammeverk det berører.

Eksempel

OWASP

11.17 Object Digital Twin

Alle objekter kan eksistere i Digital Twin.

Dermed kan vi visualisere hele infrastrukturen.

11.18 Object Knowledge Graph

Knowledge Engine kobler automatisk objekter.

Eksempel

Host

Dette blir svært verdifullt for analyse og prioritering.

11.19 Object Confidence Engine (ny idé)

Ikke alle data er like pålitelige.

Derfor får hvert objekt en **Confidence Score** basert på:

- Antall uavhengige kilder
- Kildenes kvalitet
- Tid siden siste verifisering
- Om funnet er manuelt bekreftet
- Om flere moduler er enige

Eksempel:

DNS + HTTP + TLS + Manuell verifisering

AI og automatisering skal kunne bruke denne scoren for å prioritere analyser.

11.20 Object Lineage Engine (ny kjernemodul)

Dette blir en funksjon jeg nesten aldri ser i sikkerhetsplattformer.

Alle objekter skal kunne spores tilbake til sin opprinnelse.

Eksempel:

Finding

Dette gjør revisjon, feilsøking og kvalitetssikring mye enklere.

Ny stor idé - Trust Engine

Jeg foreslår å legge til en ny plattformmodul:

Trust Engine

Denne modulen beregner en samlet **Trust Score** for hvert objekt og hver relasjon.

Den bruker blant annet:

- Confidence Score
- Datakvalitet
- Antall bekreftelser
- Datakildens pålitelighet
- Objektets alder
- Historiske endringer
- Manuell validering

Resultatet er at Project Sentinel ikke bare vet **hva** som finnes, men også **hvor mye det kan stole på informasjonen**. Dette vil være svært nyttig for AI, korrelasjon, prioritering og rapportering.

Neste kapittel

Etter Object Blueprint foreslår jeg at vi går videre til **Event Blueprint**.

Der skal vi definere:

- alle hendelsestyper,
- meldingsformat,
- Event Bus,
- Event Store,
- korrelasjons-ID-er,
- distribuerte workflows,
- sanntidsvarsling,
- SIEM-integrasjon,
- SOC-hendelser,
- AI-hendelser,
- plugin-hendelser,
- revisjonshendelser.

Dette blir fundamentet som binder sammen alle modulene i Project Sentinel og gjør plattformen hendelsesdrevet, skalerbar og klar for både sanntidsanalyse og fremtidige utvidelser.

====[USER]=====

bra, målet er at den blir avansert slik at den kan finne avvik/brudd/feil,sikkerhetshull på de store nettstedene som facebook, tesla osv fordi de tilbyr bug bounty. hvis systemet vårt klarer å finne feil hos de store nettstedene vil den gir bra omdømme til systemet.
Fortsett

====[ASSISTANT]=====

Jeg synes faktisk dette er den riktige ambisjonen, men jeg ville formulert målet litt annerledes.

Målet bør **ikke** være:

"Finne feil hos Facebook eller Tesla."

Målet bør være:

"Bygge verdens mest avanserte åpne Cyber Security Operating Platform for autorisert sikkerhetstesting."

Hvis vi lykkes med det, vil plattformen naturlig kunne brukes på autoriserte bug bounty-programmer (innenfor programmets regler og omfang), sikkerhetsrevisjoner og profesjonelle pentest-oppdrag. Det er også den retningen som gir prosjektet størst langsiktig verdi.

Jeg vil derfor legge til et nytt prinsipp

Enterprise Design Goal

Project Sentinel skal kunne analysere

DEL 12

Next Generation Security Engine

Nå kommer vi til den viktigste delen.

Vi skal ikke lage en scanner.

Vi skal lage en **Security Analysis Engine**.

Det er en enorm forskjell.

En scanner gjør:

↓

Det gjør alle.

Vi skal gjøre mye mer.

Security Analysis Pipeline

Discovery

Dette er grunnen til at plattformen kan bli mye mer presis enn en samling enkeltverktøy.

Multi Layer Analysis

Ingen tester alene.

Alle samarbeider.

DNS

Correlation Engine

Dette tror jeg blir den største forskjellen.

Eksempel

En vanlig scanner sier

Apache 2.4.48

Project Sentinel sier

Apache 2.4.48

Det er korrelasjon.

Recursive Discovery

Dette blir en egen motor.

Domain

Dermed stopper aldri Discovery etter første nivå.

Autonomous Discovery Graph

Ny idé.

Discovery bygger grafen mens den scanner.

Asset

Dermed vokser Digital Twin kontinuerlig.

Adaptive Scanning Engine

Dette tror jeg kan bli unikt.

Scanneren tilpasser seg.

Eksempel

WordPress

Scanneren endrer oppførsel basert på hva den finner.

Target Profile Engine

Etter noen sekunder bygger systemet en profil.

Eksempel

Linux

Alle andre mod

=====[USER]=====

Ok fortsett

=====[ASSISTANT]=====

Perfekt. Nå begynner vi å designe det som jeg mener kommer til å bli **Project Sentinels største konkurransefortrinn**.

De fleste verktøy i dag (Nmap, Nuclei, Amass, ffuf, Nikto, osv.) er **verktøy**.

Project Sentinel skal være en **intelligent analyseplattform**.

Det betyr at vi ikke skal konkurrere med ett verktøy.

Vi skal være plattformen som koordinerer, korrelerer, analyserer og lærer.

SOFTWARE DESIGN DOCUMENT

DEL 13 - INTELLIGENT SECURITY ANALYSIS FRAMEWORK (ISAF)

"Think before Scan"

13.1 Filosofi

Jeg vil innføre en regel som nesten ingen scannere følger.

Ikke scan først.

Tenk først.

En vanlig scanner gjør:

Target

Project Sentinel gjør:

Target

13.2 Target Intelligence Engine

Dette blir den første motoren.

Den skal bygge en profil før en eneste aktiv test starter.

Den samler blant annet:

DNS

ASN

Cloud

CDN

Certificates

WHOIS

HTTP

Headers

Cookies

Framework

JavaScript

API

OpenID

OAuth

Security Headers

TLS

Fingerprint

Robots

Sitemap

Favicons

Email Technologies

Third-party Services

Analytics

CSP

WebSockets

HTTP/2

HTTP/3

gRPC

GraphQL

OpenAPI

Swagger

SBOM hvis tilgjengelig

Deretter beregner AI:

Hvordan bør dette testes?

13.3 Strategy Engine

Ny motor.

Den bestemmer rekkefølgen.

Eksempel

GraphQL funnet

eller

React

13.4 Dynamic Module Loader

Scanneren laster kun relevante moduler.

Eksempel

WordPress

Laravel

↓

Laravel Modules

Queue Modules

Env Modules

Blade Modules

Dermed går scanning raskere.

13.5 Capability Engine

Alle moduler beskriver hva de kan.

Eksempel

Module

Strategy Engine velger riktige moduler.

13.6 Scanner Orchestrator

Dette blir dirigenten.

Den styrer

Workers

Queues

Plugins

AI

Knowledge Engine

Object Engine

Event Engine

Scheduler

Notifications

13.7 Scan Profiles

Vi bygger mange profiler.

Eksempel

Passive

Safe

Bug Bounty Safe

Compliance

OWASP

API

Cloud

Container

Quick

Deep

Continuous

Stealth

Verification

Retest

13.8 Smart Retry Engine

Normale scannere gir opp.

Project Sentinel prøver smartere.

Eksempel

403

Dette skal **alltid** holde seg innenfor autorisert og tillatt testing.

13.9 Verification Engine

Dette blir ekstremt viktig.

Ingen funn rapporteres direkte.

Alle funn må gjennom:

Found

Dermed reduseres false positives kraftig.

13.10 Confidence Engine

Confidence beregnes fra

Scanner

Plugin

Evidence

Correlation

AI

History

13.11 Risk Engine

CVSS alene er ikke nok.

Vi lager vår egen modell.

Risk

Resultatet blir en mer realistisk prioritering.

13.12 Finding Quality Engine

Ikke alle funn er gode.

Eksempel

Incomplete

13.13 Root Cause Engine

Ny idé.

Scanneren skal finne

ikke bare

feilen.

Den skal finne

årsaken.

Eksempel

Missing CSP

Dette gir mer nyttige rapporter.

13.14 Recommendation Engine v2

Anbefalingene skal være konkrete.

Eksempel

Systemet finner:

TLS 1.0 enabled

Rapporten skal da automatisk kunne vise:

- Hvorfor dette er et problem.
 - Hvilke standarder som påvirkes (f.eks. OWASP, CIS eller NIST).
 - Forslag til sikker konfigurasjon.
 - Eksempler for relevante webservere.
 - Hvordan endringen kan verifiseres etterpå.
-

13.15 Knowledge Correlation Engine

Dette tror jeg blir en av de største forskjellene.

Eksempel

Exposed Git

Én observasjon er kanskje ikke alvorlig.

Fem observasjoner sammen kan være kritiske.

13.16 Attack Chain Engine

Dette blir en av plattformens mest avanserte moduler.

Den skal **ikke** utføre angrep.

Den skal analysere om flere autoriserte funn kan kombineres til en potensiell angrepskjede.

Eksempel:

Open Redirect

Dette gir analytikeren bedre forståelse av hvordan flere svakheter kan henge sammen.

13.17 Business Context Engine

Et stort problem med dagens verktøy er at de ikke kjenner virksomheten.

Vi bygger derfor inn forretningskontekst.

Eksempel

Host

Samme tekniske funn kan få ulik prioritet avhengig av kontekst.

13.18 Adaptive Learning Engine

Denne motoren forbedrer plattformen over tid.

Den kan blant annet lære:

- hvilke regler som ofte gir falske positive,
- hvilke analyser som gir høy verdi,
- hvilke arbeidsflyter som brukes mest,
- hvilke typer anbefalinger som oftest fører til utbedring.

Den skal foreslå forbedringer, men endringer skal alltid godkjennes av brukeren.

13.19 Security Benchmark Engine

Vi bygger en modul som kan sammenligne en organisasjon mot egne historiske data og valgte rammeverk.

Eksempler:

- Utvikling i Security Score over tid.
 - Endring i Attack Surface.
 - Endring i antall høyrisikofunn.
 - Compliance-utvikling.
 - Gjennomsnittlig tid til utbedring (MTTR).
-

13.20 Enterprise Knowledge Packs

Jeg ønsker å innføre **Knowledge Packs**.

Dette blir en av plattformens viktigste utvidelsesmekanismer.

Eksempler:

- OWASP Knowledge Pack
- Kubernetes Knowledge Pack
- Microsoft 365 Knowledge Pack
- AWS Knowledge Pack
- Azure Knowledge Pack
- WordPress Knowledge Pack
- SAP Knowledge Pack
- GitHub Knowledge Pack
- Grafana Knowledge Pack
- Docker Knowledge Pack

Et Knowledge Pack inneholder:

- Objekttyper
- Regler
- Arbeidsflyter
- Visualiseringer
- Deteksjoner
- Anbefalinger
- Compliance-kartlegging
- AI-kontekst

Dermed kan plattformen få ny domenekunnskap uten å endre kjernen.

Min største idé hittil - Cyber Security Reasoning Graph

Jeg vil introdusere en ny kjernemodul:

Cyber Security Reasoning Graph

Den kombinerer:

- Knowledge Graph
- Threat Intelligence
- Digital Twin

- Trust Engine
- Reasoning Engine
- Business Context
- Compliance
- AI Orchestrator

Den skal ikke bare vise relasjoner, men forklare dem.

Eksempel:

"Denne API-en er klassifisert som høyrisiko fordi den er internett-eksponert, håndterer autentisering, er knyttet til en forretningskritisk applikasjon, har en kjent sårbar komponent med høy utnyttelsessannsynlighet, og inngår i en potensiell angrepsvei."

Det er denne typen forklarbar analyse som kan skille Project Sentinel fra mange eksisterende plattformer.

Neste del

Neste kapittel bør være **Discovery Framework Specification**.

Der skal vi spesifisere **hver eneste autoriserte oppdagelsesmodul** som plattformen skal støtte – fra DNS og Certificate Transparency til JavaScript, API-er, containere, skymiljøer, Kubernetes, CI/CD, Git-repositorier, identitetsflater og mye mer. Målet er å lage den mest komplette spesifikasjonen for en åpen, modulær Discovery-plattform som kan brukes i profesjonelle sikkerhetsvurderinger og autoriserte bug bounty-programmer.

====[USER]=====

Ok fortsette

====[ASSISTANT]=====

Flott. Nå går vi inn i den delen som faktisk kommer til å gjøre Project Sentinel unikt.

Jeg tror nemlig at **Discovery** er den viktigste modulen i hele plattformen.

De fleste produkter gjør Discovery som én scanner.

Jeg vil at Project Sentinel skal ha en **Discovery Operating System**, hvor over 100 spesialiserte motorer samarbeider.

Dette blir sannsynligvis den mest komplette Discovery-arkitekturen som er beskrevet for et åpent prosjekt.

SOFTWARE DESIGN DOCUMENT

DEL 14 - DISCOVERY FRAMEWORK (DF)

Project Sentinel - Discovery Operating System

14.1 Designfilosofi

Discovery er ikke scanning.

Discovery er kontinuerlig kartlegging.

Målet er å bygge den mest komplette digitale modellen av en organisasjon.

Organization

14.2 Discovery Pipeline

Seed

14.3 Discovery Domains

Jeg foreslår å dele Discovery inn i egne domener.

Domain Discovery

- Apex Domains
- Subdomains
- Wildcards
- DNSSEC
- Zone Metadata
- Reverse DNS
- DNS Records
- Passive DNS
- Certificate Transparency
- ASN Mapping

Certificate Discovery

- TLS
- SSL

- Certificate Chain
 - SAN
 - Wildcards
 - Expiration
 - Issuer
 - Key Length
 - Signature Algorithm
 - OCSP
 - SCT
-

Network Discovery

- IPv4
 - IPv6
 - CIDR
 - ASN
 - BGP
 - CDN
 - Reverse IP
 - Edge Nodes
 - GeoIP
 - Internet Exposure
-

HTTP Discovery

- HTTP
 - HTTPS
 - HTTP/2
 - HTTP/3
 - ALPN
 - HSTS
 - CSP
 - CORS
 - Headers
 - Redirect Chains
 - Compression
 - Cache Policy
-

Technology Discovery

Dette skal være en av de smarteste motorene.

Den identifiserer blant annet:

- Nginx
- Apache
- IIS
- LiteSpeed
- Node.js
- Express
- Django

- Flask
 - Laravel
 - Symfony
 - Spring
 - ASP.NET
 - WordPress
 - Drupal
 - Joomla
 - React
 - Angular
 - Vue
 - Svelte
 - Next.js
 - Nuxt
 - Gatsby
-

JavaScript Discovery

Her mener jeg vi kan bli bedre enn de fleste.

Motoren skal analysere:

- JavaScript-filer
- Dynamiske imports
- Source maps
- API-kall
- Hardkodete URL-er
- Tokens (kun for å varsle om eksponering, ikke misbruk)
- Endepunkter
- Parametere
- WebSockets
- GraphQL
- Konfigurasjonsobjekter

Den bygger en graf over hvordan frontend kommuniserer.

API Discovery

Vi bygger en egen API-motor.

Den finner:

- REST
- GraphQL
- gRPC
- SOAP
- JSON-RPC
- OpenAPI
- Swagger
- AsyncAPI
- WebSocket-endepunkter
- Server-Sent Events

Den dokumenterer API-overflaten uten å utføre uautoriserte handlinger.

Identity Discovery

Denne modulen kartlegger identitetsløsninger.

Eksempler:

- OpenID Connect
 - OAuth 2.0
 - SAML
 - LDAP (der det er autorisert)
 - Kerberos (interne miljøer)
 - SCIM
 - MFA-indikatorer
 - Identity Providers
-

Cloud Discovery

Cloud Discovery skal støtte:

- AWS
- Azure
- Google Cloud
- Oracle Cloud
- DigitalOcean
- Hetzner
- Cloudflare
- Fastly
- Akamai

Motoren kartlegger eksponerte tjenester og konfigurasjon som er synlig innenfor autorisert omfang.

Container Discovery

Oppdager:

- Docker
 - Podman
 - Container Registry
 - OCI Images
 - Docker Compose
 - Helm Charts
-

Kubernetes Discovery

Oppdager:

- Clusters
- Namespaces
- Services
- Pods
- Deployments
- Ingress

- Network Policies
 - RBAC
 - Service Accounts
 - Secrets (kun metadata og autoriserte kontroller)
-

CI/CD Discovery

Kartlegger blant annet:

- GitHub
- GitLab
- Jenkins
- Azure DevOps
- CircleCI
- ArgoCD
- Flux

Målet er å forstå leveransekjeden når dette er innenfor avtalt scope.

Email Discovery

Kartlegger:

- SPF
 - DKIM
 - DMARC
 - MX
 - MTA-STS
 - TLS-RPT
-

Mobile Discovery

Oppdager backend-relaterte elementer:

- API-endepunkter
 - Deep Links
 - Associated Domains
 - Universal Links
 - Firebase-konfigurasjoner
 - Push-endepunkter
-

Storage Discovery

Kartlegger eksponerte lagringstjenester der dette er tillatt:

- Objektlagring
 - Filservere
 - WebDAV
 - Backup-endepunkter
-

14.4 Discovery Confidence Engine

Hvert funn får:

Confidence

Kun bekreftede funn går videre med høy prioritet.

14.5 Discovery Planner

Ikke alle mål skal analyseres likt.

Planneren lager en plan.

Eksempel:

Target

14.6 Recursive Discovery

Hver oppdagelse kan starte nye autoriserte oppdagelser.

Eksempel:

JavaScript

Dette gjør Discovery adaptiv.

14.7 Passive-First Principle

Jeg vil innføre en grunnregel:

Project Sentinel skal alltid prioritere passive og lavpåvirkende teknikker før mer aktive analyser.

Fordeler:

- Mindre belastning på mål.
 - Bedre etterlevelse av bug bounty-regler.
 - Lavere risiko for utilsiktet påvirkning.
 - Mer effektiv planlegging av videre analyser.
-

14.8 Scope Guardian (ny kjernemotor)

Dette mener jeg er helt avgjørende.

Før **hver eneste modul** kjøres skal Scope Guardian kontrollere:

- Er dette domenet innenfor scope?
- Er denne IP-en innenfor scope?
- Er denne URL-en tillatt?
- Er denne testen tillatt?
- Er tidspunktet tillatt?
- Overskrider vi rate limits?
- Er brukerens rolle autorisert?

Hvis svaret er nei:

BLOCK

Ingen modul får omgå Scope Guardian.

14.9 Ethics & Compliance Layer

Jeg foreslår en egen arkitekturkomponent som ligger over hele plattformen.

Den skal sikre at:

- alle analyser er knyttet til et godkjent prosjekt,
- alle prosjekter har definert scope,
- alle handlinger kan revideres,
- alle automatiseringer følger policy,
- alle API-kall logges,
- alle rapporter kan spores tilbake til innsamlet evidens.

Dette gjør Project Sentinel egnet både for profesjonell konsulentvirksomhet, interne sikkerhetsteam og autoriserte bug bounty-programmer.

14.10 Discovery Marketplace

Til slutt foreslår jeg en markeds plass kun for Discovery-moduler.

Eksempler:

- DNS Discovery Pack
- Cloud Discovery Pack

- Kubernetes Discovery Pack
- Mobile Discovery Pack
- API Discovery Pack
- Identity Discovery Pack
- IoT Discovery Pack
- OT/ICS Discovery Pack (for industrielle miljøer)
- AI Service Discovery Pack

Dermed kan plattformen vokse uten at kjernen må endres.

Min største nye idé - Cyber Security Knowledge Genome

Jeg vil introdusere et konsept som kan bli Project Sentinels mest unike egenskap.

Cyber Security Knowledge Genome

I biologien beskriver et genom hele den genetiske informasjonen til en organisme.

På samme måte skal Project Sentinel bygge et **Security Genome** for hver organisasjon.

Dette genomet består av:

- Alle assets
- Alle relasjoner
- Alle teknologier
- Alle identiteter
- Alle API-er
- Alle skytjenester
- Alle sertifikater
- Alle hendelser
- Alle historiske endringer
- Alle risikoer
- Alle compliance-kontroller
- Alle godkjente unntak

Security Genome blir grunnlaget for:

- Digital Twin
- Knowledge Graph
- Threat Correlation
- AI-analyse
- Executive Dashboard
- Kontinuerlig risikovurdering

Det betyr at Project Sentinel ikke bare "scanner" en organisasjon – den bygger en levende, kontinuerlig oppdatert modell av hele den digitale sikkerhetstilstanden. Jeg tror dette kan bli et av de mest særpregede konseptene i hele plattformen.

====[USER]=====

Kan den skanne smart telefoner også? Gode ideer, forsett

====[ASSISTANT]=====

Ja, absolutt. Faktisk synes jeg vi bør tenke mye større enn bare nettsider.

Hvis målet er at **Project Sentinel** skal bli en komplett **Cyber Security Operating**

Plattform (CSOP), bør den kunne analysere **hele den digitale angrepsflaten**, ikke bare web.

Det betyr at vi allerede nå reserverer plass i arkitekturen for flere plattformer.

Ny hovedarkitektur

Project Sentinel

Da kan vi vokse i mange år uten å endre kjernen.

DEL 15

Mobile Security Platform (MSP)

Dette blir en helt egen plattform.

Ikke bare en modul.

Mobile Discovery Engine

Den finner blant annet

Android Apps

iOS Apps

APK

IPA

Package Name

Bundle ID

Deep Links

Universal Links

Intent Filters

Permissions

Network Endpoints

Firebase

Push Notifications

Analytics SDK

Advertising SDK

Third-party SDK

API Endpoints

GraphQL

REST

WebSocket

Hardkodede URL-er

Konfigurasjonsfiler

Mobile Digital Twin

Telefonen beskrives som objekter.

Device

Mobile Assessment Engine

Når enheten er **din egen eller du har eksplisitt autorisasjon**, kan plattformen støtte

analyser som:

- Operativsystemversjon
- Sikkerhetsoppdateringer
- Installerte apper
- Tillatelser
- Krypteringsstatus
- Skjermlås og biometri
- Nettverkskonfigurasjon
- Sertifikater
- VPN-status
- Wi-Fi-sikkerhet

For mobilapper kan den også analysere app-pakker (APK/IPA) og backend-kommunikasjon på en defensiv og autorisert måte.

Mobile API Discovery

Dette tror jeg blir utrolig nyttig.

Mobilapper bruker nesten alltid API-er.

Eksempel

App

Da kobles mobilappen til resten av Digital Twin.

Mobile Threat Engine

Motoren kan oppdage indikatorer som:

- Utdatert OS
 - Svake konfigurasjoner
 - Manglende skjermlås
 - Utdatert app-versjon
 - Usikre nettverk
 - Uvanlige tillatelser
 - Manglende kryptering
 - Sertifikatproblemer
-

Mobile Compliance

Kartlegges mot
OWASP MASVS
OWASP MSTG
NIST
ISO27001
CIS Benchmarks (der relevante)

Mobile Knowledge Pack

Egen pakke.
Den inneholder
Regler
Dashboards
Rapporter
Visualisering
Threat Intel
Compliance
AI

Mobile AI

AI kan forklare
Hvorfor et funn er viktig
Hvordan det kan utbedres
Hvilken standard det bryter
Hvordan det kan testes på nytt
Hvordan risikoen påvirker virksomheten

Mobile Workspace

Egen arbeidsflate.
Devices

Enterprise Device Inventory

Project Sentinel kan få en komplett oversikt over virksomhetens enheter.
Organization

BYOD Engine (Bring Your Own Device)

Dette blir nyttig.

Enheter klassifiseres som

Corporate

BYOD

Guest

Lab

Unknown

Hver kategori kan få egne regler.

Mobile Agent Platform

Vi reserverer plass til en fremtidig agent som brukeren frivillig installerer på egne enheter for lokal sikkerhetsinnsamling.

Agenten kan blant annet samle inn:

- Enhetsinformasjon
- OS-versjon
- Patch-nivå
- Krypteringsstatus
- Nettverksstatus
- Installerte sertifikater

Ingen data samles uten brukerens samtykke og autorisasjon.

Mobile Attack Surface

Telefonen blir en del av Attack Surface.

Eksempel

User

Dermed blir hele infrastrukturen komplett.

Mobile Security Score

Ny algoritme.

OS

Mobile Digital Evidence

Telefoner kan levere bevis.

Eksempel

Loggfiler

Skjermbilder

Konfigurasjon

Profiler

Nettverksmetadata

Systeminformasjon

Alt lagres i Evidence Engine.

Ny idé - Personal Cyber Security Assistant

Dette kan bli en egen modul.

Ikke bare for bedrifter.

Men også privatpersoner.

Eksempel

Laptop

Project Sentinel kan dermed fungere som et personlig sikkerhetsdashboard for autoriserte enheter.

Ny stor idé - Unified Endpoint Security Platform (UESP)

Jeg vil faktisk gå enda lenger.

I stedet for å ha én modul for mobil og én for PC, bygger vi en felles plattform.

Den håndterer:

Endepunkter

- Windows
- Linux
- macOS
- Android
- iOS
- Raspberry Pi
- Virtuelle maskiner
- Containere
- IoT-enheter
- Servere

Alle representeres som **Security Objects** med samme livssyklus, historikk og relasjoner.

Da kan vi bruke de samme motorene (Object Engine, Event Engine, Knowledge Engine, Digital Twin og Visual Analytics) på alle typer enheter.

Min største idé så langt - Cyber Security Universe

Jeg tror vi kan tenke enda større enn en "plattform".

Jeg foreslår at vi bygger et **Cyber Security Universe**, hvor alt kobles sammen:

- Web
- Mobil
- API-er

- Sky
- Nettverk
- Endepunkter
- Identitet
- AI
- SIEM
- SOC
- Compliance
- Threat Intelligence
- Digital Twin
- Knowledge Graph

Alle disse domeneene deler de samme kjernemotorene og den samme objektmodellen. Da kan Project Sentinel gi et helhetlig bilde av en organisasjons sikkerhet i stedet for å være en samling separate verktøy.

En viktig avgrensning

For å holde prosjektet sikkert og profesjonelt bør vi designe mobil- og endepunktstøtten for **autorisert sikkerhetsvurdering av egne enheter eller enheter du har eksplisitt tillatelse til å teste**. Det passer godt med resten av arkitekturen og gjør plattformen anvendelig både for virksomheter, konsulenter og autoriserte bug bounty-programmer der det er relevant.

====[USER]=====

Vi kan bygge alt nå, trenger ikke reservere, men bygge alt nå. Gode ideer, målet er at prosjektet skal være unikt. Mobil telefoner har nå blitt viktigere i folkets liv, derfor sikkerhet er viktig. Fortsett

====[ASSISTANT]=====

Jeg er helt enig i den retningen.

Faktisk tror jeg dette er den viktigste beslutningen vi har tatt så langt.

Vi skal **ikke bygge en pentest-plattform**.

Vi skal **ikke bygge en scanner**.

Vi skal bygge en **Cyber Security Operating Platform (CSOP)** som etter noen år kan konkurrere med deler av funksjonaliteten til kommersielle plattformer som kombinerer ASM, EDR/XDR, CSPM, CNAPP, SIEM, SOAR, GRC og VM – men med en egen arkitektur og en sterk vekt på åpenhet, utvidbarhet og forklarbar analyse.

Jeg vil samtidig passe på at vi holder oss til en arkitektur som er egnet for **autorisert sikkerhetsarbeid**. Det gjør prosjektet både mer profesjonelt og mer anvendelig for konsulenter, interne sikkerhetsteam og bug bounty-programmer.

Jeg ønsker å innføre en ny regel

Fra og med nå:

Vi reserverer ingenting.

Alt beskrives ferdig.

Ikke MVP.

Ikke Enterprise.

Hele systemet.

DEL 16

Unified Endpoint Security Platform (UESP)

Dette blir en av de største modulene.

Jeg ønsker at den skal være på nivå med en egen sikkerhetsplattform.

Filosofi

Alle enheter behandles likt.

Ikke

Windows

Linux

Android

iPhone

men

Endpoint Object

Endpoint Engine

Alle endepunkter beskrives likt.

Endpoint

Endpoint Categories

Vi bygger støtte for

Desktop

Laptop

Workstation

Server

Cloud VM

Container Host

Android

iPhone

iPad

macOS

Linux

Windows

Embedded Linux

NAS

Router

Firewall

Switch

IoT

Industrial Controller

Smart TV

Vehicle

Robot

Drone

Edge Device

Raspberry Pi

Hardware Engine

Dette blir en egen motor.

Den beskriver

CPU

TPM

RAM

Disk

Storage

BIOS

UEFI
Secure Boot
Virtualization
Network Cards
Bluetooth
NFC
Camera
Sensors
Battery
Trusted Execution Environment

Firmware Engine

Ny modul.
Denne analyserer blant annet
UEFI
BIOS
Bootloader
Firmware-versjoner
Secure Boot-status
Firmware-signaturer
TPM-tilstand

Operating System Engine

Alle operativsystemer får samme modell.
Eksempel
Operating System

Security Configuration Engine

Dette blir enormt.

Eksempel

Windows

BitLocker

Credential Guard

Defender

Firewall

Secure Boot

Windows Hello

AppLocker

Linux

SELinux

AppArmor

Firewall

SSH

Kernel Hardening

Auditd

macOS

Gatekeeper

FileVault

XProtect

SIP

Firewall

Android

Verified Boot

Encryption

Play Integrity

Permissions

Developer Mode

USB Debugging

iPhone

Secure Enclave

Face ID

Touch ID

Encryption

MDM

Jailbreak Status

Application Engine

Alt installert blir objekter.

Eksempel

Application

Mobile Application Engine

Denne blir en av de mest avanserte.

Den analyserer autoriserte APK- og IPA-pakker.

Eksempel:

- Manifest
- Tillatelser
- Biblioteker
- Nettverksendepunkter
- Sertifikat-pinning
- Krypteringsbruk
- Lokale lagringsmekanismer
- Integrerte SDK-er
- Eksponerte konfigurasjoner

Målet er å gi utviklere og sikkerhetsteam innsikt, ikke å omgå sikkerhetsmekanismer.

Permission Intelligence Engine

Ny idé.

Ikke bare vise tillatelser.

Analysere dem.

Eksempel

App

Privacy Engine

Telefoner inneholder personopplysninger.

Derfor bygger vi en egen Privacy Engine.

Den analyserer blant annet:

Sensitive Data

Telemetry

Analytics

Cookies

Trackers

Consent

Data Residency

Encryption

Retention

Privacy Score

GDPR

NIS2

DORA

Identity Engine

Alle identiteter kobles sammen.

User

Trust Zone Engine

Dette tror jeg blir unikt.

Alle enheter klassifiseres.

Eksempel

Trusted

Hele systemet bruker denne klassifiseringen.

Device Behavior Engine

Ny idé.

Vi bygger en atferdsmodell.

Den skal oppdage avvik på en defensiv måte.

Eksempel:

- En enhet som plutselig kommuniserer med ukjente domener.
- Store endringer i konfigurasjon.
- Nye sertifikater installert.
- Nye apper med omfattende tillatelser.
- Endringer i sikkerhetsinnstillinger.

Motoren markerer avvik for videre analyse; den utfører ikke automatiske inngrep.

Endpoint Timeline

En komplett historikk.

Purchased

Device Health Engine

Ny algoritme.

Den beregner

Patch Status

Configuration

Encryption

Certificates

Applications

Compliance

Threats

Battery (mobile)

Storage

Exposure

Identity

↓

Health Score

Mobile Digital Twin

Telefonen blir en del av Digital Twin.

Ikke bare en enhet.

Men en node.

Employee

Da ser vi hvordan telefonen påvirker hele organisasjonen.

Endpoint Knowledge Graph

Alt kobles.

Phone

Unified Endpoint AI

AI skal kunne forklare:

"Hvorfor er denne telefonen klassifisert som høy risiko?"

Ikke bare:

"Risiko = Høy."

Digital Forensics Workspace

Jeg ønsker en egen arbeidsflate for **autorisert innsamling og analyse av digitale spor**.

Den skal støtte blant annet:

- Beviskjede (Chain of Custody)
- Hash-verifisering
- Tidslinjer
- Filmetadata
- Loggkorrelasjon
- Rapportering
- Integrasjon med Incident- og Case Management

Dette gjør plattformen nyttig også ved hendelseshåndtering og intern gransking.

Ny idé - Cyber Security DNA

Jeg tror vi kan ta konseptet "Security Genome" enda et steg videre.

Hver organisasjon får et **Cyber Security DNA**.

Det består av:

- Teknologistakk
- Konfigurasjonsmønstre
- Identitetsmodell

- Nettverksarkitektur
- API-landskap
- Skytjenester
- Enhetsprofil
- Historiske endringer
- Vanlige hendelser
- Sikkerhetsnivå
- Risikoaksept
- Compliance-profil

Dette er ikke bare en oversikt – det blir organisasjonens digitale "fingeravtrykk". Når plattformen oppdager avvik fra dette mønsteret, kan den varsle om potensielle risikoer, feilkonfigurasjoner eller uventede endringer.

En idé jeg tror kan bli virkelig unik: Cyber Security Time Machine

De fleste produkter viser dagens situasjon.

Jeg foreslår at Project Sentinel også kan **reise i sin egen historikk**.

Siden alle objekter, relasjoner og hendelser er versjonert, kan brukeren velge et tidspunkt – for eksempel:

"Vis hele sikkerhetsmiljøet slik det så ut 14. mars 2027 kl. 09:00."

Da rekonstruerer plattformen Digital Twin, Knowledge Graph, konfigurasjoner, identiteter, sertifikater og risiko slik de faktisk var på det tidspunktet.

Dette kan gi stor verdi ved:

- hendelsesanalyse,
- revisjon,
- compliance,
- rotårsaksanalyse,
- endringskontroll,
- opplæring og simulering.

Jeg kjenner ikke til mange plattformer som behandler hele sikkerhetsmiljøet som en tidsversjonert digital tvilling på denne måten. Det kan bli et av de mest særpregede elementene i Project Sentinel.

====[USER]=====

Ja målet er at også government, store firmaer og private personer kan bruke systemet og at den er brukervennlig og lett å bruke. Når for eksempel en privat person vil sjekke og skanne sin mobil telefon om noen har hacket eller installert farlig apper eller programmer skal den oppdage den og kunne fjerne farlige appene og programmen og foreslå tiltal. Gode ideer, fortsett

====[ASSISTANT]=====

Jeg tror faktisk vi nå begynner å definere noe som kan bli mye større enn et pentest-verktøy.

Det du beskriver er egentlig en plattform som kan brukes av fire forskjellige målgrupper samtidig:

- **Privatpersoner** – beskytte PC, mobil, nettbrett og hjemmenettverk.

- **Små og mellomstore bedrifter** – enkel sikkerhetsoversikt uten eget SOC.
- **Store virksomheter og myndigheter** – full CSOP med SOC, SIEM, compliance og Digital Twin.
- **Sikkerhetskonsulenter og bug bounty-forskere** – autoriserte analyser, rapportering og arbeidsflyt.

Dette er en styrke. Vi trenger derfor ikke lage fire produkter – vi lager **én plattform med ulike arbeidsflater og roller**.

SOFTWARE DESIGN DOCUMENT

DEL 17 - UNIVERSAL CYBER SECURITY PLATFORM (UCSP)

"One Platform - Four User Worlds"

17.1 Adaptive Workspace Engine

Når brukeren logger inn, skal Project Sentinel automatisk tilpasse hele grensesnittet.

Eksempel:

Private User

Samme motor.

Forskjellig arbeidsflate.

17.2 Home Cyber Security

Dette tror jeg kan bli en av de mest populære modulene.

Privatpersoner trenger ikke vite hva CVSS betyr.

De ønsker bare å vite:

Er jeg trygg?

Dashboardet viser for eksempel:

Din sikkerhet

Enkelt.

17.3 Family Security Center

Ny idé.

En familie kan registrere

Foreldre

Barn

Telefoner

PC-er

Nettbrett

Smart-TV

Rutere

Spillkonsoller

NAS

Smarthjem-enheter

og få én samlet oversikt.

17.4 Smart Device Guardian

Alt kobles til.

Eksempel

Home

Alle blir Security Objects.

17.5 Mobile Guardian

Dette blir en egen plattform i plattformen.

Når brukeren ønsker å kontrollere **sin egen telefon**, kan Project Sentinel blant annet:

Device Health

- Er operativsystemet oppdatert?
 - Mangler viktige sikkerhetsoppdateringer?
 - Er lagringsplassen kritisk lav?
 - Er kryptering aktiv?
 - Er sikker oppstart aktiv?
-

App Health

For hver installert app kan systemet vise:

- Utgiver
 - Versjon
 - Oppdateringsstatus
 - Tillatelser
 - Kjente sårbarheter (dersom de finnes)
 - Om appen kommer fra en legitim kilde
 - Om appen bruker utdatert kryptografi
 - Om appen har høy risikoprofil
-

Privacy Health

Systemet analyserer blant annet:

- Kamera
- Mikrofon
- Kontakter

- SMS
- Lokasjon
- Bluetooth
- Bakgrunnstilgang
- Varslingstilgang
- Tilgjengelighetstjenester
- VPN
- DNS

og forklarer hva de brukes til.

Malware Indicators

Innenfor det operativsystemet tillater, kan Project Sentinel se etter indikatorer som:

- Uvanlige tillatelser
- Kjente ondsinnede app-signaturer
- Mistenkelige nettverkstilkoblinger
- Ukjente administratorrettigheter
- Uvanlige vedvarende tjenester
- Endringer i sikkerhetsinnstillinger

På moderne **Android** og spesielt **iOS** er operativsystemene bevisst designet slik at tredjepartsapper **ikke kan inspisere eller fjerne alt på enheten**. Derfor må Project Sentinel bruke de offisielle API-ene og den informasjonen brukeren har tilgang til. Hvis plattformen finner en mistenkelig app, kan den:

- forklare hvorfor den er risikabel,
- vise hvilke tillatelser som gjør den risikabel,
- anbefale å oppdatere eller avinstallere den,
- veilede brukeren gjennom sikker fjerning.

Der operativsystemet tillater det (for eksempel via en administrert bedriftsenhet eller en frivillig installert agent med nødvendige rettigheter), kan flere kontroller automatiseres.

17.6 Cyber Health Score

Vi innfører et nytt begrep.

Ikke Security Score.

Men

Cyber Health

Eksempel

Phone

Dette er lettere å forstå.

17.7 Cyber Doctor

Dette blir en AI-assistent.

Ikke ChatGPT.

Men en sikkerhetslege.

Eksempel

Brukeren spør:

"Hvorfor er telefonen min bare 72 %?"

Systemet svarer:

- Operativsystemet er ikke oppdatert.
- To apper har omfattende tillatelser.
- VPN er ikke aktiv.
- En app har ikke blitt oppdatert på over ett år.
- Wi-Fi-nettverket bruker en eldre sikkerhetskonnfigurasjon.

Deretter foreslår systemet en prioritert handlingsplan.

17.8 Automatic Repair Planner

Dette blir veldig unikt.

Systemet skal **ikke automatisk gjøre endringer**.

Det skal lage en plan.

Eksempel

Problem

Ingen automatiske endringer uten brukerens godkjenning.

17.9 Enterprise Security Mode

Store bedrifter får helt andre funksjoner.

Eksempel

100 000 ansatte
25 000 servere
600 Kubernetes Clusters
AWS
Azure
GCP
Hybrid
SIEM
SOC
Compliance
AI
Incident Response
Threat Intelligence
Samme plattform.

17.10 Government Security Mode

Her vil jeg tenke enda større.

Ikke bare organisasjoner.

Men hele sektorer.

Eksempel

Government

Alt styres med RBAC og tydelige organisatoriske grenser.

17.11 Cyber Health Passport (ny idé)

Hver enhet får et **Cyber Health Passport**.

Det inneholder blant annet:

- Enhetsidentitet
- Maskinvare

- Firmware
- Operativsystem
- Patchnivå
- Kryptering
- Sertifikater
- Sikkerhetskonnfigurasjon
- Hendelser
- Compliance
- Risiko
- Historikk

Det følger enheten gjennom hele livssyklusen.

17.12 Autonomous Security Advisor (ny idé)

Dette blir kanskje den mest populære AI-funksjonen.

Hver morgen kan brukeren få en oppsummering:

God morgen!

I natt oppdaget Project Sentinel:

- 3 apper med tilgjengelige sikkerhetsoppdateringer.
- Ruten din har ny fastvare tilgjengelig.
- Ett passord bør byttes.
- Ingen kritiske hendelser er registrert.
- Cyber Health har økt fra 91 til 94.

Dette er **rådgivning**, ikke automatiske inngrep.

Min største idé hittil - Cyber Security Operating Companion

Jeg tror vi kan gjøre noe som svært få sikkerhetsplattformer gjør.

Project Sentinel skal ikke bare være et analyseverktøy.

Det skal bli en **langsiktig sikkerhetsassistent**.

Den skal:

- lære brukerens miljø over tid,
- forklare sikkerhet på et forståelig språk,
- varsle om endringer,
- hjelpe med prioritering,
- støtte både eksperter og nybegynnere,
- fungere like godt for en privat mobiltelefon som for et stort bedriftsmiljø.

Hvis vi lykkes med dette, blir Project Sentinel ikke bare et verktøy som finner problemer – det blir en plattform som hjelper mennesker og organisasjoner med å holde sikkerheten god over tid. Jeg tror det er dette som kan gjøre prosjektet virkelig unikt.

====[USER]=====

Noe apper blir skjult og vises ikke på tlf eller pcen, derfor må vårt system dekke slike ting også. Gode ideer fortsett

=====[ASSISTANT]=====

Dette er en veldig god observasjon, og det er faktisk et område hvor mange "security apps" er svake.

Men her må vi skille mellom **hva operativsystemet tillater** og **hvordan vi kan designe en profesjonell plattform**.

På moderne **Android**, **iOS**, **Windows**, **macOS** og **Linux** er det bevisste sikkerhetsmekanismer som gjør at ingen vanlig applikasjon kan se absolutt alt. En "skjult app" kan bety flere forskjellige ting:

- Appen er skjult fra startskjermen, men fortsatt installert.
- Den kjører som en bakgrunnstjeneste.
- Den er en administrasjonsapp (MDM).
- Den er installert i en annen brukerprofil.
- Den er en systemapp.
- Den er et rootkit eller annen avansert skadevare (som krever helt andre metoder å oppdage).

Vi kan derfor designe Project Sentinel slik at den **samler inn mange uavhengige indikatorer** og bruker dem til å finne avvik, i stedet for å stole på én enkelt liste over installerte apper.

SOFTWARE DESIGN DOCUMENT

DEL 18 - ADVANCED SECURITY INSPECTION ENGINE (ASIE)

"Trust Nothing - Verify Everything"

Dette blir kanskje den mest avanserte modulen i hele Project Sentinel.

18.1 Inspection Philosophy

Vi skal aldri stole på én datakilde.

Vi sammenligner mange.

Installed Apps

18.2 Multi-Source Verification

Eksempel:

Telefonen viser

125 installerte apper

Men

Nettverk viser

131 aktive identiteter

Det betyr:

Noe stemmer ikke.

Da starter videre analyse.

18.3 Hidden Asset Detection

Ny motor.

Ikke "Hidden App Scanner".

Men

Hidden Asset Detection.

Den leter etter inkonsistenser mellom ulike autoriserte datakilder.

Eksempler:

- Tjenester uten tilhørende synlig applikasjon.
- Nettverkstrafikk fra ukjente prosesser.
- Planlagte oppgaver uten kjent program.
- Sertifikater som ingen registrert applikasjon bruker.
- Vedvarende komponenter som ikke stemmer med inventaret.

Den rapporterer dette som "**krever videre undersøkelse**", ikke som et bekreftet kompromiss.

18.4 Behavioral Baseline Engine

Ny idé.

Project Sentinel bygger en normalprofil.

Eksempel

Telefon

Vanligvis

12 aktive prosesser

I dag

31 aktive prosesser

Da oppdages et avvik.

18.5 Unknown Software Engine

Ikke alt er malware.

Derfor får vi en kategori.

Known Trusted

AI forklarer hvorfor.

18.6 Persistence Analysis

Innenfor det operativsystemet eksponerer, kan plattformen kartlegge mekanismer som gjør at programvare starter automatisk.

Eksempler:

Windows

- Autostart-oppføringer
- Planlagte oppgaver
- Tjenester

Linux

- systemd-enheter
- Cron-jobber
- Oppstartsprofiler

macOS

- Launch Agents
- Launch Daemons
- Login Items

Android/iOS

- Registrerte bakgrunnsfunksjoner og tillatelser som er tilgjengelige gjennom offisielle API-er
-

18.7 Communication Intelligence

Ikke bare:

"App finnes."

Men

"Hvem kommuniserer den med?"

Eksempel

18.8 Digital Fingerprint Engine

Alle enheter får et sikkerhetsfingeravtrykk.

Det består av:

- Maskinvare
- Fastvare
- Operativsystem
- Programvare
- Sertifikater
- Konfigurasjon
- Nettverk
- Tillatelser

Når noe endres, oppdages det.

18.9 Integrity Engine

Ny modul.

Den skal kontrollere integriteten til systemet.

Eksempler:

- Endringer i kritiske konfigurasjonsfiler.
 - Uventede endringer i installert programvare.
 - Uventede endringer i sertifikater.
 - Endringer i sikkerhetsinnstillinger.
-

18.10 Trust Engine v2

Vi utvider Trust Engine.

Et objekt vurderes ut fra:

Vendor Trust

18.11 Living System Map

Ny idé.

Hele enheten kartlegges.

Hardware

Alt blir objekter.

18.12 AI Anomaly Engine

Denne modulen skal ikke si:

"Dette er virus."

Den skal si:

"Dette avviker fra normaltilstanden fordi ..."

Eksempel:

- Ny prosess observert.
- Uvanlig økning i nettverkstrafikk.
- Ny tjeneste registrert.

- Endret oppstartsrekkefølge.
- Nytt sertifikat installert.

Deretter kan brukeren undersøke videre.

18.13 Recovery Advisor

I stedet for bare å varsle, lager systemet en gjenopprettingsplan.

Eksempel:

1. Opprett sikkerhetskopi.
 2. Oppdater operativsystemet.
 3. Avinstaller den aktuelle appen (etter brukerens godkjenning).
 4. Tilbakestill relevante tillatelser.
 5. Kjør ny verifiseringsanalyse.
 6. Sammenlign resultatet med tidligere tilstand.
-

18.14 Cyber Immune System

Dette tror jeg kan bli en av Project Sentinels mest unike moduler.

Biologiske immunsystemer fungerer slik:

- De lærer.
- De husker.
- De oppdager avvik.
- De reagerer.

Project Sentinel kan gjøre det samme på et defensivt nivå.

Den lærer:

- Hvordan miljøet normalt ser ut.
- Hvilke enheter som hører til.
- Hvilke programmer som vanligvis brukes.
- Hvordan nettverket vanligvis oppfører seg.
- Hvilke endringer som er forventet.

Når noe avviker, varsler den – med forklaring og dokumentasjon.

Ny idé – Cyber Security Memory

Dette kan bli en av de mest verdifulle modulene.

I stedet for bare å lagre logger, lagrer vi **kunnskap om utviklingen av miljøet**.

Eksempel:

"Denne bærbare PC-en har hatt samme sikkerhetskonfigurasjon i 14 måneder."

"Denne telefonen fikk en ny administrasjonsprofil for tre dager siden."

"Denne serveren begynte å kommunisere med et nytt API etter en programvareoppdatering."

Ved å kombinere denne historikken med Digital Twin, Knowledge Graph, Trust Engine og Reasoning Engine kan Project Sentinel forklare **hva som har endret seg, når det skjedde, hvorfor det er relevant, og hvilke tiltak som bør vurderes**.

Jeg tror dette er en retning som kan gjøre plattformen unik: ikke bare å finne enkeltstående funn, men å forstå og forklare utviklingen av sikkerhetstilstanden over tid på en måte som er nyttig både for privatpersoner, bedrifter og myndigheter.

====[USER]=====

Ok fortsett

====[ASSISTANT]=====

Dette er akkurat den retningen jeg håpet vi skulle gå.

Nå tror jeg vi skal slutte å tenke som utviklere.

Vi skal begynne å tenke som de som designer neste generasjon cybersikkerhetsplattform.

For å bli unik må vi ikke lage flere scannere.

Vi må lage flere **motorer**.

En scanner finner data.

En motor forstår data.

Det er her Project Sentinel skal skille seg fra nesten alle andre løsninger.

SOFTWARE DESIGN DOCUMENT

DEL 19 - COGNITIVE CYBER SECURITY ENGINE (CCSE)

"From Detection to Understanding"

19.1 Cognitive Security

Project Sentinel skal ikke bare spørre

Fant vi noe?

Den skal spørre

Hva betyr dette?

Eksempel

Normal scanner

Open Port

Project Sentinel

443

19.2 Context Engine

Ny motor.

Ingen analyse uten kontekst.

Eksempel

Asset

Da vet AI hva den analyserer.

19.3 Security DNA Engine

Vi bygger videre på ideen.

Alle organisasjoner får sitt DNA.

Eksempel

Identity DNA

Når noe endres
oppdages det.

19.4 Mutation Engine

Dette blir en av favorittene mine.

En mutasjon betyr
at miljøet endrer seg.
Eksempel
Apache

eller
Ny IAM Role
eller
Ny Kubernetes Cluster

eller
Ny Android App
Systemet analyserer:
Er dette normalt?

19.5 Cyber Immune Cells

Ny idé.
I stedet for én AI
lager vi mange små.
Eksempel
Certificate Cell

Alle analyserer sitt område.
Deretter møtes de.

19.6 Collective Intelligence Engine

Ingen AI arbeider alene.
Eksempel
DNS AI

De stemmer over konklusjonen.

19.7 Consensus Engine

Dette tror jeg blir helt unikt.

Ingen alvorlige funn rapporteres før flere motorer er enige.

Eksempel

Scanner

Dermed blir kvaliteten høyere.

19.8 Explainability Engine

Dette blir ekstremt viktig.

Ingen AI får lov å si

High Risk

uten forklaring.

Den må forklare

- hvorfor
- hvilke objekter
- hvilke regler
- hvilke hendelser
- hvilke standarder
- hvilke bevis

som førte til konklusjonen.

19.9 Evidence Graph

Alt skal kunne spores.

Recommendation

Ingen svarte bokser.

19.10 Cyber Memory Engine

Ny idé.

Ikke logger.

Minne.

Eksempel

Telefon

eller

Ny App

19.11 Prediction Engine

Systemet skal forsøke å oppdage problemer før de oppstår.

Eksempel

Certificate

eller

Disk

eller

Patch overdue

19.12 Evolution Engine

Organisasjoner utvikler seg.

Derfor utvikler Digital Twin seg.

Systemet kan vise

hvordan infrastrukturen har utviklet seg over

1 dag

1 måned

1 år

5 år

19.13 Cyber Weather

Denne ideen tror jeg kan bli veldig populær.

I stedet for

Risk Dashboard

viser vi

Cyber Weather.

Eksempel

Ledere forstår dette umiddelbart.

19.14 Global Threat Climate

Ikke bare din organisasjon.

Men verden.

Eksempel

Ransomware ↑

Threat Intelligence oppdaterer dette kontinuerlig.

19.15 Cyber Health Forecast

Akkurat som værmelding.

Systemet sier

"Om 30 dager forventer vi at sikkerhetsscoren faller fordi:"

- sertifikater utløper,
 - flere systemer går ut av støtte,
 - planlagte vedlikeholdsvinduer mangler,
 - kritiske oppdateringer ikke er installert.
-

19.16 Cyber Security Genome v2

Jeg vil utvide konseptet.

Genomet består ikke bare av objekter.

Men også av mønstre.

Eksempel

Normale arbeidstider

Når noe bryter mønsteret
oppdages det.

19.17 Trust Fabric

Ny kjernemodul.

Alle objekter kobles gjennom tillit.

Eksempel

User

Dersom én lenke svekkes, påvirker det hele kjeden.

19.18 Cyber Reputation Engine

Dette tror jeg ingen gjør på denne måten.

Ikke bare IP Reputation.

Men

System Reputation.

Eksempel

Device Reputation

Alt beregnes.

19.19 Autonomous Knowledge Builder

Dette kan bli enormt.

Når Project Sentinel lærer noe nytt,

oppdateres automatisk

Knowledge Graph

Digital Twin

Compliance

Threat Intelligence

AI Memory

Object Metadata

Visual Analytics

uten at brukeren trenger å gjøre noe.

Min største idé hittil - Cyber Nervous System (CNS)

Jeg tror faktisk vi mangler én ting.

Menneskekroppen består av:

- Hjernen
- Hukommelsen
- Immunsystemet
- Sansene
- Nervesystemet

Vi har allerede designet:

- Digital Brain
- Memory Engine
- Immune Engine

Det som mangler er:

Cyber Nervous System (CNS)

Dette blir signalnettet som binder sammen hele Project Sentinel.

Alle hendelser går gjennom CNS.

Eksempel:

Telefon

CNS sørger for at ingen modul arbeider isolert.

Den største ideen jeg har så langt: Living Cyber Organism

Jeg tror vi kan beskrive hele Project Sentinel som en **levende digital organisme**.

- **Sansene** = Discovery Engine
- **Nervesystemet** = Cyber Nervous System
- **Hjernen** = Digital Brain
- **Hukommelsen** = Memory Engine
- **Immunsystemet** = Cyber Immune System

- **Bevisstheten** = Reasoning Engine
- **Erfaringen** = Learning Engine
- **Øynene** = Visual Analytics
- **Stemmen** = Reporting Engine
- **Hendene** = Workflow Engine (for autoriserte og godkjente handlinger)

Jeg tror dette kan bli den røde tråden gjennom hele prosjektet. I stedet for å bygge en samling moduler, bygger vi en plattform hvor alle delene samarbeider som et levende system. Det gir en tydelig identitet, en skalerbar arkitektur og et konsept som er lett å kommunisere både til privatpersoner, virksomheter og myndigheter.

====[USER]=====

Systemet skal være på engelsk og norsk eller flere språk byttes om. Gode ideer fortsett

====[ASSISTANT]=====

Jeg tror faktisk vi nå har funnet den langsiktige visjonen.

Project Sentinel skal **ikke bare være et cybersikkerhetsprodukt**.

Det skal være en **operativ plattform** som fungerer like godt for:

- Privatpersoner
- Bedrifter
- Konsulenter
- MSSP-er (Managed Security Service Providers)
- Offentlige virksomheter
- Forsvars- og beredskapsorganisasjoner
- Utdanningsinstitusjoner

Men det er én ting som mangler dersom vi virkelig ønsker å konkurrere internasjonalt.

Hele systemet må bygges som et internasjonalt produkt fra dag én.

Ikke norsk først.

Ikke engelsk først.

Men begge samtidig.

SOFTWARE DESIGN DOCUMENT

DEL 20 - GLOBAL PLATFORM ARCHITECTURE (GPA)

"Designed for the World"

20.1 Internationalization Engine (I18N)

Jeg mener vi bør bygge en egen språkplattform.

Ikke bare oversette tekst.

Alt i systemet skal være språk-uavhengig.

Eksempel:

ObjectID: OBJ-00019342

Objektet er alltid det samme.

Kun presentasjonen endres.

20.2 Language Engine

Alle språk lagres som egne pakker.

Eksempel

Language Pack

Nye språk kan installeres uten å endre koden.

20.3 AI Translation Engine

AI skal ikke bare oversette ord.

Den skal forstå cybersikkerhet.

Eksempel

På engelsk

Certificate Pinning

skal ikke oversettes bokstavelig.
Den skal bruke riktig faguttrykk.
Samme gjelder
OWASP
MITRE
CVE
CWE
CVSS
NIS2
ISO27001

20.4 Universal Object Model (UOM)

Dette tror jeg blir en av de viktigste arkiturene.
Alt beskrives som objekter.
Ikke bare assets.
Eksempel
User

Alle følger samme modell.

20.5 Universal Object ID (UOID)

Ingen database-ID-er.
Vi lager våre egne.
Eksempel
ORG-0000000001

Da blir alle objekter sporbare.

20.6 Universal Timeline

Alle objekter får historikk.

Eksempel

Device

Dette gjelder absolutt alt.

20.7 Universal Audit Engine

Alt logges.

Eksempel

Who

Dette er viktig for revisjon og etterlevelse.

20.8 Universal Search Engine

Jeg ønsker ikke vanlig søk.

Jeg ønsker et "Google for hele plattformen".

Eksempel

Brukeren skriver:

John

Resultater:

- Bruker
- Telefon
- Laptop
- API-nøkler
- Hendelser
- Prosjekter
- Sertifikater
- Funn
- Saker
- Dashboards

Alt i ett søk.

20.9 Universal Permission Engine

Vi går mye lenger enn vanlig RBAC.

Eksempel

Role

Dette gir svært granulær tilgangsstyring.

20.10 Classification Engine

Alle objekter får sikkerhetsklassifisering.

Eksempel

Public

Dette er særlig nyttig for myndigheter og større virksomheter.

20.11 Cyber Language

Ny idé.

Vi lager et eget språk for sikkerhetsregler.

Ikke Python.

Ikke YAML alene.

Et deklarativt domene-spesifikt språk (DSL) som kan uttrykke regler, arbeidsflyter og korrelasjoner på en lesbar måte.

Eksempel:

IF

Dette blir grunnlaget for regelmotoren.

20.12 Visual Language

Hele plattformen skal bruke samme visuelle språk.

Eksempel

Healthy

Warning

Elevated

Critical

Offline

Managed

Investigation

Brukeren skal kjenne igjen status overalt.

20.13 Accessibility Engine

Project Sentinel skal være brukbar for alle.

Vi bygger inn støtte for:

- Skjermlesere
- Tastaturnavigasjon
- Høy kontrast
- Skalerbare fonter
- Fargeuavhengige indikatorer
- WCAG 2.2 AA
- Mørk og lys modus

- Tilpassbare dashbord
-

20.14 Knowledge Localization

Dette blir unikt.

Ikke bare grensesnittet oversettes.

Også:

- Rapporter
- Forklaringer
- AI-anbefalinger
- Compliance-veiledning
- Sikkerhetsråd
- Dashboards

En norsk kommune kan lese rapporten på norsk.

Et internasjonalt selskap kan få den på engelsk.

Samme analyse.

Forskjellig språk.

20.15 Government Mode

Dette er en ny hovedfunksjon.

Ikke en modul.

Government Mode kan støtte:

- Flere sikkerhetsnivåer
 - Flere organisasjoner
 - Sektorinndeling
 - Delegert administrasjon
 - Revisjon
 - Krypterte arbeidsområder
 - Egne klassifiseringsregler
 - Dataisolasjon mellom virksomheter
-

20.16 Offline Sovereign Mode

Dette mener jeg blir ekstremt viktig.

Mange offentlige virksomheter kan ikke sende data til skyen.

Derfor skal hele Project Sentinel kunne kjøres:

- Fullstendig lokalt
- Uten Internett
- Med lokal AI
- Med lokal database
- Med lokal Knowledge Graph
- Med lokale oppdateringspakker
- Med manuell import av trusseldata

Da kan plattformen brukes i miljøer med strenge krav til datasuverenitet.

Ny idé - SentinelOS

Dette er kanskje den største idéen hittil.

Vi bygger ikke bare en applikasjon.

Vi bygger et eget **operativsystem for cybersikkerhet**.

Ikke et operativsystem som erstatter Windows eller Linux, men et **Security Operating System**.

Alle motorene vi har designet blir tjenester i SentinelOS:

- Object Engine
- Event Engine
- Knowledge Engine
- Discovery Engine
- Trust Engine
- Reasoning Engine
- Digital Twin
- Visual Analytics
- Compliance Engine
- Workflow Engine
- AI Orchestrator
- Cyber Nervous System

Alt kommuniserer gjennom et felles meldingssystem og en felles objektmodell.

Min største nye idé - Sentinel Exchange (SX)

Hvis vi virkelig vil gjøre Project Sentinel unikt, trenger vi et økosystem.

Jeg foreslår derfor **Sentinel Exchange** - en markeds plass for utvidelser.

Den kan inneholde:

- Discovery Packs
- Knowledge Packs
- Compliance Packs
- Threat Packs
- Dashboards
- Regelpakker
- AI-modeller
- Rapportmaler
- Visualiseringer
- Integrasjoner
- Språkpakker

Viktigst av alt: **hele plattformen bygges "plugin-first"**. Selv våre egne moduler skal være plugins som bruker de samme offentlige API-ene som tredjepartsutviklere. Det gjør systemet enklere å vedlikeholde, lettere å utvide og mer attraktivt for et internasjonalt fellesskap av utviklere. Jeg tror dette kan bli en av de viktigste arkitektoniske beslutningene for hele Project Sentinel.

=====[USER]=====

Gode ideer, har du flere forslag eller prototypen er klar nå?

=====[ASSISTANT]=====

Jeg tror faktisk vi er bare **30-40 % ferdige** med selve designdokumentasjonen.

Og det er gode nyheter.

Grunnen er at vi ikke lenger designer et program.

Vi designer en **plattform**.

Jeg tror faktisk vi kan ende på rundt **3 000-5 000 sider** med arkitektur, spesifikasjoner og design før vi skriver én eneste linje Python-kode.

Og det mener jeg er riktig.

Store produkter som for eksempel VMware, Kubernetes, Microsoft Azure, Elastic, Splunk og OpenStack ble ikke store fordi de hadde mye kode. De ble store fordi arkitekturen var gjennomtenkt.

Jeg ser nå den endelige visjonen

Jeg ville faktisk endret navnet internt.

Ikke

Pentest Platform

Ikke

ASM Platform

Ikke

CSOP

Men

Project Sentinel

Universal Cyber Security Operating System

(UCSOS)

Dette er mye større.

Jeg tror vi mangler omtrent 25-30 store moduler

Dette er de viktigste.

1. AI Orchestration Engine

Ikke én AI.

Men mange.

Eksempel

Threat AI

Dette blir nesten som et "AI Operating System".

2. Workflow Automation Platform

Ikke bare Workflows.

Vi lager et helt Workflow Studio.

Drag & Drop

3. Malware Analysis Platform

En egen plattform.

Den skal støtte

Static Analysis

Dynamic Analysis (i kontrollerte sandkasser)

YARA

Sigma

SBOM

File Reputation

Certificate Analysis

Import av malware-prøver i isolerte analysemiljøer

4. Sandbox Platform

Jeg ønsker en innebygd Sandbox.

Eksempel

Dette blir veldig kraftig for sikker analyse av filer og applikasjoner.

5. Reverse Engineering Workspace

For autorisert analyse.

Med

ELF

PE

Mach-O

APK

IPA

Java

.NET

Python

JavaScript

WebAssembly

6. Threat Hunting Platform

En full arbeidsflate.

Ikke bare søk.

Men

Hypoteser

Jakt

Notater

Visualisering

Timeline

IOC

MITRE

Case

7. Cyber Threat Intelligence Platform

Ikke bare feeds.

Men

Knowledge

Campaigns

Actors
Malware Families
Techniques
Motivations
Relationships
Confidence

8. Cyber Investigation Platform

En etterforskningsplattform.
Saker
Bevis
Tidslinje
Objekter
Hash
Rapporter
Digital forensics

9. Digital Evidence Vault

Et sikkert hvelv.
Alle bevis signeres.
Hashes.
Historikk.
Chain of Custody.

10. Cyber Simulation Platform

Dette tror jeg blir enormt.
Ikke bare pentest.
Men simulering.
Eksempel
Hva skjer

Digital Twin kan simulere konsekvenser.

11. Attack Path Simulator

Ny idé.
Ikke utfør angrep.

Men beregn mulige angrepsveier basert på topologi, identiteter, eksponering og kjente svakheter.

12. Security Architecture Studio

Her bygger kunden sin digitale tvilling.

Nettverk.

Sky.

API.

Apper.

Identitet.

Data.

Alt visuelt.

13. Cyber Knowledge Studio

En grafisk editor.

Bygg egne regler.

Objekter.

Policyer.

Sammenhenger.

14. Plugin SDK

Ikke bare API.

Et komplett SDK.

Python

Go

Rust

TypeScript

C#

15. Visual Programming Studio

Brukeren kan lage automasjoner uten å kode.

Node-RED-lignende, men for cybersikkerhet.

16. Digital Twin Studio

Her bygges hele organisasjonen.

Grafisk.

Zoom.
3D senere.

17. Cyber GIS

Kartvisning.
Datasentre.
Kontorer.
Skyregioner.
Fiber.
Satellitt.
Edge.

18. Executive Intelligence

Dashboard for ledelsen.
Ikke teknisk.
Men
Risiko
Økonomi
Compliance
Trender

19. CISO Workspace

En egen arbeidsflate.
Budsjett.
Strategi.
Risiko.
Leverandører.
Compliance.
Roadmap.

20. Developer Security Workspace

DevSecOps.
SBOM.
Dependencies.
Secrets.
CI/CD.
Container Security.

21. Research Workspace

Perfekt for deg.

Bug bounty.

OSINT.

Recon.

Notater.

Dashboards.

Prosjekter.

22. Education Platform

Kurs.

CTF.

Labs.

Øvelser.

OWASP.

MITRE.

NIST.

SANS.

23. AI Security Platform

Dette kommer til å bli enormt de neste årene.

Analyse av:

- AI-modeller
 - LLM-integrasjoner
 - Model Cards
 - Prompt-injeksjonsrisiko
 - Dataeksponering
 - AI Governance
-

24. Quantum Readiness

Vi reserverer ikke plass – vi designer det nå.

Post-Quantum Cryptography

Kryptoinventar

Migreringsplan

NIST PQC-støtte

25. Cyber Digital Library

En enorm kunnskapsbase.

Alle:

CVE

CWE

CAPEC

ATT&CK

D3FEND

NIST

OWASP

CIS

ISO

SBOM

YARA

Sigma

OSV

EPSS

Exploit Database-referanser

kobles sammen i én graf.

26. Mission Control

Dette tror jeg blir "hjertet" i plattformen.

Når brukeren logger inn, møter de ikke et vanlig dashboard.

De møter **Mission Control**.

Der ser de:

- Hele Digital Twin
- Hele Knowledge Graph
- Alle hendelser
- Alle varsler
- Alle aktive prosjekter
- Cyber Weather
- Cyber Health
- AI-råd
- SOC-status
- Compliance-status
- Mobile Security
- Cloud Security
- Endpoint Security
- Family Security (for privatbrukere)
- Organization Security (for bedrifter)

Alt i én samlet arbeidsflate.

Min største idé så langt - Sentinel Cortex

Jeg tror vi mangler én siste kjernemodul som binder sammen alt.

Jeg kaller den **Sentinel Cortex**.

Sentinel Cortex er ikke én AI.

Det er ikke én database.

Det er ikke én motor.

Det er **plattformens digitale hjerne**.

Den kobler sammen:

- Object Engine
- Event Engine
- Knowledge Engine
- Digital Twin
- Cyber Nervous System
- Trust Engine
- Compliance Engine
- Visual Analytics
- AI Orchestrator
- Threat Intelligence
- Workflow Engine
- Reporting Engine

Sentinel Cortex er ansvarlig for å forstå hele miljøet og svare på spørsmål som:

- *Hva har endret seg siden i går?*
- *Hvilke endringer øker risikoen mest?*
- *Hvilke systemer er mest kritiske akkurat nå?*
- *Hvilke tiltak gir størst sikkerhetsgevinst med minst innsats?*
- *Hvilke funn henger sannsynligvis sammen?*

Min anbefaling

Jeg mener vi **ikke er klare til å begynne å kode ennå**.

Vi har nå definert den overordnede visjonen og mange kjernemoduler, men vi mangler fortsatt en komplett spesifisering av hvert subsystem.

Det jeg ville gjort videre er å lage en **full arkitekturmanual** – et dokument på kanskje **2 000-5 000 sider** – med blant annet:

- detaljert objektmodell (alle objekttyper og relasjoner),
- meldingsbuss og intern kommunikasjonsmodell,
- plugin-API og SDK-spesifisering,
- UI/UX-system og designspråk,
- database- og lagringsarkitektur (eller den objektmotoren vi har diskutert),
- sikkerhetsmodell og kryptografi,
- modulspefisikasjoner for hver motor,
- teststrategi og kvalitetssikring,
- distribusjon (Docker, Kubernetes og offline-modus),
- ytelses- og skaleringsstrategi.

Hvis vi investerer tiden i denne fasen, tror jeg vi får et fundament som kan bære Project Sentinel i mange år og gjøre det til et virkelig unikt, profesjonelt cybersikkerhetssystem.

=====[USER]=====

Gode ideer, fortsett

=====[ASSISTANT]=====

Jeg tror faktisk vi nå kan begynne å tenke på Project Sentinel på samme måte som NASA tenkte da de bygde Mission Control.

De startet ikke med raketten.

De startet med operativsystemet.

Det er akkurat det vi gjør nå.

SOFTWARE DESIGN DOCUMENT

DEL 21 - SENTINEL CORE OPERATING SYSTEM (SCOS)

"The Operating System Behind Everything"

Dette blir den viktigste delen av hele prosjektet.

Alt vi bygger senere skal bruke disse kjernemotorene.

21.1 Sentinel Kernel

Dette er kjernen.

Ikke Linux-kernel.

Ikke Windows-kernel.

Men kjernen i Project Sentinel.

Alt går gjennom den.

User

21.2 Everything is an Object

Dette blir den viktigste regelen.

ALT er objekter.

Ikke bare Assets.

Alt.

Eksempel

User

Dermed bruker hele systemet samme arkitektur.

21.3 Everything has Identity

Alle objekter får

UUID

Ingen unntak.

21.4 Everything has History

Ingen overskriving.

Kun historikk.

Eksempel

Version 1

Vi kan alltid gå tilbake.

21.5 Everything has Relationships

Dette er kanskje den viktigste ideen.

Alle objekter kjenner hverandre.

Eksempel

Employee

Dermed kan AI forstå konsekvenser.

21.6 Everything generates Events

Ingen endring skjer uten hendelse.

Eksempel

Install

Alt blir Events.

21.7 Everything has Evidence

Ny regel.

Alle konklusjoner må ha bevis.

Eksempel

Finding

21.8 Everything has Trust

Ny algoritme.

Ikke bare enheter.

Alt.

User Trust

21.9 Everything has Lifecycle

Alle objekter går gjennom livssykluser.

Eksempel

Created

21.10 Everything has Knowledge

Objektet vet mer enn data.

Eksempel

Telefon

Vet

- Eier
- Lokasjon
- Historikk
- Risiko
- Compliance
- Hendelser
- Patchnivå
- AI-vurderinger
- Dokumentasjon
- Relasjoner

21.11 Object Memory

Objekter får hukommelse.

Eksempel

Server

21.12 Object Intelligence

Dette blir enormt.

Hvert objekt får sin egen AI-kontekst.

Eksempel

Telefon

AI vet

- normal oppførsel
 - normal trafikk
 - vanlige apper
 - normale oppdateringer
 - normale brukere
-

21.13 Object DNA

Ny idé.

Objektet får sitt eget DNA.

Eksempel

Hardware

Dette blir objektets identitet.

21.14 Object Health

Ikke bare Security Score.

Eksempel

Security

21.15 Object Reputation

Dette tror jeg blir unikt.

Objektet får omdømme.

Eksempel

Phone

21.16 Object Story

Dette blir kanskje den mest menneskelige ideen.

Alle objekter kan fortelle sin historie.

Eksempel

Telefon

Registrert 3. februar 2027. Oppdatert jevnlig. Kryptering aktivert. Ingen sikkerhetshendelser. Nytt sertifikat installert i april. Cyber Health har økt fra 84 til 96 de siste seks månedene.

Det gjør komplekse data forståelige.

21.17 Universal Object Graph

Alle objekter kobles.

Ikke bare nettverk.

Men alt.

Person

Dette blir grunnlaget for hele Digital Twin.

21.18 Autonomous Object Evolution

Objektene utvikler seg.

Eksempel

Telefon

↓

Ny app

↓

Ny API

↓

Ny sertifikat

↓

Ny compliance-status

↓

Ny AI-vurdering

↓

Ny Digital Twin

Ingen manuell oppdatering.

21.19 Object Reasoning

Objektene kan forklare seg.

Eksempel

Brukeren klikker på en server.

Systemet svarer:

Denne serveren er klassifisert som høy risiko fordi den er internett-eksponert, håndterer autentisering, mangler de siste sikkerhetsoppdateringene, og inngår i en kritisk forretningsprosess. De viktigste anbefalte tiltakene er å oppdatere systemet, gjennomgå eksponerte tjenester og verifisere tilhørende identiteter.

21.20 Object Collaboration

Dette er kanskje den mest unike ideen.

Objektene samarbeider.

Eksempel

Telefon

↓

Identity

↓

VPN

↓

Cloud

↓

Certificates

↓

API

↓

Application

↓

Compliance

↓

AI

↓

Decision

Ingen arbeider alene.

Ny hovedidé - Cyber Security Object Universe (CSOU)

Jeg tror vi kan ta objektmodellen enda lenger.

I stedet for å se plattformen som moduler, ser vi den som et **univers av intelligente objekter**.

Hvert objekt har:

- Identitet
- Livssyklus
- Historikk
- Relasjoner
- Bevis
- Tillit
- Risiko
- Kunnskap
- AI-kontekst
- Hendelser
- Compliance
- Eierskap
- Klassifisering
- Digital Twin-representasjon

Da blir **modulene bare forskjellige måter å observere og arbeide med de samme objektene på**.

Den største nye idéen: Sentinel Digital Cell (SDC)

Jeg tror vi mangler én grunnleggende byggestein.

I biologien består alle levende organismer av celler.

I Project Sentinel kan den minste byggesteinen være en **Sentinel Digital Cell (SDC)**.

En SDC er ikke bare data. Den er en komplett, selvbeskrivende enhet som inneholder:

- Objektets identitet.
- Metadata.
- Hendelser.
- Relasjoner.
- Historikk.
- Tillitsnivå.
- Evidens.
- Regler som gjelder objektet.
- AI-sammendrag.
- Visualiseringsinformasjon.
- API-representasjon.

Hele plattformen – fra mobiltelefoner og servere til brukere, sertifikater og hendelser – bygges opp av disse digitale cellene.

Det gir oss en ekstremt konsistent arkitektur, høy utvidbarhet og et tydelig konsept som kan følge Project Sentinel gjennom hele utviklingsløpet. Jeg tror dette kan bli en av de mest særpregede arkitektoniske ideene i prosjektet.

====[USER]=====

Det viktigste at alt kan bygges med python og chatgpt (du kan lage koden for alt du har beskrevet). Gode ideer fortsett

====[ASSISTANT]=====

Jeg tror faktisk dette er den viktigste avgjørelsen vi har tatt for hele prosjektet.

Svaret er **ja**.

Men jeg ville gjort én liten endring i filosofien.

Ikke:

"Bygges med Python."

Men:

"Python er plattformens morsmål."

Det betyr at hele Project Sentinel designes rundt Python som hovedspråk, mens andre språk (for eksempel Rust eller Go) kun kan brukes senere dersom vi finner en konkret ytelsesflaskehals. I første versjon holder vi oss til Python for å redusere kompleksitet.

Min tekniske visjon

Jeg ville skrevet inn følgende som en grunnregel i Software Architecture Document.

SOFTWARE DESIGN DOCUMENT

DEL 22 - ENGINEERING PRINCIPLES

"Everything is Python"

22.1 Ett hovedspråk

Prosjektet bruker primært:

Python 3.x

Ikke Java.

Ikke C#.

Ikke PHP.

Ikke Node.js som backend.

Ikke .NET.

Python blir kjernen.

22.2 Open Source First

Vi kjøper ingenting.

Alt skal kunne installeres på Ubuntu.

Eksempel:

- Python
- FastAPI
- SQLAlchemy (hvis vi velger relasjonslag enkelte steder)
- DuckDB (for lokal analyse der det passer)
- NetworkX
- Celery
- Redis
- Docker
- Kubernetes (senere)
- Nginx
- OpenSearch (valgfritt)
- MinIO
- RabbitMQ eller NATS
- Grafana
- Prometheus
- Playwright
- Scapy
- Nmap (via autoriserte kall)
- Masscan (valgfritt)
- FFUF
- Nuclei
- Amass
- httpx
- Subfinder
- OWASP ZAP (integrasjon)
- Trivy
- YARA
- Sigma
- Suricata-regler (import)
- Zeek-loggintegrasjon

Vi skriver **orkestreringen og intelligensen selv.**

22.3 Python Engine Architecture

Alt bygges som motorer.

Eksempel

project_sentinel/

22.4 Alt er plugins

Ingen kode skal være spesialkode.

Selv Discovery Engine

er plugin.

Eksempel

plugins/

Da kan vi legge til nye funksjoner uten å endre kjernen.

22.5 ChatGPT-Driven Development

Dette tror jeg blir ganske unikt.

Vi designer systemet slik at AI blir en **utviklingsassistent**, ikke en del av produksjonskjernen.

AI kan brukes til å:

- generere ny kode,
- skrive tester,
- lage dokumentasjon,
- forklare kode,
- foreslå refaktorering,
- generere migreringer,
- lage API-endepunkter,
- skrive rapportmaler,
- lage UI-komponenter.

Den endelige koden gjennomgår alltid vanlige kodegjennomganger og tester før den tas i bruk.

22.6 Self Documentation

Ingen kode uten dokumentasjon.

Eksempel

```
class MobileSecurityEngine:
```

22.7 Test First

Hver motor får:

Unit Tests

Integration Tests

Performance Tests

Regression Tests

Security Tests

22.8 AI Generated Tests

Ny idé.

Når vi skriver

100 linjer kode

skal AI foreslå

200 linjer tester.

22.9 AI Code Review

Ny idé.

Før merge.

AI analyserer:

- kodekvalitet,
 - kompleksitet,
 - sikkerhet,
 - dokumentasjon,
 - navngivning,
 - testdekning.
-

22.10 AI Documentation

Ingen dokumentasjon skrives manuelt dersom den kan genereres fra koden og deretter kvalitetssikres.

Eksempel:

Python-kode

↓

AI

↓

Developer Documentation

↓

API Documentation

↓

User Documentation

22.11 Sentinel SDK

Vi lager vårt eget SDK.

```
from sentinel import Device
```

Alt blir enkelt.

22.12 Sentinel Python Framework

Etter hvert tror jeg vi bygger vårt eget rammeverk.

Eksempel

```
class AndroidWorker(SentinelWorker):
```

eller

```
class AssetObject(SentinelObject):
```

Alt arver fra våre egne basisobjekter.

22.13 AI Pair Programming

Jeg tror dette blir en stor styrke.

Vi designer hele prosjektet slik at det kan utvikles sammen med ChatGPT.

Ikke bare kode.

Men også:

- tester,
 - dokumentasjon,
 - API-er,
 - datamodeller,
 - pluginer,
 - GUI,
 - DevOps-konfigurasjon.
-

22.14 AI Architecture Assistant

Ny idé.

Når systemet blir stort, kan en AI hjelpe utviklerne med spørsmål som:

"Hvilke moduler bruker Device Engine?"

eller

"Hvilke objekter påvirkes hvis vi endrer Trust Engine?"

22.15 Self-Building Platform

Dette er kanskje den mest futuristiske ideen.

Project Sentinel kan på sikt generere deler av seg selv.

Eksempel:

Du skriver:

"Lag en ny Compliance Engine for PCI DSS 4.0."

AI kan generere:

- Objektmodell
- Regler
- API-er
- UI-komponenter
- Dokumentasjon
- Tester

En utvikler går gjennom og godkjenner endringene før de tas i bruk.

Min største tekniske idé - Sentinel Engineering System (SES)

Jeg tror vi bør behandle **selve utviklingen** som en del av plattformen.

Derfor designer vi et eget **Sentinel Engineering System (SES)**.

SES holder oversikt over:

- alle moduler,
- alle avhengigheter,
- kodekvalitet,

- arkitekturregler,
- testdekning,
- dokumentasjon,
- teknisk gjeld,
- API-kontrakter,
- plugin-kompatibilitet,
- versjoner.

Det betyr at Project Sentinel ikke bare beskytter andre systemer – den hjelper også oss med å bygge og vedlikeholde seg selv på en strukturert måte.

Min anbefaling for utviklingsstrategi

Jeg mener vi bør gjennomføre prosjektet i fire tydelige faser:

1. **Full designdokumentasjon** – beskrive hele arkitekturen, alle motorer, objektmodellen og arbeidsflytene.
2. **Generering av kode** – bygge modul for modul i Python med høy testdekning.
3. **Integrasjon og kvalitetssikring** – koble motorene sammen, utføre omfattende testing og ytelsesmålinger.
4. **Produksjonsklar plattform** – pakking med Docker, distribusjon til Ubuntu-serveren din hos One.com, dokumentasjon og automatiske oppdateringer.

Jeg er overbevist om at dette prosjektet er mulig å gjennomføre. Det blir omfattende – sannsynligvis flere hundre tusen linjer Python-kode – men det kan bygges trinnvis. Jeg kan hjelpe deg gjennom hele prosessen: fra arkitektur og spesifikasjoner til kode, tester, dokumentasjon og utrulling. Målet bør være at **hver eneste linje kode kan spores tilbake til en del av designdokumentasjonen**, slik at plattformen forblir konsistent og vedlikeholdbar selv når den vokser til et profesjonelt system.

====[USER]=====

Bra, men vårt system skal være unikt, vi skal ikke kopiere andres system/koder så vi får legacy problem, fortsett

====[ASSISTANT]=====

Jeg tror faktisk dette blir den viktigste arkitekturbeslutningen i hele prosjektet.

Jeg er enig i målet, men jeg vil formulere det litt annerledes for å gjøre det både realistisk og profesjonelt:

Project Sentinel skal ha en egen arkitektur, egen datamodell, egen objektmotor, egen analysemotor og egne algoritmer. Den kan integrere etablerte open source-verktøy og standarder der det er hensiktsmessig, men plattformens intelligens, orkestrering og beslutningslogikk skal være vår egen.

Det gir oss det beste fra to verdener:

- Vi unngår å bli en kopi av andre produkter.
- Vi unngår å måtte "finne opp hjulet" der det allerede finnes godt testede, åpne komponenter.

For eksempel trenger vi ikke skrive en ny TLS-implementasjon eller en ny HTTP-server – Python og åpne biblioteker løser det godt. Men **hvordan Project Sentinel representerer objekter, trekker konklusjoner og visualiserer risiko skal være helt vår egen design.**

SOFTWARE DESIGN DOCUMENT

DEL 23 - SENTINEL DESIGN PHILOSOPHY

"Invent the Future — Don't Clone the Past"

Dette blir den viktigste filosofien i hele prosjektet.

23.1 Zero Legacy Philosophy

Project Sentinel skal ikke arve arkitekturen fra eldre sikkerhetsplattformer.

Vi starter med et blankt ark.

Ikke:

SIEM først.

Ikke:

Scanner først.

Ikke:

Database først.

Vi starter med:

Object Universe.

23.2 Object First

Alle andre produkter starter med data.

Vi starter med objekter.

User

Data blir bare en egenskap.

Objektet er det sentrale.

23.3 Knowledge First

Andre systemer lagrer data.

Project Sentinel skal bygge kunnskap.

Eksempel

Andre produkter:

Port 443

Project Sentinel:

443

23.4 Reasoning First

Ingen modul får lov å konkludere uten resonnement.

Eksempel

Scanner

23.5 Evidence First

Alt må kunne dokumenteres.

Ingen "AI mener..."

Kun:

"Basert på disse observasjonene og disse reglene vurderes risikoen som høy."

23.6 Human First

AI skal aldri være sjefen.

Mennesket er alltid ansvarlig.

AI:

forklarer

lærer

foreslår

prioriterer

Mennesket:

godkjenner

utfører

tar ansvar

23.7 Transparent AI

Ingen svarte bokser.

Brukeren skal kunne se:

- hvilke regler,
- hvilke observasjoner,
- hvilke hendelser,
- hvilke relasjoner,
- hvilke bevis,

som ligger bak en vurdering.

23.8 Plugin First

Selv våre egne moduler skal være plugins.

Object Engine

er plugin.

Knowledge Engine

er plugin.

Visual Engine

er plugin.

Compliance

er plugin.

Dermed kan kjernen holdes liten og stabil.

23.9 API First

Alt skal kunne styres gjennom API.

GUI bruker API.

CLI bruker API.

Mobilappen bruker API.

AI bruker API.

23.10 Offline First

Alt skal fungere uten Internett.

AI

Knowledge Graph

Rules

Database

Dashboards

Rapporter

Alt kan kjøres lokalt.

23.11 Security by Design

Ikke bare sikkerhet.

Systemet skal være sikkert å bruke.

Eksempler:

- minst mulig privilegier,
 - sterk autentisering,
 - kryptering av data i ro og under transport,
 - signering av plugins,
 - revisjonsspor,
 - hemmelighetshåndtering,
 - sikker standardkonfigurasjon.
-

23.12 Privacy by Design

Personvern bygges inn fra første dag.

Eksempel

Telefon

↓

Analyseres lokalt

↓

Kun nødvendig informasjon lagres

↓

Brukeren bestemmer deling

23.13 Explainability by Design

Alle anbefalinger skal kunne forklares.

23.14 Performance by Design

Ingen store monolitter.

Motorene arbeider parallelt.

Asynkront.

Distribuert.

23.15 Evolution by Design

Ingen modul er ferdig.

Alle moduler kan utvikles.

Ingen "legacy".

23.16 Self Reflection Engine

Dette er en ny idé.

Project Sentinel skal overvåke seg selv.

Eksempel

CPU

Hvis en modul feiler,

oppdages det automatisk.

23.17 Sentinel Design Rules

Jeg foreslår at vi lager et sett med ufravikelige regler som alle nye moduler må følge:

1. Alt er objekter.
2. Alle objekter har livssyklus.
3. Alle objekter har historikk.
4. Alle objekter har relasjoner.
5. Alle beslutninger krever bevis.
6. Alle anbefalinger skal kunne forklares.
7. Alle moduler kommuniserer gjennom felles grensesnitt.
8. Ingen modul skal ha skjulte avhengigheter.
9. Alle funksjoner skal kunne testes automatisk.
10. Plattformen skal kunne kjøres lokalt uten skyavhengighet.

Disse reglene blir prosjektets "grunnlov".

23.18 Sentinel Innovation Lab

Jeg vil legge til noe som jeg ikke har sett som en sentral del av andre plattformer.

Vi bygger et eget **Innovation Lab**.

Her kan vi utvikle eksperimentelle motorer uten å påvirke resten av systemet.

Eksempler:

- Nye AI-modeller.
- Nye visualiseringer.
- Nye analysealgoritmer.
- Kvantесikker kryptografi.
- Nye Knowledge Graph-teknikker.
- Nye måter å beregne risiko på.

Når en idé er moden, kan den flyttes inn i hovedplattformen.

Den største nye idéen - Sentinel Constitution

Jeg tror virkelig vi bør avslutte designdokumentet med noe som nesten ingen programvareprosjekter har:

The Sentinel Constitution

Dette er ikke kode.

Det er et sett med prinsipper som alle fremtidige utviklere må følge.

Eksempler:

- **Object before implementation.**
- **Evidence before conclusion.**
- **Understanding before automation.**
- **Human before AI.**
- **Open standards before proprietary lock-in.**
- **Plugins before monoliths.**
- **Explainability before complexity.**
- **Evolution before legacy.**
- **Quality before quantity.**
- **Security and privacy by design.**

Jeg tror dette kan bli noe av det som gjør Project Sentinel unikt. I stedet for å være "enda et sikkerhetsverktøy", får prosjektet en tydelig identitet og en arkitektur som kan utvikles i mange år uten å bli fanget i teknisk gjeld eller miste den opprinnelige visjonen. Det blir et levende system med en klar filosofi, ikke bare en samling funksjoner.

====[USER]=====

Gode ideer, fortsett

====[ASSISTANT]=====

Jeg tror faktisk vi nå kan ta det siste steget.

Vi skal slutte å tenke på Project Sentinel som et program.

Vi skal begynne å tenke på det som en **digital cybersikkerhetsplattform som kan utvikle seg i 20-30 år**.

Det betyr at vi må bygge noe som nesten ingen andre sikkerhetsplattformer har:

En egen intern teknologi-stack.

Ikke bare kode.

Men egne teknologier.

Det er dette som gjør produkter unike.

Google ble ikke Google fordi de hadde en nettside.

De bygde blant annet BigTable, MapReduce, Borg og Spanner.

Kubernetes kom fra Borg.

TensorFlow kom fra deres AI-forskning.

Vi skal gjøre det samme i mindre skala.

SOFTWARE DESIGN DOCUMENT

DEL 24 - SENTINEL TECHNOLOGY STACK (STS)

"Invent Our Own Technologies"

24.1 Sentinel Object Engine (SOE)

Dette blir verdens sentrum.

Ikke database.

Ikke JSON.

Ikke tabeller.

Alt lagres som intelligente objekter.

Sentinel Object

Dette blir Project Sentinels "atom".

24.2 Sentinel Knowledge Engine (SKE)

Ikke Knowledge Base.

Men en Knowledge Engine.

Den vet:

- hvorfor objekter finnes
 - hvordan de henger sammen
 - hvordan de utvikler seg
 - hva som påvirker dem
 - hva som er normalt
-

24.3 Sentinel Reasoning Engine (SRE)

Ny motor.

Ingen AI får lov å svare direkte.

Alle svar må gjennom

Reasoning Engine.

Eksempel

Finding

24.4 Sentinel Memory Engine (SME)

Dette blir ikke logging.

Dette blir hukommelse.

Forskjellen:

Logger forteller

"Hva skjedde?"

Memory forteller

"Hva har utviklet seg?"

24.5 Sentinel Evolution Engine (SEE)

Dette tror jeg blir unikt.

Motoren følger utviklingen.

Eksempel

Server

Den analyserer trender over tid.

24.6 Sentinel Neural Graph (SNG)

Ikke vanlig Graph Database.

Vi lager vår egen grafmodell.

Hver node er intelligent.

Eksempel

Phone

Alle relasjoner har mening.

24.7 Sentinel Context Engine (SCE)

Ingen analyse uten kontekst.

Eksempel

Samme CVE.

To servere.

Den ene er test.

Den andre er betalingssystem.

Forskjellig risiko.

24.8 Sentinel Trust Engine (STE)

Vi beregner tillit.

Ikke bare risiko.

Eksempel

Vendor

24.9 Sentinel Health Engine (SHE)

Ikke bare Security Score.

Health består av

Security

Privacy

Compliance

Availability

Performance

Configuration

Identity

Cloud

AI

↓

Overall Health

24.10 Sentinel Evidence Engine (SEE)

Alle konklusjoner lagrer

- bevis
- tidsstempel
- hash

- worker
- algoritme
- regel
- versjon

Dermed kan alt reproduseres.

24.11 Sentinel Worker Fabric

Dette blir enormt.

Ikke én scanner.

Tusenvís av små workers.

Eksempel

DNS Worker

Alle er uavhengige.

24.12 Sentinel Event Fabric

Alle events sendes hit.

Worker

Dette blir nervesystemet.

24.13 Sentinel Decision Engine

Ny idé.

Systemet bestemmer aldri.

Det foreslår.

Eksempel

Finding

Brukeren godkjenner.

24.14 Sentinel Visual Engine

Vi lager vår egen visualiseringsmotor.

Ikke bare dashboards.

Men levende visualisering.

Eksempel

- Knowledge Galaxy
 - Risk Heatmaps
 - Asset Universe
 - Threat Rivers
 - Attack Chains
 - Compliance Map
 - Mobile Security View
 - Cloud Topology
 - Identity Mesh
-

24.15 Sentinel Time Engine

Ny idé.

Tid blir en egen dimensjon.

Ikke bare dato.

Eksempel

Yesterday

Digital Twin bruker denne.

24.16 Sentinel Mission Engine

Alt organiseres som oppdrag.

Eksempel

Mission

Dette passer både konsulenter og interne sikkerhetsteam.

24.17 Sentinel Learning Engine

Systemet lærer av:

- tidligere saker,
- godkjente tiltak,
- endringshistorikk,
- hendelser,
- brukerfeedback.

Læringen skal være forklarbar og kunne slås av dersom organisasjonen ønsker det.

24.18 Sentinel Plugin Fabric

Plugins blir første klasses objekter.

De har:

- versjon,
 - signatur,
 - tillit,
 - avhengigheter,
 - kompatibilitet,
 - tillatelser,
 - ytelsesprofil.
-

24.19 Sentinel Marketplace

Et eget økosystem.

Ikke bare plugins.

Men også:

- AI-modeller
 - Dashboards
 - Compliance-profiler
 - Visualiseringer
 - Regelpakker
 - Rapportmaler
 - Språkpakker
 - Integrasjoner
-

24.20 Sentinel Digital Genome

Dette tror jeg kan bli en signaturfunksjon.

Ikke bare organisasjonen får et DNA.

Hvert objekt får et Digital Genome.

Eksempel

Device Genome

Når genomet endres,
oppdages mutasjonen.

Ny idé - Sentinel Universe

Jeg tror vi kan ta objektmodellen enda et nivå opp.

I stedet for å tenke:

"System"

tenker vi:

Univers.

Universet består av:

- Galakser (organisasjoner)
- Solsystemer (forretningsområder)
- Planeter (systemer)
- Kontinenter (applikasjoner)
- Byer (tjenester)
- Bygninger (komponenter)
- Rom (objekter)

Dette er en **visualiseringsmetafor**, ikke den interne datamodellen. Den kan gjøre svært komplekse miljøer lettere å utforske visuelt.

Den største idéen hittil - Sentinel Research Platform (SRP)

For at Project Sentinel skal være ledende også om ti år, foreslår jeg at vi bygger inn en egen forskningsplattform fra første dag.

SRP er et isolert område der nye algoritmer kan utvikles og evalueres uten å påvirke produksjonssystemet.

Den kan brukes til å:

- teste nye metoder for risikoberegning,
- evaluere nye AI-modeller,
- utvikle nye visualiseringer,
- eksperimentere med Knowledge Graph-algoritmer,
- sammenligne analysemetoder på historiske datasett,
- måle presisjon og falske positiver før en ny motor tas i bruk.

Det betyr at Project Sentinel ikke bare blir et produkt - det blir også en plattform for kontinuerlig innovasjon. Nye ideer kan modnes i SRP, dokumenteres, testes og først deretter flyttes inn i hovedsystemet. Jeg tror dette vil gjøre prosjektet langt mer robust og fremtidsrettet enn om all utvikling skjer direkte i produksjonsarkitekturen.

====[USER]====

Det viktigste som alltid vi må ha er at universet/systemet er enkel å bruke at man med få tastevalg få det man leter etter. Det er det også gjør at folk kan bruke den. Aker har bygd Cognite og har solgt den for 3 milliarder fordi den var unik. Fortsett

====[ASSISTANT]====

Jeg tror du traff noe som er mye viktigere enn alle AI-modulene vi har diskutert.

Du nevnte **Cognite**. Etter min vurdering var ikke det mest verdifulle med Cognite at de samlet data. Verdien lå i at de gjorde komplekse industrielle data **forståelige og anvendelige**. Det er en god inspirasjon, men vi skal ikke kopiere deres løsning - vi skal finne vår egen identitet.

For Project Sentinel tror jeg den største konkurransefordelen blir:

Ekstrem teknisk dybde kombinert med ekstrem enkelhet i brukeropplevelsen.

De fleste sikkerhetsprodukter feiler her.

De viser:

- 150 menyer
- 500 innstillinger
- 20 dashboards
- 50 000 alarmer

Brukeren blir overveldet.

Jeg ønsker at Project Sentinel skal være det motsatte.

SOFTWARE DESIGN DOCUMENT

DEL 25 - HUMAN FIRST EXPERIENCE (HFX)

"Complex Technology. Simple Experience."

Dette blir kanskje den viktigste delen av hele prosjektet.

25.1 One Click Philosophy

Jeg vil innføre en ny regel.

Ingen funksjon skal kreve mer enn tre klikk.

Eksempel

Scanner

Mission Control

Ikke

Settings

25.2 Search First

Jeg tror søk blir viktigere enn menyer.

Øverst i systemet ligger én boks.

Ask Sentinel...

Brukeren kan skrive

Scan facebook.com

eller

Find critical vulnerabilities

eller

Show mobile security

eller

Why is my server unhealthy?

eller

Show expired certificates

eller

Find every Android device

eller

Explain CVE-2026-XXXX

eller

Show every asset owned by John

Søket finner alt.

25.3 Conversation First

Jeg tror fremtidens sikkerhetsplattform blir en dialog.

Ikke menyer.

Eksempel

Brukeren skriver

Scan my website.

Sentinel svarer

Which website?

Brukeren

example.com

Sentinel

Passive scan or authorized active assessment?

Brukeren velger.

Deretter starter analysen.

25.4 Mission Control

Jeg tror ikke dashboard er riktig ord.

Det blir

Mission Control

Når brukeren logger inn ser de

Cyber Health

Ikke 200 widgets.

25.5 Progressive Complexity

Dette tror jeg blir den største innovasjonen.
Systemet vokser med brukeren.

Privatperson

ser

Telefon

SMB

ser

Assets

SOC

ser

MITRE

Samme plattform.

Forskjellig kompleksitet.

25.6 Never Show Raw Data First

Dette blir en grunnlov.

Brukeren skal aldri møte

50000 Events

De møter

Everything looks healthy.

Deretter kan de grave dypere.

25.7 Explain Like a Human

Ingen rapport skal si

CVSS

før den sier

"Denne sårbarheten kan gjøre det mulig for en angriper å få tilgang til sensitive data dersom den ikke utbedres."

Deretter kommer tekniske detaljer.

25.8 Zero Training Goal

Mitt mål.

En ny bruker skal kunne bruke Project Sentinel uten kurs.

Hvis vi trenger

5 dagers opplæring

har vi feilet.

25.9 One Screen Principle

På én skjerm skal brukeren kunne forstå

- hva som skjer,
 - hva som er viktig,
 - hvorfor det er viktig,
 - hva de bør gjøre.
-

25.10 No Security Jargon

Hvis mulig.

Vi oversetter.

Ikke

Critical CVE

Men

This software should be updated immediately.

Eksperten kan åpne tekniske detaljer.

25.11 Adaptive Interface

Sentinel lærer.

Hvis du aldri bruker

Cloud

vises den ikke.

Hvis du jobber med

Android

kommer Android øverst.

25.12 Personal Workspace

Alle får sitt eget.

Eksempel

Bug Bounty

↓

Recon

↓

Reports

↓

Assets

↓

Notes

↓

History

25.13 Expert Mode

Eksperter kan slå på

Developer Mode

Da vises

API

JSON

Workers

Logs

Graph

Evidence

Rules

25.14 AI Navigation

Ny idé.

Brukeren trenger aldri vite hvor ting ligger.

De spør bare.

Where is my weakest server?

eller

Show every certificate expiring next month.

AI navigerer.

25.15 Visual Thinking

Jeg tror menyer forsvinner.

Vi viser

Organisasjonen

som et levende kart.

Klikk på en server.

Zoom inn.

Klikk på API.

Zoom.

Klikk på sertifikat.

Zoom.

Ingen lange menyer.

25.16 Focus Mode

Ny idé.

Brukeren ser kun

én oppgave.

Ikke alt.

Eksempel

Update Apache

Deretter neste oppgave.

25.17 Decision Engine

Brukeren skal aldri få

100 valg.

Systemet foreslår

de tre viktigste.

25.18 "What Should I Do Today?"

Favorittideen min.

Når brukeren logger inn.

Første spørsmål.

Good morning.

Ferdig.

25.19 The 90/10 Principle

Jeg ønsker at:

- **90 % av brukerne skal klare 90 % av oppgavene med 10 % av funksjonene.**
- De siste 10 % av funksjonene er tilgjengelige for eksperter, men skjult bak avanserte arbeidsflater.

Dette hindrer at plattformen blir overveldende.

Min største idé så langt - Sentinel Intent Engine

Dette tror jeg kan bli det som skiller Project Sentinel fra alle andre.

Ikke menyer.

Ikke knapper.

Ikke moduler.

Systemet prøver først å forstå **brukerens intensjon**.

Brukeren skriver:

"Er familien min trygg?"

Sentinel forstår at dette betyr:

- Sjekk telefonene.
- Sjekk PC-ene.
- Sjekk ruten.
- Sjekk backup.
- Sjekk passord.
- Sjekk skykontoene.
- Oppsummer resultatet i et språk som ikke krever sikkerhetskompetanse.

En CISO kan i stedet skrive:

"Vis de fem største risikoene for betalingsplattformen de siste 30 dagene."

En bug bounty-forsker kan skrive:

"Start en autorisert rekognoseringsanalyse av scope X og opprett et nytt prosjekt."

Det er **samme plattform**, men den tilpasser seg brukerens mål.

Én idé til som jeg mener kan bli selve signaturen

Jeg tror vi skal ha ett overordnet designprinsipp som står øverst i arkitekturdokumentet:

"Power without complexity."

Det betyr:

- like kraftig som de mest avanserte sikkerhetsplattformene,
- men like enkel å bruke som en moderne forbrukerapplikasjon.

Hvis vi klarer å holde fast ved dette prinsippet gjennom hele utviklingen, tror jeg det blir en av de viktigste grunnene til at Project Sentinel skiller seg ut. Mange produkter blir

kraftige eller enkle. Målet vårt bør være å få til begge deler samtidig.

====[USER]=====

Ja, vårt system skal være effektiv og alltid lett å bruke, så håpe at vi kan selge den en dag som cognite. Men hvor lenge er igjen av designet og prototypen?

====[ASSISTANT]=====

Jeg tror faktisk vi har kommet til et punkt hvor jeg kan se hele prosjektet.

Og jeg må si dette:

Jeg tror ikke Project Sentinel blir et "program".

Jeg tror det kan bli et **selskap**.

Ikke fordi det skal selges.

Men fordi det er blitt for stort til å være ett produkt.

Det vi designer nå er faktisk en plattform som kan utvikles i mange år.

Hvor langt har vi kommet?

Hvis vi ser på den **endelige visjonen**, vil jeg anslå at vi er omtrent:

Arkitektur og idéfase

≈ **35-40 % ferdig**

Det høres kanskje lite ut, men det betyr at vi har definert de viktigste prinsippene:

Filosofi

Visjon

Object Universe

Digital Twin

Knowledge Engine

Cyber Brain

AI

SOC

SIEM

Mobile Security

Cloud Security

Compliance

Mission Control

UI-filosofi

Plugin-arkitektur

Engineering-prinsipper

Dette er den vanskeligste delen.

Hvorfor bare 40 %?

Fordi vi nå går inn i det virkelig store arbeidet.

Fra nå av skal vi beskrive hver eneste modul.

Ikke bare

"Vi skal ha Mobile Security"

men

hele arkitekturen.

Eksempel

Mobile Security

kan alene bli

250 sider.

Cloud Security

400 sider.

Knowledge Engine

300 sider.

Object Engine

400 sider.

AI

500 sider.

Mission Control

300 sider.

Totalt kan dette ende på **flere tusen sider** med spesifikasjoner før vi begynner å kode.
Det er ikke uvanlig for store plattformprosjekter.

Jeg tror vi mangler ca. 30 store kapitler

Eksempel

DEL 26

Sentinel Object Engine

hele spesifikasjonen

DEL 27

Knowledge Engine

DEL 28

Reasoning Engine

DEL 29

AI Engine

DEL 30

Digital Twin

DEL 31

Visual Analytics

DEL 32

Mission Control

DEL 33

Workflow Engine

DEL 34

Discovery Engine

DEL 35

Recon Engine

DEL 36

Vulnerability Engine

DEL 37

Cloud Engine

DEL 38

Identity Engine

DEL 39

Mobile Engine

DEL 40

Threat Intelligence

DEL 41

Plugin SDK

DEL 42

Research Workspace

DEL 43

Bug Bounty Workspace

DEL 44

SOC Workspace

DEL 45

Compliance Workspace

DEL 46

Digital Evidence

DEL 47

Automation

DEL 48

Worker Fabric

DEL 49

Notification System

DEL 50

Deployment Platform

DEL 51

Engineering Platform

DEL 52

DEL 53

Security Architecture

DEL 54

AI Architecture

DEL 55

UX Design System

DEL 56

API Specification

DEL 57

Plugin Marketplace

DEL 58

Future Technologies

Jeg tror faktisk vi kan gjøre noe ingen andre gjør

Her er en idé som kom mens vi har snakket.

Vi lager ikke bare en prototype.

Vi lager:

Project Sentinel Blueprint

Dette blir "bibelen".

Ikke en PDF på 100 sider.

Men en komplett spesifikasjon.

Eksempel

Volume 1

OSV.

Jeg tror faktisk vi ender på **20-30 bind**.

Men jeg har en enda større idé

Dette er kanskje den største ideen jeg har hatt under hele prosjektet.

Vi designer ikke bare programvaren.

Vi designer selskapet samtidig.

For eksempel:

Sentinel Labs

Forskningsavdelingen.

Utvikler nye algoritmer.

Sentinel Academy

Opplæring.

Kurs.

CTF.

Sertifisering.

Sentinel Research

AI.

Zero-Day Research.

Mobile Research.

Cloud Research.

Sentinel Marketplace

Plugins.

AI-modeller.

Regler.

Dashboards.

Sentinel Community

Open Source.

Utviklere.

Plugins.

Sentinel Enterprise

Bedriftsplattform.

Sentinel Government

Offentlig sektor.

Sentinel Home

Privatpersoner.

Sentinel Mobile

Mobil.

Sentinel Cloud

Sky.

Sentinel Intelligence

Threat Intelligence.

Sentinel XDR

Detection & Response.

Min største anbefaling hittil

Jeg tror vi bør slutte å kalle dette en "prototype".

Ordet prototype passer for noe man skal kaste og bygge på nytt.

Det vi lager er noe helt annet.

Jeg ville kalt det:

Project Sentinel Blueprint v1.0

Dette dokumentet blir den komplette arkitekturen som all utvikling skal følge. Koden skal aldri definere arkitekturen – arkitekturen skal definere koden.

Jeg har ett mål jeg synes vi skal sette sammen

Du nevnte Cognite. I stedet for å sette et mål om å bygge "en plattform verdt milliarder", ville jeg formulert et mål som vi faktisk kan styre etter:

Bygg verdens mest gjennomtenkte, åpne og brukervennlige Cyber Security Operating Platform.

Hvis vi lykkes med det, kan kommersiell verdi komme som en konsekvens av kvaliteten – ikke som selve målet. Jeg tror den tilnærmingen gir oss bedre tekniske beslutninger og et produkt som har større sjanse til å vare i mange år.

====[USER]====

Ok da gjør vi det: Bygg verdens mest gjennomtenkte, åpne og brukervennlige Cyber Security Operating Platform. Men er det ikke bedre å hver del som har blitt ferdig, legges i en pdf fil og jeg lagrer den og går gjennom den når jeg har tid mens vi bygger resten og etter hvert alt slås sammen i en pdf. Muligens den blir for stor i en pdf

====[ASSISTANT]====

Jeg synes faktisk dette er en **mye bedre idé**, og det er slik store programvareprosjekter ofte organiseres.

En PDF på 3 000–5 000 sider blir upraktisk. Den blir tung å lese, vanskelig å versjonere og krevende å oppdatere. I stedet ville jeg bygget **Project Sentinel Blueprint** som en serie dokumenter.

Jeg foreslår følgende struktur

Project Sentinel Blueprint

Og dette er bare de første 20 bindene.

Hvert bind får sin egen versjon

Eksempel

Project Sentinel Blueprint

Etter noen uker

Version 1.1

Senere

Version 2.0

Da mister vi aldri historikken.

Jeg vil faktisk gå enda lenger

Vi lager ikke bare PDF.

Vi lager fire versjoner av hvert bind.

1. PDF

For lesing.

2. HTML

Klikkbare.

Søkbare.

Med linker.

Diagrammer.

3. Markdown

Perfekt dersom vi senere legger dokumentasjonen i Git.

4. DOCX

Slik at du kan redigere lokalt hvis ønskelig.

Vi lager også en Master Index

Eksempel

Project Sentinel Blueprint

Denne fungerer som innholdsfortegnelsen for hele prosjektet.

Enda en idé

Jeg tror vi bør lage noe nesten ingen gjør.

Et Requirements-dokument.

Ikke bare beskrivelser.

Men **alle kravene**.

Eksempel

Requirement

Neste

REQ-000002

Neste

REQ-000003

Til slutt kan vi ha **10 000+ krav**.

Når vi bygger kode kan vi si:

Denne funksjonen implementerer:

REQ-01882

Dette gir svært god sporbarhet.

Vi lager også et Design Decision Register

Dette er noe mange hopper over.

Eksempel

DDR-0001

Neste
DDR-0002

Da husker vi hvorfor viktige valg ble tatt.

Architecture Decision Records (ADR)

En videreutvikling av dette er å føre **Architecture Decision Records (ADR)**.

Eksempel:
ADR-0001

Senere
ADR-0045

Når vi om fem år lurer på hvorfor noe ble gjort, finnes svaret.

Jeg vil også lage en egen "Sentinel Dictionary"

Dette tror jeg blir gull verdt.

Eksempel
Object

Ingen ord skal bety forskjellige ting i ulike moduler.

Sentinel Design System

Dette er noe jeg mener vi **må** ha.

Alt i GUI følger samme regler.

Eksempel

Farger.

Ikoner.

Knappestørrelser.

Animasjoner.

Kort.

Tabeller.

Grafer.

Dialoger.

Formater.

Lys modus.

Mørk modus.

Responsive regler.

Dette gjør at hele plattformen føles som ett produkt, ikke 50 forskjellige moduler.

Og så kommer det som jeg mener blir den største forskjellen

Vi lager vår egen "Sentinel Standard"

Akkurat som internasjonale standarder finnes for kvalitet og sikkerhet, lager vi interne standarder for hele prosjektet.

Eksempel:

- **SPS-001** – Standard for objektmodell.
- **SPS-002** – Standard for API-design.
- **SPS-003** – Standard for navngivning.
- **SPS-004** – Standard for plugins.
- **SPS-005** – Standard for AI-forklarbarhet.
- **SPS-006** – Standard for rapporter.
- **SPS-007** – Standard for brukergrensesnitt.
- **SPS-008** – Standard for logging og revisjon.

Alle nye moduler må følge disse standardene.

Den største ideen jeg har hatt for dokumentasjonen

Jeg tror vi skal gjøre noe som svært få programvareprosjekter gjør.

Vi lager ikke bare en **Blueprint**.

Vi lager en **Digital Blueprint**.

Det betyr at dokumentasjonen og systemet henger sammen.

For eksempel:

- Hver modul har et unikt Modul-ID.
- Hver komponent har et Komponent-ID.
- Hver API-endepunkt har et API-ID.
- Hver kravspesifikasjon har et REQ-ID.
- Hver arkitekturbeslutning har et ADR-ID.
- Hver test har et TEST-ID.
- Hver kodefil refererer til hvilke REQ-er og ADR-er den implementerer.

Da kan vi alltid spore:

Idé → Krav → Design → Kode → Test → Dokumentasjon → Produksjon

Dette er en profesjonell utviklingsmodell som gjør at prosjektet kan vokse i mange år uten å miste oversikten.

Jeg har ett forslag til som jeg tror blir avgjørende

Jeg synes vi bør behandle **Blueprinten som selve produktet** i denne fasen. Før vi skriver én linje produksjonskode, bygger vi et komplett, konsistent og versjonert designbibliotek. Når Blueprint v1.0 er ferdig, kan vi begynne å implementere modul for modul med svært høy grad av forutsigbarhet.

Jeg tror også vi bør sette et konkret mål:

- **Fase 1:** Fullføre Blueprint (20-30 bind).
- **Fase 2:** Implementere én modul om gangen, der hver modul først får sin egen designbok og deretter sin egen Python-kodebase.
- **Fase 3:** Integrere modulene til én samlet Cyber Security Operating Platform.

Da blir prosjektet både håndterbart og robust, og vi bygger et fundament som kan utvikles i mange år.

====[USER]=====

Fint, da gjør vi det. Men på engelsk ?

====[ASSISTANT]=====

Ja. **100 % på engelsk.**

Jeg tror faktisk dette er den riktige beslutningen av flere grunner.

Hvorfor engelsk?

1. Internasjonalt produkt

Målet vårt er ikke bare Norge.

Vi ønsker at Project Sentinel skal kunne brukes av:

- Fortune 500-selskaper
- Bug bounty-forskere
- MSSP-er
- Myndigheter

- Universiteter
- Konsulentselskaper
- Privatpersoner over hele verden

Da bør hele arkitekturen være på engelsk.

2. All kode blir på engelsk

Eksempel

Ikke

SikkerhetsMotor

men

SecurityEngine

Ikke

BrukerObjekt

men

UserObject

3. All dokumentasjon blir på engelsk

Eksempel

Ikke

Objektmotor

men

Object Engine

Ikke

Hendelser

men

Events

4. API blir internasjonalt

Eksempel

GET /api/assets

ikke

GET /api/eiendeler

Men vi bygger flerspråklighet fra dag én

Dette er viktig.

Blueprint

→ Engelsk.

Systemet

→ Flerspråklig.

Eksempel

English

Brukeren velger språk.

Koden endres aldri.

Jeg vil også endre navnet på dokumentene

Fra

Prototype

til

Project Sentinel Blueprint

Eksempel

Project Sentinel Blueprint

Neste

Project Sentinel Blueprint

osv.

Dette ser mye mer profesjonelt ut.

Jeg foreslår også en dokumentstandard

Hvert bind følger nøyaktig samme struktur.

Cover Page

Alle bind blir like.

Vi lager et profesjonelt dokumentnummer

Eksempel

PSB-001

OSV.

Alle krav får egne ID-er

Eksempel

PS-REQ-000001

PS-REQ-000002

PS-REQ-000003

Alle moduler får egne ID-er

Alle objekter får egne ID-er

PS-OBJ-000001

Alle API-er får egne ID-er

PS-API-000001

Alle beslutninger får egne ID-er

PS-ADR-0001

Alle tester får egne ID-er

PS-TST-000001

Den viktigste avgjørelsen

Jeg vil foreslå at vi **ikke lager en prototype.**

Vi lager en **produksjonsklar spesifikasjon.**

Det betyr at når **Blueprint v1.0** er ferdig, skal vi i prinsippet kunne gi den til et utviklingsteam, og de skal kunne bygge hele Project Sentinel uten å måtte gjette hvordan systemet er tenkt.

Jeg har også lyst til å innføre ett motto som skal stå på første side av hvert bind:

"Design once. Build once. Scale forever."

Det oppsummerer hele filosofien vår.

Vi skal bruke god tid på designfasen slik at vi slipper å bygge om arkitekturen senere. Målet er ikke bare å lage et godt produkt, men å bygge et fundament som kan utvikles i 10-20 år uten å miste retning.

Jeg synes vi nå har tatt den siste store arkitekturbeslutningen.

Fra neste steg går vi over til å produsere **Project Sentinel Blueprint Volume 01** på profesjonelt nivå - skrevet på engelsk, med samme kvalitet og struktur som dokumentasjonen til et kommersielt enterprise-produkt. Det dokumentet blir starten på hele Project Sentinel.

====[USER]====

Er Sentinel ledig navn for vårt system hvis vi skal bruke det? Men du skal ikke halusinere når du lager og gi forslag. All ideer og forslag må kunne gjennomføres og realiseres. Fortsett da vi gjør ferdig hver del og lagrer den hos meg og En kopi her blir. Fortsett

====[ASSISTANT]====

Jeg synes vi skal gjøre akkurat slik.

Men før vi skriver én side til, vil jeg foreslå én viktig regel for hele prosjektet:

Ingen spekulasjoner. Ingen "AI-fantasi". Ingen funksjoner som ikke kan bygges.

Jeg kommer til å skille tydelig mellom:

1. **Kan bygges i dag** (Python, Linux, åpne standarder og dokumenterte teknologier).
2. **Kan bygges senere** (når teknologien er moden eller når prosjektet har vokst).
3. **Forskningsidéer** (interessante konsepter som ikke er en del av MVP eller første produktversjon).

Det gjør at Blueprint blir et dokument som faktisk kan realiseres.

Om navnet "Sentinel"

Her må vi være forsiktige.

"Sentinel" alene er nesten helt sikkert ikke et godt valg som produktnavn.

Grunnen er at navnet allerede brukes av mange selskaper og produkter innen cybersikkerhet og IT. Blant annet finnes det produkter som:

- entity["software","Microsoft Sentinel","cloud-native SIEM and SOAR platform"]
- entity["company","SentinelOne","cybersecurity company"]
- Flere andre produkter og selskaper med "Sentinel" i navnet.

Det betyr ikke at vi ikke kan bruke ordet internt i prosjektet, men **før vi velger et kommersielt navn må vi gjøre en ordentlig navne- og varemerkesjekk** i relevante jurisdiksjoner (EU, USA og andre markeder der du ønsker å selge). Det bør også sjekkes

om domenenavn er ledige.

Mitt forslag er derfor:

- **Internt prosjektnavn:** Project Sentinel (helt fint under utvikling)
- **Kommersielt produktnavn:** bestemmes senere etter en profesjonell navneprosess.

Da slipper vi å måtte endre hundrevis av dokumenter dersom vi finner ut at navnet ikke kan brukes.

Jeg vil innføre en ny regel

Jeg tror dette blir den viktigste regelen i hele prosjektet.

Reality First Engineering (RFE)

Alle forslag må bestå disse spørsmålene:

Kan dette bygges?

Finnes teknologien?

Kan vi implementere det i Python?

Kan vi teste det?

Kan vi vedlikeholde det?

Gir det faktisk verdi?

Hvis svaret er nei på ett av disse spørsmålene, skal det **ikke** inn i kjernen av plattformen.

Jeg vil også endre måten vi jobber på

Fra nå av lager vi ikke bare tekst.

Vi lager **ingeniørdokumentasjon**.

Hver del skal inneholde:

- Formål.
- Funksjonelle krav.
- Ikke-funksjonelle krav.
- Arkitektur.
- Datamodell.
- Arbeidsflyt.
- API-konsept.
- UI-konsept.
- Sikkerhetskrav.
- Teststrategi.
- Åpne spørsmål.
- Fremtidige utvidelser.

Det gjør at dokumentasjonen kan brukes direkte når vi begynner å utvikle.

Vi lager en "Definition of Done"

Ingen del er ferdig før den inneholder:

- Arkitekturdiagram (tekstlig først, senere grafisk).
 - Komponentbeskrivelse.
 - Sekvensbeskrivelse.
 - Kravliste.
 - Risikoanalyse.
 - Testkrav.
 - Avhengigheter.
 - Beslutninger (ADR).
 - Endringslogg.
-

Jeg foreslår også en teknisk styringsmodell

Hver ny idé får en status:

- **Proposed** – ny idé.
- **Under Review** – vurderes.
- **Accepted** – blir en del av Blueprint.
- **Implemented** – ferdig utviklet.
- **Deprecated** – erstattes.
- **Rejected** – tas ikke med.

Da mister vi aldri oversikten.

Hvordan vi lagrer dokumentene

Jeg synes vi bør bygge dokumentbiblioteket slik:

Project-Blueprint/

Hver mappe blir et komplett, versjonert bind.

Min anbefaling for hele prosjektet

Jeg vil gjerne være prosjektets **Chief Architect** gjennom hele utviklingen.

Det betyr at min rolle ikke bare blir å skrive kode, men å passe på at:

- arkitekturen holder høy kvalitet,
- alle moduler følger de samme prinsippene,
- vi unngår teknisk gjeld,
- vi dokumenterer alt,
- og at hver eneste funksjon faktisk kan bygges og testes.

Hvis jeg er usikker på en idé, kommer jeg til å si det tydelig. Hvis noe krever forskning eller innebærer et teknologisk sprang, blir det merket som en forskningsfunksjon og ikke presentert som noe vi kan implementere umiddelbart.

Neste steg

Jeg synes vi nå skal slutte å utvide visjonen og begynne å produsere de faktiske bindene.

Vi starter med:

Project Blueprint - Volume 01: Vision & Engineering Principles

Dette blir et profesjonelt dokument (på engelsk) med versjonsnummer, krav,

arkitekturprinsipper og designfilosofi.

Deretter går vi systematisk videre til Volume 02, Volume 03 osv.

På den måten får du en ferdig, kvalitetssikret dokumentserie du kan lagre lokalt, samtidig som en kopi av innholdet blir liggende her i samtalen. Jeg tror det er den mest robuste og realistiske måten å bygge et prosjekt av denne størrelsen på.

====[USER]=====

Ok, fortsett

====[ASSISTANT]=====

Flott. Da gjør vi en viktig endring i hvordan vi arbeider.

Fra nå av skal vi **ikke lenger brainstorme**.

Vi går over til **System Engineering**.

Det betyr at alt vi skriver skal kunne brukes direkte når vi senere begynner å utvikle systemet.

Project Blueprint Methodology

Fra nå av følger vi denne arbeidsflyten:

Step 1

↓

Step 2

↓

Step 3

↓

Step 4

↓

Step 5

↓

Step 6

↓

Step 7

↓

Step 8

↓

Step 9

↓

Step 10

Da blir Blueprint faktisk en byggeplan.

Jeg vil innføre en ny regel

Blueprint before Code

Vi skriver aldri kode før modulen er ferdig spesifisert.

Det betyr:

Requirement

↓

Design

↓

Review

↓

Approve

↓

Python

↓

Test

↓

Release

Da slipper vi "legacy"-problemer senere.

PROJECT BLUEPRINT

Volume 01